

```
| iMac | Apple |
+-----+-----+
7 rows in set (0.00 sec)
```

17.5 Updating view-based tables

In some cases, a view can be used to update the tables it references, allowing operations such as INSERT, UPDATE, and DELETE to be performed. To demonstrate this, use the following SQL statement to create a view named `simpleview` that lists companies based in California from the `supplier` table:

```
CREATE VIEW simpleview
AS SELECT company, address FROM supplier
WHERE address LIKE '%CA';

SELECT * FROM simpleview;
+-----+-----+
| company | address |
+-----+-----+
| Apple   | Cupertino, CA |
| NZXT    | City of Industry, CA |
| Corsair | Milpitas, CA |
| Nvidia  | Santa Clara, CA |
+-----+-----+
5 rows in set (0.01 sec)
```

The simplicity of `simpleview` allows us to update the underlying `supplier` table without restrictions. For instance, we can use the following INSERT statement to add a supplier:

```
INSERT INTO simpleview VALUES ('Intel', 'Palo Alto, CA');
```

If we check the product table, we can see that the new supplier was added via the view:

```
SELECT * FROM simpleview;
+-----+-----+
| company | address |
+-----+-----+
| Apple   | Cupertino, CA |
| NZXT    | City of Industry, CA |
| Corsair | Milpitas, CA |
| Nvidia  | Santa Clara, CA |
| Intel   | Palo Alto, CA |
+-----+-----+
5 rows in set (0.00 sec)
```

Update operations can also be performed using `simpleview`, for example:

```
UPDATE simpleview
SET address='Santa Clara, CA'
WHERE company='Intel';
```

We can also refer to the view when deleting rows from the underlying table:

```
DELETE FROM simpleview WHERE company='Intel';
```

The results can sometimes be confusing when using a view to update an underlying table. For example, use the following INSERT statement to add another supplier row:

An Introduction to MySQL Views

```
INSERT INTO simpleview VALUES ('Silicon Labs', 'Austin, TX');
```

Next, confirm that the 'Silicon Labs' row appears in the supplier table:

```
SELECT * FROM supplier;
+----+-----+-----+
| id | company      | address          |
+----+-----+-----+
| 1  | Apple        | Cupertino, CA    |
| 2  | Dell         | Round Rock, TX   |
| 3  | NZXT         | City of Industry, CA |
| 4  | Corsair      | Milpitas, CA     |
| 5  | Nvidia       | Santa Clara, CA  |
| 7  | Silicon Labs | Austin, TX    |
+----+-----+-----+
6 rows in set (0.00 sec)
```

The unexpected behavior occurs when we try to remove Silicon Labs from the supplier table using the DELETE statement on simpleview:

```
DELETE FROM simpleview WHERE company='Silicon Labs';
```

In theory, Silicon Labs should no longer exist in the supplier table. When we look, however, we find that not to be the case:

```
SELECT * FROM supplier;
+----+-----+-----+
| id | company      | address          |
+----+-----+-----+
| 1  | Apple        | Cupertino, CA    |
| 2  | Dell         | Round Rock, TX   |
| 3  | NZXT         | City of Industry, CA |
| 4  | Corsair      | Milpitas, CA     |
| 5  | Nvidia       | Santa Clara, CA  |
| 7  | Silicon Labs | Austin, TX    |
+----+-----+-----+
6 rows in set (0.00 sec)
```

The issue is that Silicon Labs is located in Texas, while the records from simpleview only include companies based in California. When we try to delete the Silicon Labs row, the DELETE operation searches the company column of the simpleview result set, finds no matching rows, and concludes there is nothing to delete. This leaves the Silicon Labs row intact in the supplier table.

Essentially, inserting rows into a view does not guarantee they will appear in the result set. Instead, we must execute the deletion directly on the supplier table to remove Silicon Labs:

```
DELETE FROM supplier WHERE company='Silicon Labs';
```

One limitation of view-based table updates is that we can only reference columns that exist within the context of the view. For example, consider our desktop_computers view, which only includes the product.name column. When we attempt to add a new row, we can only provide a value for the product name. We will encounter an error if we try to reference other columns, such as the description field:

```
INSERT INTO desktop_computers (name, description)
VALUES ('Mac Mini M4', 'Apple desktop');
ERROR 1054 (42S22): Unknown column 'description' in 'field list'
```

We can, however, add the row if we only provide the name value:

```
INSERT INTO desktop_computers VALUES ('Mac Mini M4');
```

If we inspect the new row, we see that the other fields, except the auto-incremented id have been set to NULL:

```
SELECT * FROM product WHERE name='Mac Mini M4';
+----+-----+-----+-----+-----+-----+
| id | name       | description | price | quantity | supplier_id |
+----+-----+-----+-----+-----+-----+
| 22 | Mac Mini M4 | NULL       | NULL | NULL     | NULL        |
+----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

This type of update is only possible if the table columns outside the view have been assigned a default value or can store null values. To demonstrate this requirement, delete the 'Mac Mini M4' row and modify the product table to disallow null values in the description column:

```
DELETE from product WHERE name='Mac Mini M4';

ALTER TABLE product MODIFY description varchar(50) NOT NULL;
```

Next, attempt to re-insert the 'Mac Mini M4' product row:

```
INSERT INTO desktop_computers VALUES ('Mac Mini M4');
ERROR 1423 (HY000): Field of view 'sampledb.desktop_computers' underlying table
doesn't have a default value
```

Inserting the row is no longer possible because the description field cannot store null, and no default value has been configured.

MySQL also does not support view-based updates for columns derived from aggregate functions. We cannot, therefore, update our supplier table using the supplier_summary view because the CONCAT() function was used within the original CREATE VIEW statement:

```
INSERT INTO supplier_summary VALUES ('HP', 'Palo Alto, CA');
ERROR 1348 (HY000): Column 'CONCAT(UPPER(company), ' - ', address)' is not
updatable
```

Before continuing, revert the description column to a nullable state:

```
ALTER TABLE product MODIFY description varchar(50) NULL;
```

17.6 Updating join-based views

Views based on joins can also be updated, though only inner joins are supported by MySQL. For example, we can change the product name field for the iMac row via the join_demo view as follows:

```
UPDATE join_demo SET name='iMac M2' WHERE name='iMac';
```

Rows can also be inserted into views based on joins, but the same restrictions that apply to other view updates must be followed. Specifically, only the columns in the view can be referenced, and all other columns must either have default values or be nullable. Additionally, only one underlying table can be updated at a time when working with a join view. For instance, if we want to insert rows into the product and supplier tables using the join_demo view, we need to use separate INSERT statements for each table:

```
UPDATE join_demo SET name='iMac' WHERE name='iMac M2';
```

```
INSERT INTO join_demo (name) VALUES ('iMac Classic');  
  
INSERT INTO join_demo (company) VALUES ('Cisco');
```

Note that we must specify the column names when inserting values. For this reason, a view cannot contain duplicate column names.

Execute the following statements to revert the product and supplier tables to their original state:

```
DELETE FROM product WHERE name = 'iMac Classic';  
  
DELETE FROM supplier WHERE company = 'Cisco';
```

17.7 Altering existing views

The structure of an existing view can be changed by providing a new SELECT query to the ALTER TABLE statement. For example, the statement below expands the scope of our desktop_computers view to include the product description column:

```
ALTER VIEW desktop_computers  
AS SELECT name, description FROM product WHERE  
description LIKE '%desktop%' ORDER BY name;  
SELECT * FROM desktop_computers;
```

```
+-----+-----+  
| name           | description |  
+-----+-----+  
| Dell Model 4823 | Dell desktop |  
| Dell XPS Model 7823 | Dell desktop |  
| iMac           | Apple desktop |  
| Mac Mini M3    | Apple desktop |  
| Mac Studio     | Apple desktop |  
+-----+-----+  
5 rows in set (0.00 sec)
```

17.8 Deleting and replacing views

To remove an existing view from a database, use the DROP VIEW statement. Execute the following statements to delete the views created in this chapter:

```
DROP VIEW simpleview;  
DROP VIEW supplier_summary;  
DROP VIEW desktop_computers;
```

Alternatively, a view may be replaced using either the CREATE OR REPLACE VIEW statement combined with a SELECT query:

```
CREATE OR REPLACE VIEW <view_name> AS SELECT <conditions>;
```

The above statement will either replace or create the specified view depending on whether the view already exists.

17.9 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL views:

<https://www.answerstopia.com/pmqi>



17.10 Reference points

The main points covered in this chapter are as follows:

- **Definition of Views**
 - A view is a virtual table created by encapsulating a `SELECT` query.
 - Views enable reusable queries, making it easier to reference complex queries as if they were tables.
- **Creating Views**
 - Use the `CREATE VIEW` statement with a `SELECT` query.
 - Views can be created to represent a specific subset or transformation of data from one or more tables.
 - Views can include operations such as filtering, joining, and formatting data.
- **Querying Views**
 - Treat views like regular tables in `SELECT` statements.
 - Views behave like regular tables and can be queried to retrieve data.
 - Views update dynamically, reflecting changes in the underlying tables.
- **Using Views with Joins**
 - Views can be created based on join operations to combine data from multiple tables.
- **Updating Tables via Views**
 - Updates (`INSERT`, `UPDATE`, `DELETE`) can be performed on views under certain conditions:
 - The view must reference updatable columns.
 - Columns outside the view must have default values or allow `NULL`s.
 - Aggregate or derived columns are not updatable.
- **Limitations of Updates via Views**
 - Cannot modify aggregate or computed columns.
 - Updates on views with restrictive filters may behave unexpectedly.
 - Views based on joins can only update one table at a time.
- **Altering Views**
 - Modify an existing view using `ALTER VIEW`.

- Views can be modified to change their structure or logic without altering the underlying tables
- If a view already exists, it can be replaced with a new definition.
- **Replacing Views**
 - Use `CREATE OR REPLACE VIEW` to update an existing view or create a new one if it doesn't exist:
- **Deleting Views**
 - Remove a view using the `DROP VIEW` statement:

18. Understanding MySQL Indexes

A lot happens behind the scenes of the MySQL server to ensure that data retrieval requests are handled quickly and efficiently. Maintaining this high-performance level becomes increasingly critical as databases grow in size and complexity. Simply put, the more data that needs to be searched, the longer it can take MySQL to find what it is looking for. In this chapter, we will introduce MySQL indexes and explain how they are used to maintain and improve database performance. We will also look at the limitations of indexes and explain why indexes can sometimes be too much of a good thing.

18.1 Understanding MySQL indexes

Much like a book index helps you find specific content without reading every page, database indexes allow MySQL to quickly fulfill retrieval requests without searching an entire table for matching rows. In addition to improving database performance, indexes control whether a column can contain duplicate values.

Indexes can be declared when a table is created or added to existing tables, and each index entry comprises the values of one or more columns within the indexed table.

MySQL supports several index types, each of which will be explained before we look at some examples.

18.2 Primary index

We have already been using primary indexes in many of the book examples so far. Primary indexes are automatically created when a PRIMARY KEY constraint is declared and ensure that indexed column values are unique and non-null. As discussed in the “*Creating Databases and Tables*” chapter, a primary key can be based on a single column or a combination of table columns, for example:

```
CREATE TABLE customer
(
  id INT NOT NULL AUTO_INCREMENT,
  name char(30) NOT NULL,
  address VARCHAR(60) NULL,
  email VARCHAR(25) NOT NULL,
  PRIMARY KEY (id, name)
);
```

18.3 Unique index

Like primary indexes, unique indexes ensure that the values in indexed columns are unique. Unique indexes differ, however, in that the indexed columns can store null values. The syntax to declare a unique index when creating a table is as follows:

```
CREATE TABLE <table_name>
(
  <column_1> <type>,
  <column_2> <type>,
  .
  .
  .
  UNIQUE INDEX <index_name> (<column_name>),
  .
  .
  .
```

```
.  
);
```

Use the CREATE UNIQUE INDEX statement with the following syntax to add a unique index to a table:

```
CREATE UNIQUE INDEX <index_name> ON <table_name>(<column_name>);
```

The following statement, for example, adds a unique index to the address column of the customer table:

```
CREATE UNIQUE INDEX idx_address ON customer(address);
```

Once the index has been created, only rows with unique or null address values can be added to the table.

18.4 Regular index

A regular index (also known as a secondary or non-unique index) improves the performance of queries involving the indexed column but does not enforce the uniqueness of the column's values. The syntax for adding a regular index during table creation is as follows:

```
CREATE TABLE <table_name>  
(  
    <column_1> <type>,  
    <column_2> <type>,  
    .  
    .  
    INDEX <index_name> (<column_name>),  
    .  
    .  
);
```

Regular indexes are added to existing tables using the CREATE INDEX statement as follows:

```
CREATE INDEX <index_name> ON <table_name>(<column_name>);
```

18.5 Full-text indexes

A full-text index is used for improved text searching performance and can be included at table creation using the following syntax:

```
CREATE FULLTEXT INDEX <index_name> ON <table_name>(<column_name>);
```

Full-text indexes are optimized for operations like searching for words or phrases in text-heavy fields using the MATCH and AGAINST keywords, for example:

```
SELECT <column_1>, <column_2>  
FROM <table_name>  
WHERE MATCH(<column_name>[, ...]) AGAINST ('<search_string>' [<search_mode>]);
```

The AGAINST keyword is used exclusively for searching in columns with a full-text index and can search for words, phrases, or patterns in natural language or boolean mode. Searches performed using AGAINST return a relevance score for each row ranked on how well the content matches the search text and the frequency with which matches occurred. The following is an example of a natural language search using the AGAINST keyword:

```
SELECT *  
FROM manuscript  
WHERE MATCH(subtitle, body_text) AGAINST ('quantum coherence');
```

Boolean mode searches can include logical operators such as +, -, <, and > to fine-tune the search criteria, such as excluding matches that contain certain words. The following search uses AGAINST in boolean mode to include the rows containing the term “Linux” in the results but excludes those containing “macOS”:

```
SELECT *
FROM manuscript
WHERE MATCH(subtitle, body_text) AGAINST ('+Linux -macOS' IN BOOLEAN MODE);
```

Queries can also be extended to include terms related to the search pattern using the WITH QUERY EXPANSION clause, for example:

```
SELECT *
FROM manuscript
WHERE MATCH(subtitle, body_text)
AGAINST ('quantum computing' WITH QUERY EXPANSION);
```

Full-text indexes are intended for text-heavy fields and should be avoided for short or repetitive text-based data.

18.6 Composite index

Composite indexes comprise two or more columns and improve performance for queries involving multiple columns. All index types covered so far in this chapter support composite columns. The following syntax is used when creating a composite index during table creation:

```
CREATE TABLE <table_name>
(
    <column_1> <type>,
    <column_2> <type>,
    .
    .
    <index_type> <index_name> (<column_1>, <column_2>, ...),
    .
    .
);
```

Composite indexes can also be applied to existing tables using the syntax below:

```
CREATE <index_type> <index_name> ON <table_name>(<column_1>, <column_2> ... );
```

Use the following statement to create a composite regular index for the customer table based on the name and email columns:

```
CREATE INDEX idx_composite ON customer(name, email);
```

A composite unique index ensures that the combination of values across multiple columns is unique. In other words, while individual columns in the composite index can have duplicate values, the combination of values across all the indexed columns must be unique.

When creating composite indexes, the order in which the indexed columns are declared can impact query performance. As a general rule, the column names should be ordered from high to low based on the frequency with which they are included in query requests. For instance, if searches are more likely to be performed based on the customer’s email address than their name, the above index should be created as follows:

```
CREATE INDEX idx_composite ON customer(email, name);
```

18.7 Spatial index

A spatial index is used to index spatial data types, such as geographic information, and is supported only on tables using the InnoDB or MyISAM storage engines. Spatial indexes are typically used for specialized data such as map coordinates, positions of polygon vertices, or line start and end points using the POLYGON() and LINESTRING() functions, respectively.

Spatial indexes are declared using a syntax similar to previous indexes:

```
CREATE TABLE <table_name>
(
    <column_1> <type>,
    <column_2> <type>,
    .
    .
    SPATIAL INDEX <index_name> (<column_name>),
    .
    .
);
```

The example below generates a spatial index while creating a table:

```
CREATE TABLE poly (
    id INT AUTO_INCREMENT PRIMARY KEY,
    zone_name VARCHAR(100),
    area POLYGON NOT NULL,
    SPATIAL INDEX (area)
);
```

Alternatively, use the syntax below to add a spatial index to an existing table:

```
CREATE SPATIAL INDEX <index_name> ON <table_name>(<column_name>);
```

Unlike the other index types, it is not possible to create composite spatial indexes.

18.8 Getting index details

Information about the indexes associated with a table is accessed using the SHOW INDEX statement and referencing the table name. Run the following statement to generate a list of indexes associated with the customer table:

```
SHOW INDEX FROM customer\G
```

The output from the SHOW INDEX statement will include a row for each index similar to the one below:

```
***** 3. row *****
      Table: customer
      Non_unique: 0
      Key_name: idx_address
      Seq_in_index: 1
      Column_name: address
      Collation: A
      Cardinality: 0
      Sub_part: NULL
      Packed: NULL
```

```

Null: YES
Index_type: BTREE
Comment:
Index_comment:
Visible: YES
Expression: NULL

```

Figure 18-1 describes the columns contained within the SHOW INDEX output:

Column	Description
Non_unique	Indicates whether the index allows duplicate values (0 for unique indexes, 1 for non-unique indexes).
Key_name	The name of the index.
Seq_in_index	The sequence number of the column in the index (used in composite indexes).
Column_name	The name of the column included in the index.
Collation	How the column is sorted in the index (A for ascending, NULL for no sorting).
Cardinality	An estimate of the number of unique values in the index.
Sub_part	If the index is on a prefix of the column, this indicates the number of indexed characters. NULL if the entire column is indexed.
Packed	Indicates whether the index is compressed. NULL if not.
Null	Specifies whether the indexed column can contain NULL values.
Index_type	The type of index (e.g., BTREE, FULLTEXT, or HASH).
Comment	Additional information about the index.

Figure 18-1

18.9 Deleting indexes

To remove an index, use the DROP INDEX statement:

```
DROP INDEX idx_address ON customer;
```

18.10 Limitations of indexes

The old saying that nothing in life is free also applies to database indexes. While indexes can improve database query performance when used correctly, some drawbacks must be considered.

Like most server data, indexes take up disk space. As the number and size of the indexes in a database grow, they will consume increasing amounts of disk space. In a world where most servers are hosted in the cloud and storage is charged by the Gigabyte, consideration needs to be given to the potential costs of indexing large tables.

Additionally, while indexes can improve the performance of database queries, they can have the opposite effect on INSERT, UPDATE, or DELETE operations. Each time a table is updated, those changes must be reflected in the corresponding indexes. For example, adding or modifying a row in a table with multiple indexes requires MySQL to update each index, reducing overall database performance.

Although indexes are useful tools for improving database performance, they require planning, management, and monitoring to ensure optimal results.

18.11 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL indexes:

<https://www.answerstopia.com/6ywj>



18.12 Reference points

The main points covered in this chapter are as follows:

- **Overview**

- Indexes enhance MySQL query performance by allowing faster data retrieval.
- They function similarly to a book index, allowing quick searches without scanning entire tables.
- MySQL supports different types of indexes, each serving a specific purpose.
- Indexes can be added when creating a table or later applied to existing tables.

- **Types of Indexes**

- **Primary Index**

- Automatically created with a `PRIMARY KEY`, ensures uniqueness and non-null values.

- **Unique Index**

- Guarantees unique values in a column but allows `NULL` values.

- **Regular (Non-Unique) Index**

- Improves query performance but does not enforce unique values.

- **Full-Text Index**

- Optimized for text searching with support for natural language and Boolean mode.

- **Composite Index**

- Indexes multiple columns together to improve multi-column queries.

- **Spatial Index**

- Used for indexing geographic or spatial data types, supported in InnoDB and MyISAM.

- **Managing Indexes**

- Indexes can be created during table creation or added later.
 - The `SHOW INDEX` command provides details about indexes in a table.
 - Indexes can be removed using the `DROP INDEX` statement.

- **Limitations**

- Indexes consume disk space, increasing storage costs.

- They can slow down `INSERT`, `UPDATE`, and `DELETE` operations due to required updates.
- Overuse of indexes can lead to performance degradation instead of improvements.
- Requires proper planning and monitoring to ensure effective optimization.

19. MySQL Stored Routines

By now, it should be clear that SQL statements can often become lengthy and complex. While we have been typing these statements directly into the MySQL client, a significant part of your role as a database administrator will involve writing and maintaining sequences of SQL statements that are executed repeatedly. These statements might be run manually using a MySQL client, automatically through server or client-side scripts, or in response to a front-end client such as a website or software application. The key objective in such scenarios is reusability, the cornerstone of which, and this chapter's topic, is *stored routines*. While exploring stored routines, we will also look at variables and Common Table Expressions.

19.1 An overview of stored routines

Stored routines are database objects that encapsulate SQL code for repeated execution. They come in two forms: *stored procedures* and *stored functions*. Stored procedures perform actions without necessarily returning a value, whereas stored functions are designed to return a value, much like functions in programming languages such as C#, Python, or Java. These routines efficiently centralize logic within the database, providing reusability and consistency in database operations.

19.2 The benefits of stored routines

To perform database operations, a client will send SQL statements to the server and await the results. Based on the results, further instructions may be sent by the client, followed by yet more based on subsequent return values, and so on. If the client is remote from the server, each transaction will introduce a delay due to network latency. Stored routines eliminate much of this communication by storing the SQL logic on the server. Instead of a repetitive back and forth between the client and server, the client can call the routine and await the final result.

When working with distributed systems like client-server databases, it is important to minimize the number of places where changes to code logic need to be made. Consider, for example, a set of SQL statements common to various clients of a specific database. If this code is stored within the clients instead of the server, any bug fixes or code updates must be applied to every client. If the code is instead stored centrally on the database server, however, the code updates only need to be made in one location, resulting in easier code maintenance.

Stored routines also provide an extra layer of database security. Instead of providing a user with access privileges to a table containing sensitive data, the user can be allowed to run a stored routine that performs only the necessary actions to complete a task.

19.3 Creating stored procedures

Stored procedures are created in MySQL using the CREATE PROCEDURE statement, defining the procedure's name, specifying its parameters (if any), and encapsulating the SQL logic within a BEGIN...END block. The syntax for declaring stored procedures is as follows:

```
DELIMITER //
CREATE PROCEDURE
    <procedure_name>(IN/OUT/INOUT <param> <type>, <param> <type> .. )
BEGIN
    <SQL statements>
END //
DELIMITER ;
```

MySQL Stored Routines

The DELIMITER statements in the syntax above require some clarification. Throughout this book, we have used the semi-colon (;) to indicate the end of a statement and to initiate execution. However, when writing a procedure, we still need to use the semi-colon, but we want to prevent the code from executing until we call the procedure later. To address this issue, we temporarily change the delimiter to something else (typically '/' or '\$\$') while we write the procedure and then switch it back to the semi-colon once we have finished.

Stored routines support three parameter types: IN, OUT, and INOUT. The IN parameter allows a value to be passed into the routine, while an OUT parameter enables the routine to return a value to the caller. The INOUT parameter serves as both an input and an output, allowing the caller to pass in a value and receive an updated value in return.

19.4 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the “*Start Here*” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampled;"  
mysql -u root -p sampled < mysql_routines.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampled;
```

19.5 Working with MySQL variables

Before we look at some stored routine examples, we need to briefly discuss MySQL variables, which form the basis of output parameters.

MySQL variables store temporary values that can be referenced and manipulated during a session or within a query. MySQL supports both user-defined and system variables.

User-defined variable names are prefixed with the '@' character and created using the SET statement:

```
SET @myVar = 150;
```

User-defined variables can also be set from query results using the SELECT INTO statement, the syntax for which is as follows:

```
SELECT <column_1>, <column_2> ...  
  INTO <variable_1>, <variable_2>, ...  
  FROM <table>;
```

These variables are session-specific, persist until the session ends and can be inspected using the SELECT statement:

```
SELECT @myVar;
```

System variables, on the other hand, control the configuration and behavior of the MySQL server and can be either global or session-specific. They are predefined by MySQL and can be viewed or modified using the SHOW VARIABLES and SET commands, respectively.

19.6 Stored procedure examples

The following code creates a procedure named *ProductsBySupplier()* that takes a company name as an input parameter and retrieves a list of associated products:

```
DELIMITER //

CREATE PROCEDURE ProductsBySupplier(IN company_name varchar(25))
BEGIN
    SELECT product.name
    FROM product
    RIGHT JOIN supplier
    ON product.supplier_id = supplier.id
    WHERE company = company_name;
END //

DELIMITER ;
```

To execute this procedure, we use the CALL statement, as shown below:

```
CALL ProductsBySupplier('Apple');
+-----+
| name                |
+-----+
| MacBook Pro M4 14-in |
| MacBook Air M3       |
| Mac Mini M3          |
| Mac Studio           |
| iPhone 17 Pro        |
| iPhone 17            |
| iMac                 |
+-----+
7 rows in set (0.00 sec)
```

The procedure below demonstrates the use of IN and OUT parameters in the same routine. The input parameter is the supplier name, while the output parameters will contain the lowest and highest prices of the supplier's products:

```
DELIMITER //

CREATE PROCEDURE SupplierMinMax(
    IN company_name varchar(25),
    OUT min_price DECIMAL(6,2),
    OUT max_price DECIMAL(6,2))
BEGIN
    SELECT MIN(price), MAX(price)
    INTO min_price, max_price
    FROM product
    RIGHT JOIN supplier
    ON product.supplier_id = supplier.id
    WHERE company = company_name;
END //

DELIMITER ;
```

When we call the procedure, we will pass references to variables for the OUT parameters. After the procedure

MySQL Stored Routines

finishes, these variables will hold the minimum and maximum product prices. We assign these values to the variables within the procedure using the following `SELECT INTO` statement:

```
SELECT MIN(price), MAX(price) INTO min_price, max_price ...
```

Use the following `CALL` statement to execute the procedure:

```
CALL SupplierMinMax('Apple', @min_price, @max_price);
```

After calling the procedure, run the following `SELECT` statement to inspect the two result variables:

```
SELECT @min_price, @max_price;
+-----+-----+
| @min_price | @max_price |
+-----+-----+
|      799.99 |     2499.99 |
+-----+-----+
1 row in set (0.00 sec)
```

We could also, for example, use these variables in a `SELECT` statement to identify products with minimum and maximum prices:

```
SELECT name, price
FROM product
WHERE price = @min_price OR price = @max_price;
+-----+-----+
| name                | price    |
+-----+-----+
| MacBook Pro M4 14-in | 2499.99 |
| iPhone 17            | 799.99  |
+-----+-----+
2 rows in set (0.00 sec)
```

19.7 Creating stored functions

MySQL stored functions are blocks of SQL code that perform a task and return a result. A set of keywords are used when writing functions, including `RETURNS`, `RETURN`, and `DETERMINISTIC`, as well as *interaction clauses* like `NO SQL`, `CONTAINS SQL`, `MODIFIES SQL DATA`, and `READS SQL DATA`. These keywords and clauses define how the function interacts with the database and how MySQL optimizes its execution. We will start by outlining the syntax for declaring a function:

```
DELIMITER //

CREATE FUNCTION <function_name>(<param> <type>, <param> <type> ..)
RETURNS <return_type>
NO SQL | READS SQL DATA | MODIFIES SQL DATA | CONTAINS SQL
DETERMINISTIC | NOT DETERMINISTIC
BEGIN
    RETURN <expression>;
END //

DELIMITER ;
```

Before we try out some example functions, we need to explain each of the clauses included in the above syntax:

19.7.1 RETURNS

The RETURNS keyword declares the data type of the function's return value, which is mandatory for all stored functions. Declaring the incorrect return type may result in unpredictable behavior.

19.7.2 RETURN

The RETURN keyword is used within a function's body to indicate the value that will be returned to the caller. Each function can have only one RETURN statement for every possible execution path, and the value returned must be consistent with the data type defined in the RETURNS clause. For instance, if the RETURNS clause specifies a DECIMAL type, attempting to return a CHAR or any other incompatible type will result in an error.

19.7.3 DETERMINISTIC

When you write functions, you must declare them as either deterministic or non-deterministic. If a function is declared as deterministic, it must return the same result each time it is called when passed the same input parameters. This predictability allows MySQL to cache return values and return the results from previous matching calls without executing the overhead of rerunning the function.

Do not declare a non-deterministic function as deterministic. Doing so often results in inaccurate return values.

19.7.4 NON DETERMINISTIC

The return results of non-deterministic functions vary, even when passed the same input parameters. This variability is usually the result of a dependency on dynamic values within the function's logic, such as the current date or random number generation.

19.7.5 NO SQL

The NOSQL interaction clause indicates to MySQL that the function body does not execute SQL statements. In other words, the function relies entirely on its input parameters and internal logic to perform calculations or operations without interacting with the database.

19.7.6 READS SQL DATA

A function declared with READS SQL DATA can execute SQL queries that retrieve data from the database but cannot modify any data. This clause is used for functions that depend on database queries to calculate their results.

19.7.7 MODIFIES SQL DATA

The MODIFIES SQL DATA clause tells MySQL that the function will perform database updates such as inserting, updating, or deleting data. Stored functions are generally intended to return results without altering data, so this clause is not frequently used.

19.7.8 CONTAINS SQL

The CONTAINS SQL clause tells MySQL that the function uses SQL statements that neither read from tables nor modify the database. While the function may perform operations such as declaring variables or calling other stored routines, it does not directly interact with table data.

Figure 19-1 provides a summary of the interaction clauses that are supported by the CREATE FUNCTION statement:

Clause	Description	Example Use Case
NO SQL	The function does not execute any SQL statements and relies entirely on input parameters.	Mathematical calculations.

READS SQL DATA	The function retrieves data from the database but does not modify it.	Table queries.
CONTAINS SQL	The function contains SQL statements but does not read or modify table data.	Internal processing or reading and manipulating variables.
MODIFIES SQL DATA	The function modifies table data by inserting, updating, or deleting records.	Data updates, deletions, and insertions.

Figure 19-1

19.8 Stored function examples

With the basics of stored functions covered in the previous section, the next step is to try out some examples, starting with the following example:

```
DELIMITER //

CREATE FUNCTION
CalculateTax(price DECIMAL(10, 2), tax_rate DECIMAL(10, 2))
  RETURNS DECIMAL(10, 2)
  NO SQL
  DETERMINISTIC
  BEGIN
    RETURN price * (tax_rate / 100);
  END //

DELIMITER ;
```

The CalculateTax() function accepts two values representing the product price and the tax rate percentage. The function body converts the tax rate to the decimal equivalent and then multiplies the price by the result. The function does not use SQL statements and will return the same value when provided the same input values, so it is declared using the NO SQL and DETERMINISTIC clauses. After adding the function to the database, execute it as follows:

```
SELECT CalculateTax(220, 5.25);
+-----+
| CalculateTax(220, 5.25) |
+-----+
|                11.55 |
+-----+
1 row in set (0.00 sec)
```

The following example declares a function called FormatName(), which uses the customer's first and last names as parameters. The returned result is a concatenation of the first and last names, separated by a space character:

```
DELIMITER //

CREATE FUNCTION FormatName(first_name VARCHAR(20), last_name VARCHAR(20))
  RETURNS VARCHAR(40)
  CONTAINS SQL
  DETERMINISTIC
  BEGIN
    DECLARE full_name VARCHAR(40);
    SET full_name = CONCAT(first_name, ' ', last_name);
    RETURN full_name;
  END //
```



```
DELIMITER ;
```

In the above example, the `FormatName()` function is deterministic and uses the SQL `CONCAT` function. Since the function does not access the database, it was declared using the `CONTAINS SQL` clause. Use the following `SELECT` statement to test the function:

```
SELECT FormatName('Davy', 'Crockett');
+-----+
| FormatName('Davy', 'Crockett') |
+-----+
| Davy Crockett                  |
+-----+
1 row in set (0.00 sec)
```

The final example is a function designed to return a random item from the product table to be awarded as a customer prize:

```
DELIMITER //

CREATE FUNCTION GetRandomPrize()
  RETURNS VARCHAR(25)
  READS SQL DATA
  NOT DETERMINISTIC
  BEGIN
    DECLARE prize VARCHAR(25);
    SELECT name INTO prize FROM product ORDER BY RAND() LIMIT 1;
    RETURN prize;
  END //

DELIMITER ;
```

The random behavior of `GetRandomPrize()` requires that we declare it a non-deterministic function. Since the function body queries but does not update the product table, we have included the `READS SQL DATA` interaction clause. The function then declares a local variable named `prize` to store the name of the selected product and executes the following `SELECT` statement:

```
SELECT name INTO prize FROM product ORDER BY RAND() LIMIT 1;
```

The above statement uses the `RAND()` function to generate a random number and uses it to shuffle the result set randomly. The `LIMIT 1` operator narrows the result set to a single row, which is stored in the `prize` variable using the `INTO` operator and returned to the caller. Test the function using the following statement:

```
SELECT GetRandomPrize();
+-----+
| GetRandomPrize() |
+-----+
| One2One USB-A Hub |
+-----+
1 row in set (0.00 sec)
```

We can also assign the result to a variable and use it in subsequent statements, for example:

```
SET @random_prize = GetRandomPrize();

SELECT name, price FROM product WHERE name = @random_prize;
```

```

+-----+-----+
| name           | price |
+-----+-----+
| Corsair Full Tower | 179.99 |
+-----+-----+
1 row in set (0.00 sec)

```

19.9 Using stored functions as subqueries

Stored functions may also be used within subqueries, though doing so can sometimes result in inconsistent results, mainly when using non-deterministic functions. To experience this first-hand, run the following SELECT statement repeatedly, which invokes `GetRandomPrize()` as a subquery and note the results:

```
SELECT name FROM product WHERE name = (SELECT GetRandomPrize());
```

Occasionally, the queries will return a single, random product name as intended, but you will also see empty result sets and even results containing multiple products like the following:

```

SELECT name FROM product WHERE name = (SELECT GetRandomPrize());
+-----+-----+
| name           |
+-----+-----+
| iPhone 17      |
| Dell XPS Model 7823 |
| One2One USB-A Hub |
+-----+-----+
3 rows in set (0.04 sec)

```

There are several potential causes for the above behavior:

- MySQL uses a query optimizer when it executes queries. Depending on the query, the optimizer may evaluate a subquery more than once during a single query operation.
- When used within a WHERE clause, MySQL may treat some functions as dynamic expressions that need to be evaluated for each row. While this isn't an issue for deterministic functions, it can cause problems with non-deterministic functions that produce different results each time they are called.
- MySQL does not generally cache or optimize the results of non-deterministic expressions. Functions like `RAND()` inherently produce different values with each call, which leads MySQL to treat them as dynamic. As a result, the server re-executes these functions whenever they are referenced. This means that even within a single query execution, the subquery may generate unpredictable results.

There are several ways to avoid unpredictable behavior when using stored functions in subqueries. One option, as we have already seen, is to assign the function result to a variable, which is then used within the subsequent query:

```

SET @random_prize = GetRandomPrize();
SELECT name, price FROM product WHERE name = @random_prize;

```

Another way to do this is by wrapping the subquery in another query to store its result temporarily. For example, we can use a subquery to compute the result of `GetRandomPrize()` and then reference that result in the main query:

```

SELECT name FROM product
WHERE name = (SELECT random_prize FROM (SELECT GetRandomPrize()
AS random_prize) AS temp_prize);

```

```
+-----+
| name   |
+-----+
| iPhone 17 |
+-----+
1 row in set (0.00 sec)
```

A third alternative is to use a Common Table Expression (CTE) to pre-compute the random value and reuse it without multiple evaluations.

19.10 Introducing Common Table Expressions

Common Table Expressions (CTE) allow you to collect results from a query in a temporary result set and reference that set by name elsewhere in the same query expression. CTEs are defined using the WITH keyword with the following syntax:

```
WITH <cte_name> AS (
    SELECT ...
)
SELECT ...
FROM <table_name>;
```

CTEs are useful for simplifying complex queries and for reusing the result of a subquery multiple times within the same query. They exist only for the duration of the query in which they are used.

The following statement, for example, uses a CTE to produce consistent results from the GetRandomPrize() function:

```
WITH prize_cte AS (
    SELECT GetRandomPrize() AS random_prize
)
SELECT name
FROM product
WHERE name = (SELECT random_prize FROM prize_cte);
+-----+
| name           |
+-----+
| Dell XPS Model 7823 |
+-----+
1 row in set (0.00 sec)
```

19.11 Viewing procedures and functions

Information about routines is stored in the *routines* and *functions* tables of the built-in MySQL *information_schema* database and is accessed using the following syntax:

```
SELECT <column names>
FROM information_schema.routines
WHERE routine_type = '[PROCEDURE | FUNCTION]'
AND routine_schema = '<database name>'\G
```

The following statement, for example, displays all the fields for each function declared within the *sampledb* database:

```
SELECT * FROM information_schema.routines
WHERE routine_type = 'FUNCTION'
AND routine_schema = 'sampledb'\G
```

MySQL Stored Routines

```
***** 1. row *****
    SPECIFIC_NAME: CalculateTax
    ROUTINE_CATALOG: def
    ROUTINE_SCHEMA: sampledb
    ROUTINE_NAME: CalculateTax
    ROUTINE_TYPE: FUNCTION
    DATA_TYPE: decimal
CHARACTER_MAXIMUM_LENGTH: NULL
CHARACTER_OCTET_LENGTH: NULL
    NUMERIC_PRECISION: 10
.
.
***** 2. row *****
    SPECIFIC_NAME: FormatName
    ROUTINE_CATALOG: def
    ROUTINE_SCHEMA: sampledb
    ROUTINE_NAME: FormatName
    ROUTINE_TYPE: FUNCTION
    DATA_TYPE: varchar
CHARACTER_MAXIMUM_LENGTH: 40
CHARACTER_OCTET_LENGTH: 160
    NUMERIC_PRECISION: NULL
.
.
```

Similarly, the following statement displays the name and definition of all functions within the database:

```
SELECT ROUTINE_NAME, ROUTINE_DEFINITION FROM information_schema.routines
WHERE routine_type = 'PROCEDURE'
AND routine_schema = 'sampledb'\G
***** 1. row *****
    ROUTINE_NAME: ProductsBySupplier
ROUTINE_DEFINITION: BEGIN
    SELECT product.name
    FROM product
    RIGHT JOIN supplier
    ON product.supplier_id = supplier.id
    WHERE company = company_name;
END
***** 2. row *****
    ROUTINE_NAME: SupplierMinMax
ROUTINE_DEFINITION: BEGIN
    SELECT MIN(price), MAX(price)
    INTO min_price, max_price
    FROM product
    RIGHT JOIN supplier
    ON product.supplier_id = supplier.id
    WHERE company = company_name;
END
2 rows in set (0.00 sec)
```

19.12 Deleting stored procedures and functions

Stored routines are deleted using the DROP FUNCTION and DROP PROCEDURE statement. Before continuing, execute the following statements to delete the example routines:

```
DROP PROCEDURE SupplierMinMax;
DROP PROCEDURE ProductsBySupplier;
DROP FUNCTION CalculateTax;
DROP FUNCTION FormatName;
DROP FUNCTION GetRandomPrize;
```

19.13 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL stored routines:

<https://www.answerstopia.com/dxg6>



19.14 Reference points

The main points covered in this chapter are as follows:

- **Overview**
 - Stored routines encapsulate reusable SQL logic in the database.
- **Routine Types**
 - **Stored Procedures:** Execute SQL statements without necessarily returning a value.
 - **Stored Functions:** Return a value based on SQL logic, similar to programming functions.
- **Benefits**
 - **Reduced Client-Server Communication:** Minimize network latency by centralizing logic on the database server.
 - **Simplified Maintenance:** Updates to logic are applied once on the server instead of across multiple clients.
 - **Enhanced Security:** Restrict user access to sensitive data by granting permission to execute routines rather than direct table access.
- **MySQL Variables**
 - **User-defined variables:** Session-specific and prefixed with @ (e.g., @myVar).
 - **System variables:** Predefined by MySQL for configuration and operational control.
 - Variables can store query results using `SELECT INTO` and are used extensively in stored procedures.
- **Stored Procedures**
 - Created using the `CREATE PROCEDURE` statement and structured with a `BEGIN . . . END` block.
 - Support `IN`, `OUT`, and `INOUT` parameters for passing and returning values.
 - Useful for handling complex operations, such as filtering and aggregations.
- **Stored Functions**
 - Created with `CREATE FUNCTION`, they must return a value.

MySQL Stored Routines

- Use clauses like RETURNS, RETURN, DETERMINISTIC, and interaction clauses (NO SQL, READS SQL DATA, etc.).
- Examples include mathematical operations, formatting, and retrieving specific data.
- Deterministic functions return consistent results for the same input.
- **Interaction Clauses**
 - NO SQL: Function does not interact with database tables.
 - READS SQL DATA: Function reads from the database but does not modify data.
 - MODIFIES SQL DATA: Function updates, deletes, or inserts data.
 - CONTAINS SQL: Function uses SQL but does not interact with tables directly.
- **Common Table Expressions (CTEs)**
 - Simplify complex queries by creating temporary named result sets.
 - Defined using the WITH clause and used within the same query.
 - Useful for ensuring consistent results, especially when working with non-deterministic functions.
 - Avoid multiple evaluations of non-deterministic expressions.
- **Viewing Stored Procedures and Functions**
 - MySQL stores information about stored routines in the information_schema.routines table.
 - Routine details can be retrieved using SQL queries, filtering by ROUTINE_TYPE (PROCEDURE or FUNCTION) and ROUTINE_SCHEMA (database name).
 - The ROUTINE_NAME and ROUTINE_DEFINITION fields provide the name and logic of stored procedures and functions.
- **Deleting Stored Procedures and Functions**
 - The DROP FUNCTION and DROP PROCEDURE statements permanently remove stored routines from a database.

20. Working with Control Flow

Control flow allows us to build logic into our code and move beyond executing linear sequences of SQL statements. MySQL includes a set of statements that we can use to control the execution path of our code, including conditional statements for decision-making and iteration loops. This chapter will explore MySQL's various control flow constructs and demonstrate how to use them in your code.

20.1 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the “*Start Here*” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampled;"  
mysql -u root -p sampled < mysql_control_flow.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampled;
```

20.2 Looping in MySQL

We will begin by looking at loop-based control flow. Loops are sequences of SQL statements to be executed repeatedly until a specified condition is met. The first looping statement we will explore is the WHILE loop.

20.2.1 The WHILE loop

The WHILE loop executes a block of SQL statements repeatedly until a specified condition is no longer true. For example, a loop might iterate through table records until a counter variable exceeds a maximum value. A key characteristic of WHILE loops is that the condition is evaluated before executing the code block. Consequently, if the condition is initially evaluated as false, the loop body will not run, as illustrated in Figure 20-1:

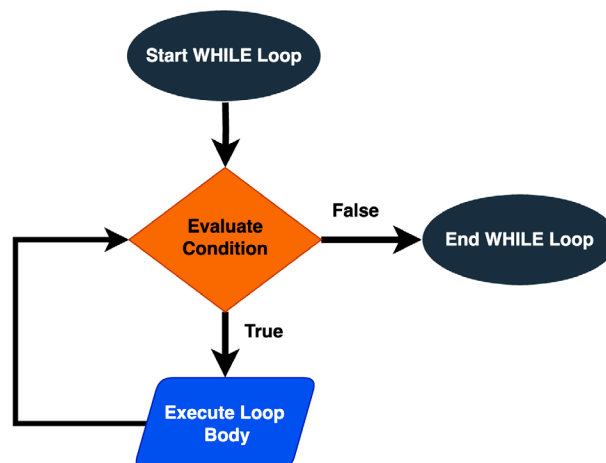


Figure 20-1

Working with Control Flow

WHILE loops are declared in MySQL using the following syntax:

```
WHILE <condition> DO
    <sql statements>
END WHILE;
```

Enter the following into the MySQL client to declare a procedure called SquareNumbers() that uses a WHILE loop to output the square of the numbers within a range defined by the start and end parameters:

```
DELIMITER //

CREATE PROCEDURE SquareNumbers(IN start INT, IN end INT)
BEGIN
    DECLARE result INT;

    WHILE start <= end DO
        SET result = start * start;
        SELECT result;
        SET start = start + 1;
    END WHILE;
END //

DELIMITER ;
```

The procedure begins by creating a variable to store the calculation result before declaring a WHILE loop to be performed until the start variable is greater than the end value. The loop's body calculates the square of the current start value, displays the result, and increments the start value.

Use the statement below to test the loop:

```
CALL SquareNumbers(5, 8);
+-----+
| result |
+-----+
|      25 |
+-----+
1 row in set (0.01 sec)

+-----+
| result |
+-----+
|      36 |
+-----+
1 row in set (0.01 sec)

.
.
1 row in set (0.01 sec)
```

The above output shows that the loop was executed for each number in the specified range. If, instead, we provide a start value that is already greater than the end value, we will see that the condition initially evaluates to false and the loop is not executed:

```
CALL SquareNumbers(8, 5);
Query OK, 0 rows affected (0.00 sec)
```


20.2.2 The REPEAT loop

You can think of the REPEAT loop as an inverted WHILE loop. As previously discussed, a WHILE loop evaluates a condition before executing the code in the loop code block. The code is not executed if the condition is evaluated as false on the first check. On the other hand, the REPEAT loop is provided for situations where you know that the code contained in the code block of the loop will always need to be executed at least once. For example, you may want to keep updating the rows in a table until a specific match is found. You know that you have to check the first row at least to have any hope of finding the record you need. The diagram in Figure 20-2 illustrates the REPEAT loop execution path:

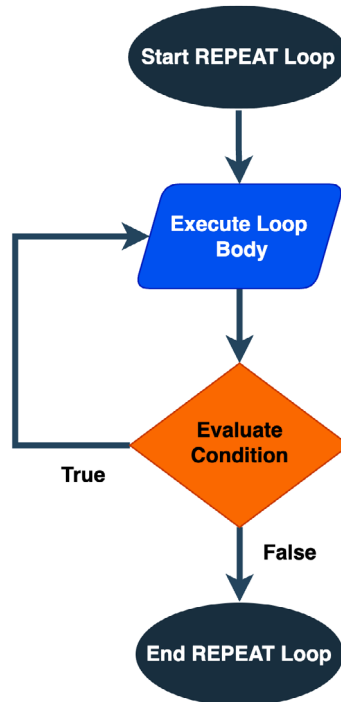


Figure 20-2

The syntax for the REPEAT loop is as follows:

```

REPEAT
    <sql statements>
UNTIL <condition>
END REPEAT;
```

To demonstrate the REPEAT loop, we will write a procedure to calculate the factorial of a number. Enter the following statements to declare a procedure called Factorial():

```

DELIMITER //

CREATE PROCEDURE Factorial(IN num INT)
BEGIN
    DECLARE result INT DEFAULT 1;

    REPEAT
        SET result = result * num;
        SET num = num - 1;
```

Working with Control Flow

```
UNTIL num = 0
END REPEAT;

SELECT result;
END //

DELIMITER ;
```

Test that the procedure works using the following CALL statement:

```
CALL Factorial(10);
+-----+
| result |
+-----+
| 3628800 |
+-----+
1 row in set (0.01 sec)
```

20.3 The LOOP construct

The LOOP statement provides a way to execute a block of code repeatedly, but unlike the WHILE and REPEAT loops, it does not include a built-in condition to control when the loop must exit. Instead, the LEAVE clause is used within the LOOP code block to indicate points where the loop must terminate. This moves the condition logic into the loop code block, allowing for greater flexibility and multiple exit conditions:

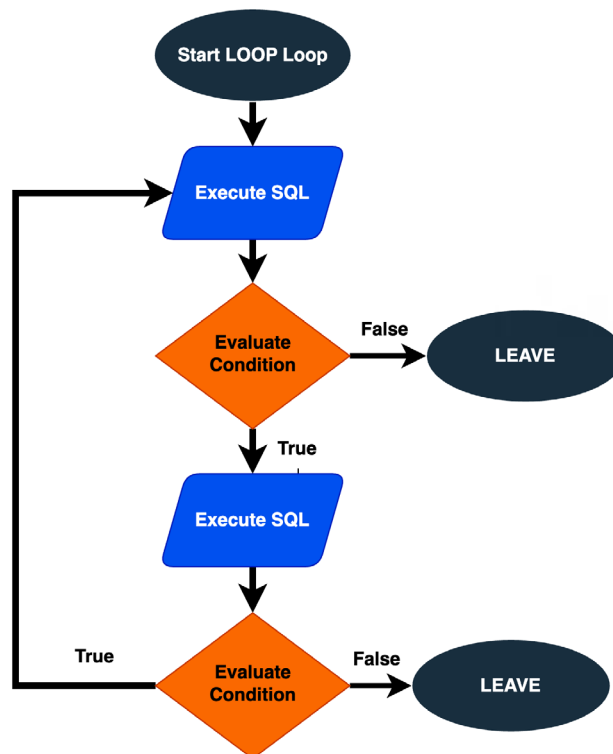


Figure 20-3

Each LOOP construct must be assigned a label, which, in turn, is referenced in the LEAVE statement. The LOOP construct uses the following syntax:

```

<loop_label>: LOOP
    <sql statements>
    LEAVE <loop_label>;
END LOOP;

```

Moving the conditional logic into the code block makes it possible to implement multiple exit points within the loop, for example:

```

<loop_label>: LOOP
    <sql statements>
    LEAVE <loop_label>;
    <sql statements>
    LEAVE <loop_label>;
.
.
END LOOP;

```

The LOOP statement also supports nested loops using the following syntax:

```

<loop_1_label>: LOOP
    <sql statements>

    <loop_2_label>: LOOP
        <sql statements>
        LEAVE <loop_2_label>
    END LOOP;

    LEAVE <loop_1_label>;
END LOOP;

```

The following procedure accepts parameters named *count* and *max* and contains a loop that doubles *count* until it exceeds the maximum value:

```

DELIMITER //

CREATE PROCEDURE SingleLoop(IN count INT, IN max INT)
BEGIN
    count_loop: LOOP
        IF count > max THEN
            LEAVE count_loop;
        END IF;

        SELECT count;
        SET count = count * 2;
    END LOOP;
END //

DELIMITER ;

```

Use the following CALL statement to try out the procedure:

```

CALL SingleLoop(2, 10);
+-----+
| count |
+-----+
|      2 |

```

Working with Control Flow

```
+-----+
1 row in set (0.01 sec)

+-----+
| count |
+-----+
|      4 |
+-----+
1 row in set (0.01 sec)

+-----+
| count |
+-----+
|      8 |
+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

The next example demonstrates a nested LOOP construct that accepts a third parameter that controls the number of times count_loop executes:

```
DELIMITER //

CREATE PROCEDURE NestedLoop(IN count INT, IN max INT, IN reps INT)
BEGIN

    DECLARE outer_count INT DEFAULT 1;
    SET @saved_count = count;

    outer_loop: LOOP
        IF outer_count > reps THEN
            LEAVE outer_loop;
        END IF;

        inner_loop: LOOP
            IF count > max THEN
                LEAVE inner_loop;
            END IF;
            SELECT count;
            SET count = count * 2;
        END LOOP;

        SET count = @saved_count;
        SET outer_count = outer_count + 1;
    END LOOP;
END //
```

```
DELIMITER ;
```

In the NestedLoop() example, inner_loop is nested within outer_loop. Before outer_loop starts, the count parameter value is saved so that it can be restored at the end of each repetition of the inner loop.

The inner loop will run until the count parameter reaches the maximum value, at which point the LEAVE statement will exit the inner loop, passing control to the following line of the outer loop, where the saved initial

count value is restored, and `outer_count` is incremented.

The outer loop will continue to execute `inner_loop` until the `outer_count` value exceeds the designated number of repetitions (as defined by the *reps* parameter), resulting in the following output:

```
CALL NestedLoop(2, 8, 2);
```

```
+-----+
| count |
+-----+
|      2 |
+-----+
1 row in set (0.01 sec)
```

```
+-----+
| count |
+-----+
|      4 |
+-----+
1 row in set (0.01 sec)
```

```
+-----+
| count |
+-----+
|      8 |
+-----+
1 row in set (0.01 sec)
```

```
+-----+
| count |
+-----+
|      2 |
+-----+
1 row in set (0.01 sec)
```

```
+-----+
| count |
+-----+
|      4 |
+-----+
1 row in set (0.00 sec)
```

```
+-----+
| count |
+-----+
|      8 |
+-----+
1 row in set (0.00 sec)
```

```
Query OK, 0 rows affected (0.00 sec)
```

When working with nested loops, the `LEAVE` statement does not have to reference the loop in which it is declared. We could, for example, use the `LEAVE` statement in the inner loop to exit from the outer loop, essentially exiting both loops in a single operation:

```
DROP PROCEDURE nestedloop;
```

```

DELIMITER //

CREATE PROCEDURE NestedLoop(IN count INT, IN max INT, IN reps INT)
BEGIN

    DECLARE outer_count INT DEFAULT 1;
    SET @saved_count = count;

    outer_loop: LOOP
        IF outer_count > reps THEN
            LEAVE outer_loop;
        END IF;

        inner_loop: LOOP
            IF count > max THEN
                LEAVE outer_loop;
            END IF;
            SELECT count;
            SET count = count * 2;
        END LOOP;

        SET count = @saved_count;
        SET outer_count = outer_count + 1;
    END LOOP;
END //

DELIMITER ;

```

When the procedure runs, the outer loop will only run once, irrespective of the value of the *reps* parameter:

```

CALL NestedLoop(2, 8, 100);
+-----+
| count |
+-----+
|      2 |
+-----+
1 row in set (0.01 sec)

+-----+
| count |
+-----+
|      4 |
+-----+
1 row in set (0.01 sec)

+-----+
| count |
+-----+
|      8 |
+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

```

20.4 Conditional Statements in MySQL

Programming is an exercise in applying logic and primarily involves writing code that makes decisions based on one or more criteria. Such decisions define which code gets executed and which is bypassed during execution. Control flow is common to most programming languages and, in the case of MySQL, is provided by the IF and CASE statements, each of which will be covered in the remainder of this chapter:

20.4.1 The IF statement

The IF statement allows you to execute an SQL code block only when a specified condition is evaluated as true. The IF statement can be combined with the ELSEIF clause to test for multiple conditions and the ELSE clause to provide a default action if none of the conditions are evaluated to be true.

The basic syntax of an IF statement is as follows:

```
IF <condition_1> THEN
    <code to execute if the condition_1 is true>
ELSEIF <condition_2> THEN
    <code to execute if the condition_2 is true>
.
.
ELSE
    <code to execute if no conditions are met>
END IF;
```

The flow diagram in Figure 20-4 below provides a visual representation of an IF statement containing three ELSEIF conditions:

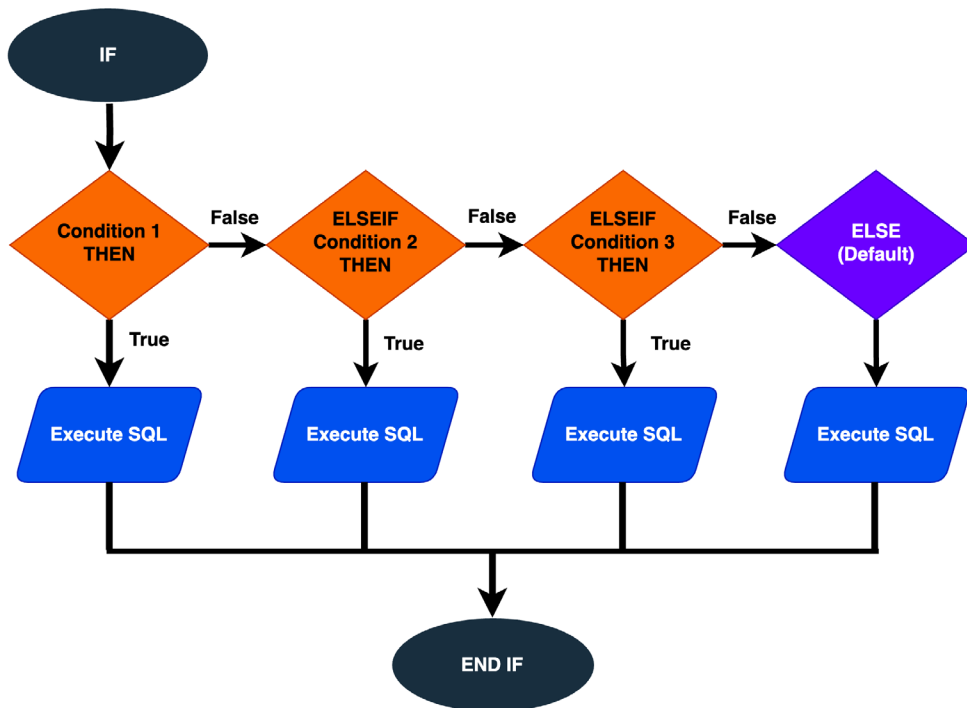


Figure 20-4

Suppose we need a way to identify an item's stock level in our product database. A procedure that uses an IF

statement to check stock levels based on the product id might read as follows:

```
DELIMITER //
```

```
CREATE PROCEDURE StockCheck(IN product_id INT, OUT stock_status VARCHAR(50))
BEGIN
    DECLARE product_name VARCHAR(25);
    DECLARE stock_quantity INT;

    SELECT quantity
    INTO stock_quantity
    FROM product
    WHERE id = product_id;

    IF stock_quantity = 0 THEN
        SET stock_status = 'Out of Stock';
    ELSEIF stock_quantity < 20 THEN
        SET stock_status = 'Low Stock';
    ELSEIF stock_quantity > 80 THEN
        SET stock_status = 'Excess Stock';
    ELSE
        SET stock_status = 'Product not found';
    END IF;
END //
```

```
DELIMITER ;
```

If we call the StockCheck() procedure with a product id value, the stock_status variable will update to reflect the stock level of the corresponding product:

```
CALL StockCheck(4, @stock_status);

SELECT @stock_status;
+-----+
| @stock_status |
+-----+
| Low Stock     |
+-----+
1 row in set (0.00 sec)
```

We can also test the default ELSE statement by providing a non-existent product id:

```
CALL StockCheck(100, @stock_status);

SELECT @stock_status;
+-----+
| @stock_status      |
+-----+
| Product not found  |
+-----+
1 row in set (0.00 sec)
```

20.4.2 The CASE statement

The CASE statement provides functionality similar to IF .. ELSEIF constructs but uses a more concise format, making it easier to evaluate many conditions in a single statement. CASE constructs also provide a convenient

way to integrate control flow directly into SQL queries.

There are two ways to use the CASE statement. The first method is ideal for use in stored routines, allows SQL statements to be executed for each matching condition, and uses the following syntax:

```
CASE <case_value>
  WHEN <value_1> THEN <sql_code>
  WHEN <value_2> THEN <sql_code>
  ELSE <sql_code>
END CASE;
```

The diagram in Figure 20-5 illustrates the structure of the above CASE syntax:

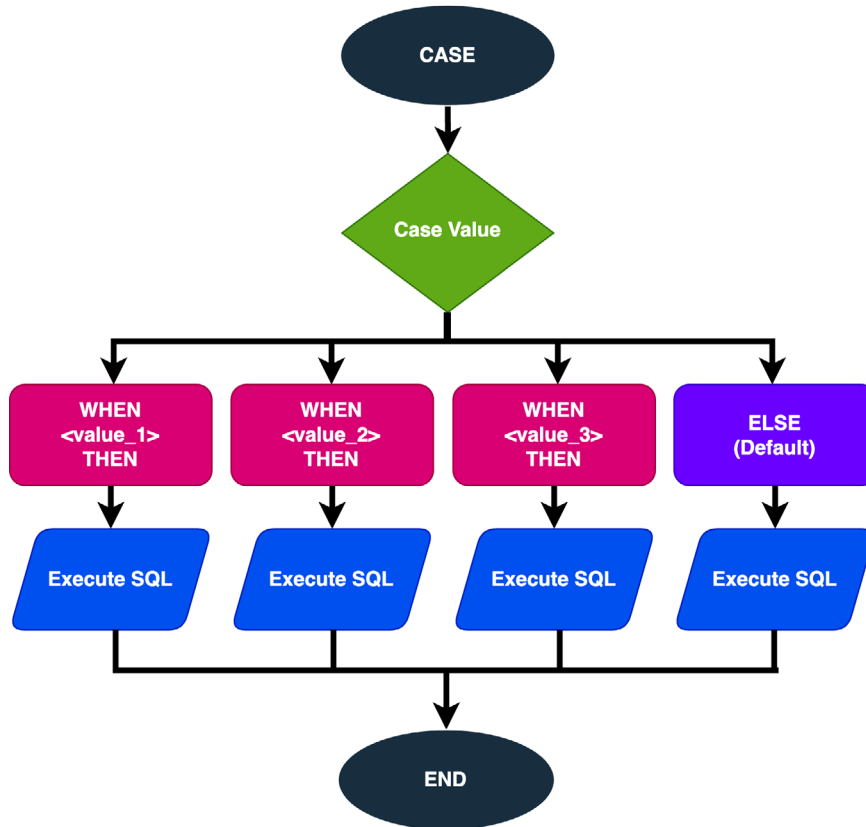


Figure 20-5

The second CASE variant returns a result for each condition instead of executing SQL statements and is ideal for integrating control flow directly into SQL queries. The syntax for this type of CASE expression reads as follows:

```
CASE
  WHEN <condition_1> THEN <result_1>
  WHEN <condition_2> THEN <result_2>
  ELSE <default_result>
END
```

Figure 20-6 outlines the flow of the result-based CASE constructs:

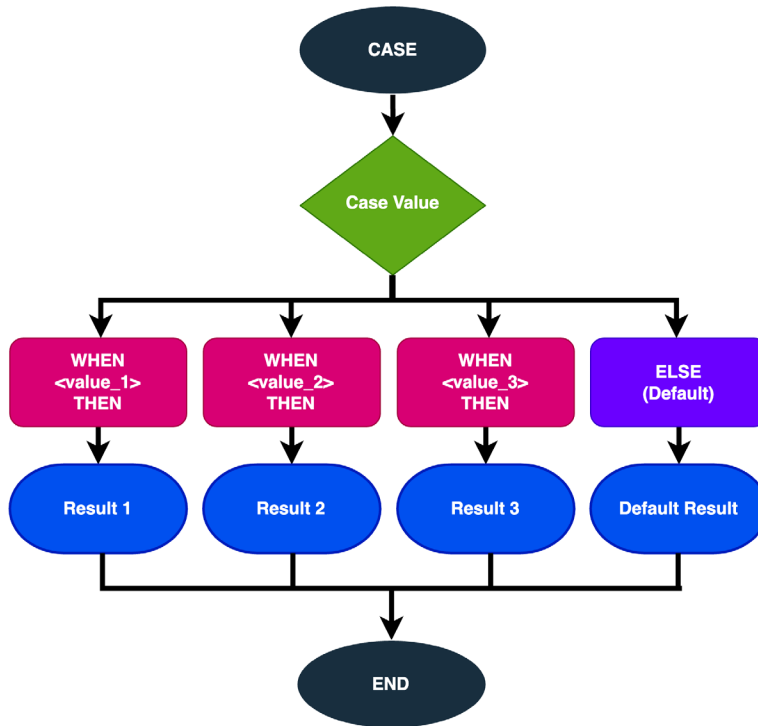


Figure 20-6

Execute the following SELECT statement to generate the stock status for each item in the product table using an embedded CASE statement:

```

SELECT name, quantity,
CASE
    WHEN quantity = 0 THEN 'Out of Stock'
    WHEN quantity > 60 THEN 'Excess Stock'
    WHEN quantity <= 20 THEN 'Low Stock'
    ELSE 'Medium Stock'
END AS 'stock_level'
FROM product;

```

name	quantity	stock_level
MacBook Pro M4 14-in	60	Medium Stock
MacBook Air M3	55	Medium Stock
Mac Mini M3	100	Excess Stock
Mac Studio	15	Low Stock
iPhone 17 Pro	30	Medium Stock
iPhone 17	30	Medium Stock
iMac	30	Medium Stock
NZXT Mid Tower	100	Excess Stock
Corsair Full Tower	15	Low Stock
Dell Model 4823	35	Medium Stock
Dell XPS Model 7823	16	Low Stock
Dell Dock Model [4807]	35	Medium Stock
One&One Touch 200	60	Medium Stock
One2One USB-A Hub	60	Medium Stock

```
| One4One USB-A Hub      |      60 | Medium Stock |
+-----+-----+-----+
15 rows in set (0.00 sec)
```

If you do not intend to use the routines from this chapter in the future, delete them using the following statements:

```
DROP PROCEDURE SquareNumbers;
DROP PROCEDURE Factorial;
DROP PROCEDURE SafeFactorial;
DROP PROCEDURE SingleLoop;
DROP PROCEDURE NestedLoop;
DROP PROCEDURE StockCheck;
```

20.5 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL control flow:

<https://www.answerptopia.com/6c7l>



20.6 Reference points

The main points covered in this chapter are as follows:

• Overview

- Control flow enables dynamic decision-making and repetitive execution of SQL statements.
- Includes looping constructs (`WHILE`, `REPEAT`, and `LOOP`) and conditional statements (`IF` and `CASE`) to structure logic.

• Loops

- Use `WHILE` for precondition checks, `REPEAT` for guaranteed execution, and `LOOP` for custom termination logic.
- Nested loops allow complex iterations and multiple exit points.

• WHILE Loop

- Executes a block of SQL statements repeatedly as long as a condition is true.
- Evaluates the condition before the loop starts, ensuring the loop does not run if the condition is initially false.

• REPEAT Loop

- Guarantees at least one execution of the code block before the condition is checked.
- Ideal for scenarios where the code must always run at least once before validation.

• LOOP Statement

- Executes a block of code repeatedly without built-in conditions for termination.
- Requires the `LEAVE` statement to terminate the loop.

- Supports nested loops, where an inner loop can terminate the outer loop for more complex logic.

- **Conditional Statements**

- Use `IF` for versatile decision-making logic with multiple branches.
- `CASE` offers a compact, readable alternative, ideal for query-based conditions.

- **IF Statement**

- Tests conditions and executes specific SQL code based on evaluation.
- Supports multiple branches using `ELSEIF` and a default action with `ELSE`.
- Commonly used to perform logic such as evaluating stock levels in a database.

- **CASE Statement**

- A concise alternative to `IF . . ELSEIF` for evaluating multiple conditions.
- Two forms:
 - Executes SQL code for each matching condition.
 - Returns results directly in SQL queries for embedded logic.

21. Error Handling in MySQL

Like any programming environment, errors can occur when executing code on a MySQL database. Some code is more susceptible to errors than others and usually arises from invalid data, logic errors, or miscalculations. Regardless of the cause, errors can cause code execution to terminate, potentially leaving the database in an indeterminate state. For this reason, it is important to identify the potential risks and develop defensive code to handle them. MySQL provides extensive error-handling features that will allow you to respond gracefully to errors when they occur. This chapter will cover error handling in MySQL using handlers, conditions, states, and exceptions.

21.1 Overview of MySQL error handling

Error handling in MySQL is configured using the `CREATE HANDLER` statement, which allows us to specify actions to be taken in response to specific errors or conditions. The three key elements of error handling in MySQL are *handlers*, *conditions*, and *custom conditions*, and we need to understand these before we explore error handling in more detail:

- **Handlers** - Handlers define the code to execute when a specified error or condition occurs and are declared using the `CREATE HANDLER` statement.
- **Conditions** - Conditions are codes that indicate whether an operation was successful and, if unsuccessful, the type of problem encountered. These are either the `SQLSTATE` codes built into MySQL or the custom conditions you specify.
- **Custom conditions** - Custom conditions allow you to handle issues within your application logic that MySQL would not otherwise consider problems.

21.2 Deciphering SQLSTATE codes

When MySQL encounters an error while executing a statement, it returns a `SQLSTATE` result code identifying the problem type. Otherwise, MySQL returns a code indicating that the operation was successful.

`SQLSTATE` codes consist of five characters and are grouped using specific character sequences. The first two characters of a `SQLSTATE` code designate the class. A return value that begins with 00 indicates success, 01 a warning, and 02 a result that contains no data. All other class codes indicate that an exception occurred.

The remaining four characters represent the `SQLSTATE` subclass and identify the specific problem. For example, `SQLSTATE` code 00000 is returned when a task is successfully completed. Code 01002, on the other hand, is a warning that indicates a connectivity issue between the client and server, while code 46121 is an exception caused by an invalid column name reference.

The `SQLSTATE` classes are further divided into one of four categories: success (S), warning (W), and exception (X). For a complete list of the `SQLSTATE` codes used by MySQL, refer to the following web page:

<https://en.wikipedia.org/wiki/SQLSTATE>

21.3 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the "Start Here" chapter.

Error Handling in MySQL

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampled;"  
mysql -u root -p sampled < mysql_error_handling.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampled;
```

21.4 Declaring a handler

The DECLARE HANDLER statement allows you to define an action for a particular error or condition. MySQL provides two types of handlers:

- **CONTINUE** - A continue handler allows the code to execute at the following statement after handling the error. Only use this option for non-critical conditions that do not prevent subsequent statements from completing successfully.
- **EXIT** - An EXIT handler stops the routine's execution after handling the error. This is typically used for critical errors where proceeding further could lead to incorrect results, data corruption, or other unintended consequences.

The syntax for creating a handler is as follows:

```
DECLARE <handler_type> HANDLER FOR <condition>  
    <sql_statement>;
```

When using the above syntax to declare a handler, you will need to provide the following values:

- **handler_type**: Defines how to proceed after handling the error (CONTINUE or EXIT).
- **condition**: The type of error or condition to handle (SQLEXCEPTION, SQLWARNING, NOT FOUND, or SQLSTATE).
- **statement**: The SQL statement to be executed when the condition occurs.

To execute multiple statements within the handler body, enclose them in BEGIN and END statements:

```
DECLARE <handler_type> HANDLER FOR <condition>  
BEGIN  
    <sql_statements>  
END;
```

21.5 Error handling with SQLEXCEPTION

The SQLEXCEPTION condition type catches all exception errors that occur within a routine. In an earlier chapter, we created a procedure to calculate factorials. Let's create a similar procedure that we can use to demonstrate exception handling. Within the MySQL client, enter the following statements:

```
DELIMITER //  
  
CREATE PROCEDURE SafeFactorial(IN num INT)  
BEGIN  
    DECLARE result INT DEFAULT 1;  
  
    REPEAT
```

```

        SET result = result * num;
        SET num = num - 1;
    UNTIL num = 0
    END REPEAT;

    SELECT CONCAT('Factorial = ', result) AS Result;
END //

DELIMITER ;

```

Begin by calling the SafeFactorial() procedure with a valid parameter:

```

CALL SafeFactorial(10);
+-----+
| Result |
+-----+
| Factorial = 3628800 |
+-----+
1 row in set (0.00 sec)

```

The output shows that the procedure completed the calculation and returned a result. One shortcoming of factorials is that they cannot handle negative numbers. To experience the effect of calculating the factorial of a negative number, run the following command:

```

CALL safefactorial(-10);
ERROR 1264 (22003): Out of range value for column 'result' at row 1

```

The error message describes the cause of the problem together with the corresponding SQLSTATE code (22003). From the first two characters (22), we can tell that this is an exception error, and if we look it up in the Wikipedia SQLSTATE list provided earlier, we will find the following entry:

SQLSTATE	Category	Class	Class Text	Subclass	Subclass Text
22003	X	22	data exception	003	numeric value out of range

Figure 21-1

If the factorial calculation occurred within the code of an application client, we would need to catch this error and take remedial action to prevent the application from crashing. To do so, we must add a SQLEXCEPTION handler to the SafeFactorial() procedure. Implement this change by dropping the current procedure and redeclaring it as follows:

```

DROP PROCEDURE SafeFactorial;

DELIMITER //

CREATE PROCEDURE SafeFactorial(IN num INT)
BEGIN

    DECLARE result INT DEFAULT 1;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        SELECT 'Statement to handle the error goes here' AS ErrorMessage;

    REPEAT
        SET result = result * num;

```

Error Handling in MySQL

```
        SET num = num - 1;
    UNTIL num = 0
    END REPEAT;

    SELECT CONCAT('Factorial = ', result) AS Result;
END //

DELIMITER ;
```

Future attempts to call the procedure with a negative value will execute the handler statement and exit:

```
CALL safefactorial(-10);
+-----+
| ErrorMessage |
+-----+
| Statement to handle the error goes here |
+-----+
1 row in set (0.00 sec)
```

The SafeFactorial() procedure currently uses an EXIT handler to terminate the procedure after handling the error. Converting the handler to use CONTINUE instead of EXIT will demonstrate the risks of allowing execution to continue after an error has occurred. Drop and redeclare the procedure as follows:

```
DROP PROCEDURE SafeFactorial;

DELIMITER //

CREATE PROCEDURE SafeFactorial(IN num INT)
BEGIN

    DECLARE result INT DEFAULT 1;

    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
        SELECT 'Statement to handle the error goes here' AS ErrorMessage;

    REPEAT
        SET result = result * num;
        SET num = num - 1;
    UNTIL num = 0
    END REPEAT;

    SELECT CONCAT('Factorial = ', result) AS Result;
END //

DELIMITER ;
```

Call SafeFactorial() with a negative number once more and note that although the error message appears, control does not return to the mysql> prompt. The problem is that allowing the execution to continue following the error has caused the REPEAT statement to loop infinitely. In other words, our SafeFactorial() procedure is no longer “safe.”

The only way to exit from the procedure is to type Ctrl-C on the keyboard, an option that would not be available if a client application had called this routine:

```
CALL SafeFactorial(-10);
+-----+
```



```
| ErrorMessage |
+-----+
| Statement to handle the error goes here |
+-----+
1 row in set (0.01 sec)

^C -- query aborted
ERROR 1317 (70100): Query execution was interrupted
```

In summary, only use CONTINUE handlers when doing so will not lead to critical issues in subsequent code.

21.6 Error handling with SQLWARNING and NOT FOUND

A SQLEXCEPTION condition handler will catch errors that are categorized as exceptions. The same technique can be used to create handlers specifically for warning (01) and no data (02) code classes. The following handler example will display a custom message for no data (NOT FOUND) class codes:

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
    SELECT 'No matching record found' AS ErrorMessage;
```

Conditions may also be combined when creating a handler. The following handler catches exceptions and warnings:

```
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION, SQLWARNING
    SELECT 'Warning or No data detected' AS ErrorMessage;
```

21.7 Handling specific errors with SQLSTATE

While adequate for many situations, the above examples handle errors based solely on the code class (exceptions, warnings, or no data). However, this “catch-all” approach doesn’t allow different actions to be performed based on the code subclass or to ignore specific exception codes. For instance, it allows us to handle all exceptions but not to differentiate between a divide by zero exception (22012) and a client-server connection failure (08006). This distinction may be critical in deciding the actions to handle the problem.

To catch and handle specific types of error, we must use SQLSTATE handler conditions, the syntax for which is as follows:

```
DECLARE [CONTINUE | EXIT] HANDLER FOR SQLSTATE '<sqlstate_code>'
BEGIN
    <sql_statements>
END;
```

We can declare handlers for multiple conditions in a single routine by targeting specific error codes. The following procedure, for example, performs division and multiplication calculations on two TINYINT values and declares handlers to catch division by zero (22012) and out-of-range value (22003) error codes. The routine continues executing when a division by zero is detected and exits when a value is out of range:

```
DELIMITER //

CREATE PROCEDURE CalcDemo(x TINYINT, y TINYINT)
BEGIN

    DECLARE result TINYINT DEFAULT 1;

    DECLARE CONTINUE HANDLER FOR SQLSTATE '22012'
        SELECT 'Division by zero occurred. Continuing...' AS ErrorMessage;
```

Error Handling in MySQL

```
DECLARE EXIT HANDLER FOR SQLSTATE '22003'
    SELECT 'Out of range value occurred. Exiting...' AS ErrorMessage;

SELECT x / y AS Division_Result;

SET result = x * y;

SELECT result AS Multiplication_Result;
END //
```

DELIMITER ;

An unsigned TINYINT value stores integers ranging from 0 to 255, so calling CalcDemo() with small, non-zero parameter numbers produces both division and multiplication results:

```
CALL CalcDemo(10, 2);
+-----+
| Division_Result |
+-----+
|          5.0000 |
+-----+
1 row in set (0.01 sec)

+-----+
| Multiplication_Result |
+-----+
|                   20 |
+-----+
1 row in set (0.01 sec)
```

The following CalcDemo() call will trigger the division by zero handler, display the corresponding message, and then continue execution to the multiplication:

```
CALL CalcDemo(10, 0);
+-----+
| Division_Result |
+-----+
|          NULL |
+-----+
1 row in set (0.01 sec)

+-----+
| ErrorMessage |
+-----+
| Division by zero occurred. Continuing... |
+-----+
1 row in set (0.01 sec)

+-----+
| Multiplication_Result |
+-----+
|                   0 |
+-----+
1 row in set (0.00 sec)
```

Calling `CalcDemo()` with values that exceed 255 when multiplied will divide successfully but cause an out-of-range value exception. The out-of-range handler will output the corresponding message and exit the routine:

```
CALL CalcDemo(100, 10);
+-----+
| Division_Result |
+-----+
|          10.0000 |
+-----+
1 row in set (0.00 sec)

+-----+
| ErrorMessage
+-----+
| Out of range value occurred. Exiting... |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

21.8 Custom error handling with SIGNAL

So far, we have looked at handling conditions generated internally by the server. MySQL also provides a way to create custom conditions to handle errors that may occur in the logic of our own code. We might, for example, create a custom handler to validate a value based on criteria that would not be considered an error by MySQL.

Consider a database table that stores product pricing and discount information. While the table is declared such that any integer value can be stored in the discount column, the application's business logic dictates that discounts must not exceed 20%. MySQL allows us to create custom conditions consisting of a SQLSTATE code and error message text that can be invoked using the SIGNAL statement. The syntax for throwing a custom error is as follows:

```
SIGNAL SQLSTATE '<sqlstate_code>'
SET MESSAGE_TEXT = '<message_text>';
```

The SQLSTATE code provided to the SIGNAL statement can be an existing code (ideally one that is similar to the custom error type) or a custom code consisting of five alphanumeric characters. The following routine uses a custom error code to validate a discount value parameter:

```
DELIMITER //

CREATE PROCEDURE ValidateDiscount(discount INT)
BEGIN
    IF discount > 20 THEN
        SIGNAL SQLSTATE 'MNOPO'
        SET MESSAGE_TEXT = 'Discount cannot exceed 20%';
    END IF;

    SELECT 'Discount accepted' AS Status;
END //

DELIMITER ;
```

Use the MySQL client to declare the above procedure, then call it with valid and invalid discount values:

Error Handling in MySQL

```
CALL ValidateDiscount(15);
+-----+
| Status          |
+-----+
| Discount accepted |
+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

CALL ValidateDiscount(30);
ERROR 1644 (MNOPO): Discount cannot exceed 20%
```

If you do not intend to use the routines from this chapter in the future, delete them using the following statements:

```
DROP PROCEDURE CalcDemo;
DROP PROCEDURE ValidateDiscount;
DROP PROCEDURE SafeFactorial;
```

21.9 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL error handling:

<https://www.answertopia.com/llis>



21.10 Reference points

The main points covered in this chapter are as follows:

• Overview

- Errors in MySQL may arise from invalid data, logic errors, or miscalculations.
- Errors can terminate execution and leave the database in an indeterminate state.
- Defensive coding and MySQL's error-handling features enable graceful responses to errors.

• Key Concepts

- Handlers: Define actions to execute when specific errors or conditions occur.
- Conditions: Indicate operation success, failure, or specific issues using MySQL's `SQLSTATE` codes.
- Custom Conditions: Allow handling of application-specific logic errors not inherently flagged by MySQL.

• SQLSTATE Codes

- Five-character codes indicating operation status:
 - 00: Success.
 - 01: Warning.
 - 02: No data.
- Other codes indicate exceptions.

- Example: 22003 indicates “numeric value out of range.”

- **Declaring a Handler**

- Declare handlers using the following syntax:

```
DECLARE <handler_type> HANDLER FOR <condition>  
    <sql_statement>;
```

- **Handler Condition Types**

- CONTINUE: Continue execution after handling the error.
- EXIT: Stop execution after handling the error.

- **Error Handling for Specific Conditions**

- SQLWARNING: Catches warnings (01 class codes).
- NOT FOUND: Catches “no data” conditions (02 class codes).
- SQLEXCEPTION: Catches all exception errors.
- Specific SQLSTATE Codes:
 - Example: Handling division by zero (22012) or out-of-range values (22003).

- **Custom Error Handling**

- Create custom conditions for application-specific errors.
- Trigger errors using SIGNAL statement:

```
SIGNAL SQLSTATE '<sqlstate_code>'  
    SET MESSAGE_TEXT = '<message_text>;'
```

- **Best Practices**

- Use EXIT handlers for critical errors to prevent incorrect results or infinite loops.
- Use CONTINUE handlers cautiously to avoid cascading failures.
- Combine specific error codes in handlers for fine-grained error management.

22. Automation with MySQL Triggers

When working with databases, operations often need to be performed in response to table data changes. For instance, if a customer is deleted from one table, their purchase history may need to be automatically removed from the table that contains customer orders. While it would, theoretically, be possible to write code in a stored routine to achieve this automation, it is far more efficient to use MySQL triggers.

This chapter introduces MySQL triggers, explains how to use them, and discusses their limitations.

22.1 An overview of MySQL triggers

Triggers automatically execute blocks of SQL code in response to changes to a database table, such as deleting, inserting, or updating rows. These triggers are configured to execute before or after an event occurs. For instance, you could use a trigger to validate column values before a row is inserted into a table or to add a row to an audit log table after a row has been deleted.

22.2 The syntax for creating triggers

Triggers are declared in MySQL using the CREATE TRIGGER statement combined with clauses defining the circumstance under which the trigger will activate. The CREATE TRIGGER statement uses the following syntax:

```
CREATE TRIGGER <trigger_name>
  [BEFORE | AFTER] [INSERT | UPDATE | DELETE]
  ON <table_name>
  FOR EACH ROW
  BEGIN
    <trigger_body>
  END;
```

The elements of the above syntax can be summarized as follows:

- **trigger_name** - The name of the trigger. This name must be unique within the scope of the database.
- **BEFORE | AFTER** - Defines whether the trigger execution occurs before or after the event.
- **INSERT | UPDATE | DELETE** - Specifies the event that will activate the trigger.
- **table_name** - The table on which the specified event will activate the trigger.
- **FOR EACH ROW** - Indicates that the trigger operates at the row level.
- **trigger_body** - The SQL statements to execute when the trigger is activated.

22.3 Accessing trigger event data

Triggers activate in response to changes in table data and execute the SQL code in the trigger body in response. A typical trigger body will act based on the data changes that activated the trigger. To continue our previous example, a trigger to log row deletions might need to include information about the deleted row. In the case of an UPDATE operation, the trigger body may need access to both the original and replacement data values. In the trigger code, access to the event data is available via the OLD and NEW trigger extension variables using dot notation to reference the column name. For instance, consider an UPDATE event that changes the customer name column of a table row. Within the trigger code, the old and new customer_name values would

be referenced as follows:

```
OLD.customer_name  
NEW.customer_name
```

In most cases, the event data within the trigger body is readable and writable. This allows us to identify and change the values before they are applied to the tables via the corresponding insertion or update operation.

22.4 Triggers in action

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the “*Start Here*” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampled;"  
mysql -u root -p sampled < mysql_triggers.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampledb
```

22.4.1 An AFTER INSERT example

As an initial example, we will create a trigger to log when a row is added to the supplier database. The log table will be called *supplier_log* and consist of a primary key, the name of the database user, the supplier name, and the event type and date. Using the MySQL client, create the *supplier_log* table as follows:

```
CREATE TABLE supplier_log (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(35),  
    supplier_name VARCHAR(25),  
    event_type VARCHAR(25),  
    event_date DATE  
);
```

With the supplier log table added to the database, the next step is to create the trigger:

```
DELIMITER //  
  
CREATE TRIGGER after_supplier_insert  
AFTER INSERT ON supplier  
FOR EACH ROW  
BEGIN  
    INSERT INTO supplier_log (username, supplier_name, event_type, event_date)  
    VALUES (USER(), NEW.company, 'INSERT', CURDATE());  
END //  
  
DELIMITER ;
```

The trigger body contains a semicolon delimiter, so we have to temporarily change the delimiter to `//` while entering the trigger code. We use the `USER()` and `CURDATE()` functions within the code body to identify the database user's name and the current date, respectively.

Test the trigger by inserting a new row into the supplier table:


```
INSERT INTO supplier (
    company,
    address)
VALUES (
    'The Big Box',
    'Durham, NC'
);
```

After inserting the row, run the following statement to confirm that the addition was recorded in the log table:

```
SELECT * FROM supplier_log;
+----+-----+-----+-----+-----+
| id | username      | supplier_name | event_type | event_date |
+----+-----+-----+-----+-----+
| 1  | root@localhost | The Big Box   | INSERT     | 2025-01-10 |
+----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

22.4.2 A BEFORE INSERT example

The BEFORE clause performs actions before changes are made to a database table. This allows us to perform actions such as validating or transforming the data before it is written to the table or preventing a deletion from proceeding under specific conditions. The following trigger, for example, converts the address field to uppercase when new records are inserted into the supplier table:

```
DELIMITER //

CREATE TRIGGER before_supplier_insert
BEFORE INSERT ON supplier
FOR EACH ROW
BEGIN
    SET NEW.address = UPPER(NEW.address);
END //

DELIMITER ;
```

Next, add a new row to the supplier table, then run a SELECT statement to check that the address was converted to upper case:

```
INSERT INTO supplier (
    company,
    address
)
VALUES (
    'Acme Networks',
    'boston, ma'
);

SELECT * FROM supplier WHERE company = 'Acme Networks';
+----+-----+-----+
| id | company      | address      |
+----+-----+-----+
| 10 | Acme Networks | BOSTON, MA   |
+----+-----+-----+
1 row in set (0.00 sec)
```

22.4.3 A BEFORE DELETE example

The BEFORE DELETE trigger is useful for ensuring that certain conditions are met before rows are deleted from a table. The following trigger uses a SIGNAL statement to prevent rows from being deleted from the supplier table:

```
DELIMITER //
```

```
CREATE TRIGGER before_supplier_delete
BEFORE DELETE ON supplier
FOR EACH ROW
BEGIN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Deletions are not allowed in the supplier table';
END //
```

```
DELIMITER ;
```

Test the trigger by attempting to delete the Acme Networks row from the supplier table:

```
DELETE FROM supplier WHERE company LIKE 'Acme Networks';
ERROR 1644 (45000): Deletions are not allowed in the supplier table
```

22.5 Displaying trigger information

Information about the triggers defined within a database can be displayed using the SHOW TRIGGERS statement. The following statement lists all of the active triggers for the current database:

```
SHOW TRIGGERS\G
***** 1. row *****
      Trigger: before_supplier_delete
      Event: DELETE
      Table: supplier
      Statement: BEGIN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Deletions are not allowed in the supplier table';
      END
      Timing: BEFORE
      Created: 2025-01-16 09:51:32.30
      sql_mode: ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_
DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION
      Definer: root@localhost
      character_set_client: utf8mb4
      collation_connection: utf8mb4_0900_ai_ci
      Database Collation: utf8mb4_0900_ai_ci
      .
      .
```

Note the use of the \G modifier in the above example. The result set generated by the SHOW TRIGGERS statement contains wide output lines that are best viewed using vertical formatting.

To limit the list of triggers to specific tables, combine the SHOW TRIGGERS statement with the LIKE clause, using either the full table name or wildcard matching to select multiple tables, for example:

```
SHOW TRIGGERS LIKE 'suppl%' \G
```

22.6 Deleting triggers

Triggers are deleted using the DROP TRIGGER statement, the syntax for which is as follows:

```
DROP TRIGGER <trigger_name>;
```

To avoid unexpected behavior later, run the following statements to remove the supplier_log table and the triggers created in this chapter:

```
DROP TABLE supplier_log;
DROP TRIGGER after_supplier_insert;
DROP TRIGGER before_supplier_insert;
DROP TRIGGER before_supplier_delete;
```

22.7 Limitations of triggers

Before learning about MySQL events in the next chapter, we need to outline the limitations of MySQL triggers, beginning with recursion. Triggers are non-recursive, meaning that they cannot activate themselves. Triggers also cannot start, commit, or rollback transactions. Finally, excessive use of triggers on databases with high transaction rates can result in performance degradation. Try to avoid using triggers as a “quick fix” for a problem that could be addressed more efficiently elsewhere.

22.8 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL triggers:

<https://www.answerstopia.com/k5af>



22.9 Reference points

The main points covered in this chapter are as follows:

- **Overview**

- Triggers automatically execute SQL code in response to changes in a table (INSERT, UPDATE, DELETE).
- More efficient than using stored routines for automation.

- **Creating Triggers**

- Syntax:

```
CREATE TRIGGER <trigger_name>
  [BEFORE | AFTER] [INSERT | UPDATE | DELETE]
  ON <table_name>
  FOR EACH ROW
  BEGIN
    <trigger_body>
  END;
```

- **Trigger Components**

- BEFORE | AFTER: Defines when the trigger runs relative to the event.
- INSERT | UPDATE | DELETE: Specifies the event type.

- `FOR EACH ROW`: Indicates the trigger operates at the row level.

• Accessing Trigger Event Data

- `OLD.column_name`: Accesses previous row values before an update or delete.
- `NEW.column_name`: Accesses new values being inserted or updated.

• Managing Triggers

- Show all triggers in the current database:

```
SHOW TRIGGERS\G
```

- Show triggers for a specific table:

```
SHOW TRIGGERS LIKE 'suppl%'\G
```

• Deleting Triggers

- Remove a trigger using:

```
DROP TRIGGER <trigger_name>;
```

• Limitations of Triggers

- Triggers are **non-recursive** (they cannot activate themselves).
- Cannot start, commit, or rollback transactions.
- Excessive use in high-transaction databases may impact performance.
- Avoid using triggers as a shortcut for problems that could be solved more efficiently elsewhere.

23. Scheduling Tasks with MySQL Events

The previous chapter introduced MySQL triggers and explored how they execute code automatically in response to table data changes. While triggers help automate tasks, they are not designed to handle scheduled tasks or operations that do not depend on changes in table data. MySQL events fill this gap by allowing us to schedule repetitive tasks like data cleanups, backups, or routine updates based on dates, times, and intervals.

This chapter will introduce MySQL events, explain how they work, and provide practical examples.

23.1 An introduction to MySQL events

MySQL events execute SQL statements at specified times or intervals and typically automate tasks such as generating reports, deleting old data and logs, and monitoring and logging database activity.

Unlike triggers, which respond to table operations such as INSERT, UPDATE, DELETE, events are time-based and execute independently of any specific database activity or user actions. Figure 23-1 outlines the difference between triggers and events:

Feature	Triggers	Events
Activation	Execution in response to table events (INSERT, UPDATE, DELETE).	Execution based on a time schedule.
Scope	Applies to specific tables.	Operates at the database level.
Timing	Triggered immediately during or after a table operation.	Executes at a specified time or interval.

Figure 23-1

23.2 Enabling the MySQL event scheduler

Events are controlled, managed, and executed by the MySQL event scheduler process, which can be turned on and off via the *event_scheduler* global variable. The event scheduler is activated by default when MySQL is installed, and you can check the current status using the following statement:

```
SHOW VARIABLES LIKE 'event_scheduler';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| event_scheduler | ON    |
+-----+-----+
1 row in set (0.06 sec)
```

If the event scheduler is off, use the following statement to enable it before proceeding with the examples in this chapter:

```
SET GLOBAL event_scheduler = ON;
```

23.3 Creating Events

Events are created using the CREATE EVENT statement, the basic syntax for which is as follows:

```
CREATE EVENT <event_name>
ON SCHEDULE [AT <timestamp> | EVERY <interval>]
DO
    <sql_statement>;
```

The elements of the above syntax can be summarized as follows:

- **event_name** - The event name which must be unique within the database scope.
- **ON SCHEDULE** - Defines when the event will execute using one of the following options:
 - **AT <timestamp>** - Schedules the event to run once at a specific date and time.
 - **EVERY <interval>** - Schedules the event to repeat at regular intervals (e.g., every hour).
- **DO** - The SQL statement or block of statements to execute.

Note that the BEGIN and END keywords must be used when executing more than one statement in the DO block:

```
CREATE EVENT <event_name>
ON SCHEDULE [AT <timestamp> | EVERY <interval>]
DO
BEGIN
    <sql_statements>
END;
```

23.4 Opening the sample database

This chapter assumes that you have completed the steps in the previous chapters. If you haven't, you can import the current database snapshot using the sample files provided in the “*Start Here*” chapter.

To import the snapshot, open a terminal or command prompt, navigate to the directory containing the sample files, and run the following commands:

```
mysql -u root -p -e "CREATE DATABASE sampled;"
mysql -u root -p sampled < mysql_events.sql
```

Once the database is ready, open the MySQL client and select the *sampledb* database:

```
USE sampledb
```

23.5 Adding the event_tests table

Before we look at some event examples, we need to add a table called event_tests to our sampledb database. This table will consist of id, description, and timestamp columns and will be used to verify that the example events are working. Use the statement below to create the table:

```
CREATE TABLE event_tests
(
    id int NOT NULL AUTO_INCREMENT,
    description char(20) NOT NULL,
    timestamp TIMESTAMP NOT NULL,
    PRIMARY KEY (id)
```

```
);
```

23.6 Creating a one-time event

One-time events are the simplest event type and are created by specifying when the code must execute using the following `TIMESTAMP` format:

```
YYYY-MM-DD hh:mm:ss
```

For example, to schedule an event to occur at 2:35pm on January 16, 2027, the `CREATE EVENT` statement might read as follows:

```
CREATE EVENT do_something
ON SCHEDULE AT '2027-01-16 14:35:00'
DO
    <sql_statement>;
```

While it is possible to use the timestamp format to schedule an event, it is also a common requirement to schedule a one-time event relative to the current date and time. For example, we might need to schedule an event to execute in one hour, at the same time tomorrow, or at this time exactly one week, month, or year in the future. While these calculations may seem daunting, they can easily be performed by referencing the `CURRENT_TIMESTAMP` variable and adding an `INTERVAL`:

```
CURRENT_TIMESTAMP + INTERVAL <number> <time_unit>
```

In the above syntax, `CURRENT_TIMESTAMP` contains the system data and time of the server on which MySQL is running, `<time_unit>` is the unit of time (`MINUTE`, `HOUR`, `DAY`, `WEEK`, `MONTH`, etc.), and `<number>` is the number of time units to be added.

The following statement, for example, schedules an event to occur in two minutes. When the event triggers, it inserts a row into the `event_tests` table consisting of a description and the date and time the event executed:

```
CREATE EVENT one_time_demo
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 2 MINUTE
DO
    INSERT INTO event_tests VALUES (
        NULL,
        'One-time event',
        CURRENT_TIMESTAMP()
    );
```

Run the above statement and wait for two minutes before checking the `event_tests` table:

```
SELECT * FROM event_tests;
+----+-----+-----+
| id | description | timestamp |
+----+-----+-----+
| 1  | One-time event | 2025-01-16 13:35:00 |
+----+-----+-----+
1 row in set (0.04 sec)
```

23.7 Creating recurring events

A recurring event repeats at specified intervals until it is canceled or, if one is provided, an end date is reached. By default, the first recurring event will trigger after the specified time interval has elapsed. Alternatively, scheduling can be deferred by providing a start timestamp. The `CREATE EVENT` statement uses the following syntax to declare recurring events:

Scheduling Tasks with MySQL Events

```
CREATE EVENT <event_name>
ON SCHEDULE EVERY <number> <time_unit>
STARTS <start_timestamp>
ENDS <end_timestamp>
DO
    <sql_statement>;
```

Use the statement below to create a recurring event that adds a row to the event_tests table every 2 minutes until 10 minutes have elapsed:

```
CREATE EVENT recurring_demo
ON SCHEDULE EVERY 2 MINUTE
ENDS CURRENT_TIMESTAMP + INTERVAL 10 MINUTE
DO
    INSERT INTO event_tests VALUES (
        NULL,
        'Recurring event',
        CURRENT_TIMESTAMP()
    );
```

Run the following statement occasionally to verify that records are being added to event_tests table by the recurring event and that insertions stop after 10 minutes:

```
SELECT * FROM event_tests;
+----+-----+-----+
| id | description      | timestamp                |
+----+-----+-----+
| 1  | One-time event   | 2025-01-16 14:21:24 |
| 2  | Recurring event  | 2025-01-16 14:50:42 |
| 3  | Recurring event  | 2025-01-16 14:52:42 |
| 4  | Recurring event  | 2025-01-16 14:54:42 |
| 5  | Recurring event  | 2025-01-16 14:56:42 |
| 6  | Recurring event  | 2025-01-16 14:58:42 |
+----+-----+-----+
6 rows in set (0.00 sec)
```

23.8 Viewing event details

Details of active events can be viewed using the SHOW EVENTS statement as outlined below:

```
SHOW EVENTS\G
***** 1. row *****
      Db: sampled
      Name: recurring_demo
      Definer: root@localhost
      Time zone: SYSTEM
      Type: RECURRING
      Execute at: NULL
      Interval value: 2
      Interval field: MINUTE
      Starts: 2025-01-16 14:50:42
      Ends: 2025-01-16 15:00:40
      Status: ENABLED
      Originator: 1
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
```



```
Database Collation: utf8mb4_0900_ai_ci
1 row in set (0.03 sec)
```

To display information about a specific event, use the `SHOW CREATE EVENT` statement, referencing the event name:

```
SHOW CREATE EVENT one_time_demo\G
***** 1. row *****
      Event: one_time_demo
      sql_mode: ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_
DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION
      time_zone: SYSTEM
      Create Event: CREATE DEFINER='root'@'localhost' EVENT 'one_time_demo'
ON SCHEDULE AT '2025-01-16 15:05:55' ON COMPLETION NOT PRESERVE ENABLE DO
INSERT INTO event_tests VALUES (
      NULL,
      'One-time event',
      CURRENT_TIMESTAMP()
)
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
Database Collation: utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

23.9 Preserving events

Unless instructed otherwise, the scheduler will delete events after they finish executing. As a demonstration, recreate the `one_time_demo` event and use the `SHOW EVENTS` statement to verify it is running. Listing the events after the 2-minute interval has elapsed will show that the event has been deleted.

To retain an event for future use, the `CREATE EVENT` statement must include the `ON COMPLETION` clause with the `PRESERVE` option. Use the statement below to recreate the `one_time_demo` event with preservation enabled:

```
CREATE EVENT one_time_demo
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 2 MINUTE
ON COMPLETION PRESERVE
DO
  INSERT INTO event_tests VALUES (
    NULL,
    'One-time event',
    CURRENT_TIMESTAMP()
  );
```

After the event finishes, it will be disabled and preserved by the event scheduler:

```
SHOW EVENTS\G
***** 1. row *****
      Db: sampled
      Name: one_time_demo
      Definer: root@localhost
      Time zone: SYSTEM
      Type: ONE TIME
      Execute at: 2025-01-17 09:38:35
      Interval value: NULL
```

```
Interval field: NULL
Starts: NULL
Ends: NULL
Status: DISABLED
Originator: 1
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
Database Collation: utf8mb4_0900_ai_ci
1 row in set (0.01 sec)
```

Two event properties need to be changed to reuse this event. The first issue is that the event is currently disabled, so it needs to be enabled before it will run again. In addition, the event is configured to execute at a date and time that is now in the past. Therefore, in addition to enabling the event, we will also need to schedule it to run at a date and time in the future. These and many other event properties can be changed using the ALTER EVENT statement.

23.10 Altering event properties

The ALTER EVENT statement allows us to modify the properties of existing events using similar syntax to the CREATE EVENT statement. Using ALTER EVENT, we can perform tasks such as enabling, disabling, and renaming events, changing scheduling settings, turning preservation on and off, and replacing the SQL statements in the DO code block.

Below is the basic syntax used by the ALTER EVENT statement:

```
ALTER EVENT <event_name>
ON SCHEDULE [AT <timestamp> | EVERY <interval>]
ON COMPLETION [NOT] PRESERVE
RENAME TO <new_event_name>
[ENABLE | DISABLE | DISABLE ON {REPLICA | SLAVE}]
COMMENT '<comment_string>'
DO
    <sql_statement>;
```

When using ALTER EVENT, we only need to reference the event properties that are changing. Properties that are not referenced in the statement will retain their current values.

Continuing our previous example, we can re-enable our one_time_demo event with a 1-minute interval using the following ALTER EVENT statement:

```
ALTER EVENT one_time_demo
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 1 MINUTE
ENABLE;
```

In addition to re-enabling preserved events, changes can be made to running events. To see this in action, use the following statement to re-create a slightly modified version of our recurring_demo event:

```
CREATE EVENT recurring_demo
ON SCHEDULE EVERY 1 MINUTE
ENDS CURRENT_TIMESTAMP + INTERVAL 10 MINUTE
ON COMPLETION PRESERVE
DO
    INSERT INTO event_tests VALUES (
        NULL,
        'Recurring event',
        CURRENT_TIMESTAMP())
```

```
);
```

After one minute, run a query on the `event_tests` table to verify that the event is executing:

```
SELECT * FROM event_tests;
```

id	description	timestamp
46	Recurring event	2025-01-17 10:48:37

46 rows in set (0.00 sec)

While the event continues to execute, use the statement below to decrease the interval to 30 seconds and provide a new DO statement:

```
ALTER EVENT recurring_demo
ON SCHEDULE EVERY 30 SECOND
DO
    INSERT INTO event_tests VALUES (
        NULL,
        'Altered event',
        CURRENT_TIMESTAMP()
    );
```

Recheck the test table to verify that the event occurs every 30 seconds and includes the modified description text:

```
SELECT * FROM event_tests;
```

id	description	timestamp
1	One-time event	2025-02-07 14:19:37
2	Recurring event	2025-02-07 14:19:52
3	Recurring event	2025-02-07 14:21:52
46	Recurring event	2025-01-17 10:48:37
47	Altered event	2025-01-17 10:48:46
48	Altered event	2025-01-17 10:49:16
49	Altered event	2025-01-17 10:49:46

49 rows in set (0.00 sec)

23.11 Deleting events

Events are removed from the event scheduler using the `DROP EVENT` statement. With the practical sections of this chapter completed, take this opportunity to delete `event_tests` table and the two example events:

```
DROP EVENT one_time_demo;
DROP EVENT recurring_demo;
DROP TABLE event_tests;
```

23.12 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL events:

<https://www.answertopia.com/yzqq>



23.13 Reference points

The main points covered in this chapter are as follows:

- **Overview**

- MySQL events allow tasks to be scheduled for automatic execution at predefined times.
- Used for automating repetitive processes such as backups, data cleanups, and routine updates.

- **Enabling the MySQL Event Scheduler**

- The MySQL event scheduler is responsible for executing scheduled events.
- Checking and modifying its status ensures that scheduled tasks run as expected.

```
SHOW VARIABLES LIKE 'event_scheduler';  
SET GLOBAL event_scheduler = ON;
```

- **Creating MySQL Events**

- Events are scheduled either for a specific time or at recurring intervals.
- Events can be configured to run once or to repeat at regular intervals such as minutes, hours, or days.

- **One-Time Events**

- These events execute only once at a specified date and time.
- By default, they are removed after execution unless explicitly preserved.
- Syntax:

```
CREATE EVENT event_name  
ON SCHEDULE AT 'YYYY-MM-DD HH:MM:SS'  
DO  
    SQL_statement;
```

- **Recurring Events**

- Recurring events are designed to execute repeatedly at defined time intervals.
- They continue running indefinitely or until a specified end time is reached.
- Syntax:

```
CREATE EVENT event_name  
ON SCHEDULE EVERY interval TIME_UNIT  
DO  
    SQL_statement;
```

- **Viewing and Managing Events**

- The list of scheduled events can be viewed using the `SHOW EVENTS` statement.
- View details of a specific using `SHOW CREATE EVENT` statement.
- Events can be modified, enabled, or disabled without deleting them.

- **Preserving and Altering Events**

- Events are deleted after execution unless `ON COMPLETION PRESERVE` is specified.
- Preserved events can be re-enabled and rescheduled to run at a future time.
- Modify an existing event using `ALTER EVENT`.

- **Disabling and Deleting Events**

- Disabling an event prevents it from running while keeping it stored in the database:

```
ALTER EVENT event_name DISABLE;
```

- Deleting an event permanently removes it from the system:

```
DROP EVENT event_name;
```

- **Triggers vs. Events**

- Triggers execute in response to table modifications (`INSERT`, `UPDATE`, `DELETE`), while events execute based on a time schedule.
- Triggers apply to specific tables, whereas events operate at the database level.
- Triggers execute immediately after a table modification, while events run at predefined intervals.

24. An Introduction to `mysqladmin`

Administering a MySQL database involves many tasks, including monitoring server status, managing users, and maintaining database tables. Although several tools, such as MySQL Workbench and phpMyAdmin, allow you to administer MySQL using a graphical user interface, tools like *mysqladmin* provide a quick and convenient way to perform tasks from the command line. The `mysqladmin` tool can also be called from within scripts to automate administrative tasks.

In this chapter, we will introduce `mysqladmin` and explain how you can use it to administer MySQL databases.

24.1 Setting up `mysqladmin`

The `mysqladmin` tool is provided as part of the MySQL Server installation and will be located in the same directory as the `mysql` client tool. You can verify that `mysqladmin` is included in your path by opening a terminal or command-prompt window and typing the following command:

```
mysqladmin --version
```

If `mysqladmin` is in your path, you will see output similar to the following containing version information:

```
mysqladmin Ver 9.1.0 for macos14 on arm64 (MySQL Community Server - GPL)
```

If the tool is not found, follow the steps in the chapter “*The MySQL Client*” to add the MySQL tools to your PATH environment variable.

Many tasks you will perform using `mysqladmin` will require root user access or a user account with appropriate permissions.

24.2 Basic `mysqladmin` syntax

Unlike the MySQL client, `mysqladmin` is not an interactive tool where you can enter sequences of commands in a shell environment. Instead, `mysqladmin` operates transactionally, meaning you provide instructions directly on the command line when you invoke the tool. Once the task is completed, `mysqladmin` exits and returns you to the command prompt.

The general syntax for using `mysqladmin` is as follows:

```
mysqladmin [options] command [command_options]
```

The command-line argument categories can be summarized as follows:

- **options** - Global settings like the username (-u) and password (-p) for authentication, or the host (-h) for remote server connections.
- **command** - The specific operation you want to perform (e.g., status, shutdown, create).
- **command_options** - Additional parameters specific to the chosen command.

For example, to check the status of the server as the root user, you would run the following command:

```
mysqladmin -u root -p status
```

After the root password has been entered, status information similar to the following will appear:

```
Uptime: 176854  Threads: 2   Questions: 346  Slow queries: 0   Opens: 160   Flush
tables: 3   Open tables: 81   Queries per second avg: 0.001
```

To display a list of all of the command line options supported by mysqladmin, run it with the `--help` argument:

```
mysqladmin --help
```

The help information will make it clear that mysqladmin can perform various administrative tasks. While a detailed overview of all of these capabilities is beyond the scope of this book, we will cover many of the more common uses in the remainder of this chapter.

24.3 Checking server status

Several options are available to check various server status properties. As we saw previously, we can request the overall server status as follows:

```
mysqladmin -u root -p status
Uptime: 177747  Threads: 2   Questions: 348  Slow queries: 0   Opens: 160   Flush
tables: 3   Open tables: 81   Queries per second avg: 0.001
```

The MySQL server operates as a collection of background processes rather than a single entity. These processes start and stop depending on the volume and nature of the incoming requests. A server managing hundreds of requests per minute will have more active background processes than an idle server. However, having too many active processes can affect the server's performance.

You can view the list of currently running processes using the mysqladmin tool with the `processlist` command:

```
mysqladmin -u root -p processlist
+-----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host      | db | Command | Time | State | Info |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 5  | event | localhost |    | Daemon  | 177  | Wait  |      |
| 14 | root  | localhost |    | Query   | 0    | init  | show processlist |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

24.4 Shutting down the server

The server can be stopped from the command line using the `shutdown` option as follows:

```
mysqladmin -u root -p shutdown
```

The above command will gracefully shut down the server and close all active connections. Although the `start` command can be used to launch the server, this is best performed using a service manager such as `systemctl` on Linux or a Windows Service.

24.5 Resetting user passwords

The password assigned to root or any other user can be changed using the mysqladmin `password` command. The following command changes the password for the root account:

```
mysqladmin -u root -p password 'new_password'
```

The tool will prompt for the current password before applying the new one. If you know the current password for a user, you can change it to a new password as follows:

```
mysqladmin -u <username> -p password 'new_password'
```


You will be prompted to enter the user's current password when the above command runs. If you do not know the password, you will need to make the change using the ALTER USER statement, as outlined in the “MySQL Users, Privileges, and Security” chapter.

24.6 Flushing privileges and logs

When making changes to a database configuration, it may be necessary to perform a flush operation before the changes take effect. For example, if user privileges need to be reloaded, run the following command:

```
mysqladmin -u root -p flush-privileges
```

To reload the privilege tables in addition to any other configuration files or settings that need to be refreshed, use the reload command:

```
mysqladmin -u root -p reload
```

Similarly, you can clean up the server log files as follows:

```
mysqladmin -u root -p flush-logs
```

24.7 Creating and dropping databases

To create a new database from the command line, run mysqladmin with the create command:

```
mysqladmin -u root -p create new_database
```

To delete a database, use the following drop command:

```
mysqladmin -u root -p drop new_database
```

24.8 Viewing server variables

To see a list of server variables defined on the server and their values, run mysqladmin as follows:

```
mysqladmin -u root -p variables
```

24.9 Connecting to remote servers

The mysqladmin tool does not need to run on the same system as the MySQL server. In fact, it is more likely that you will use the tool to administer database servers remotely. To manage a MySQL server on a remote system, use the -h option to specify the hostname or IP address:

```
mysqladmin -u root -p -h 133.787.2.101 create new_database
```

24.10 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of the mysqladmin tool:

<https://www.answerstopia.com/iea7>



24.11 Reference points

The main points covered in this chapter are as follows:

- **Overview**

- A command-line tool for administering MySQL databases, enabling tasks like monitoring, managing

users, and maintaining database tables.

- Launched from the command line or used in scripts to automate tasks.
- Provided as part of the MySQL Server installation and located in the same directory as the MySQL client.
- Use `mysqladmin --version` to confirm the tool is installed and accessible in your system's PATH.

- **Setup**

- Many commands require root access or a user account with the necessary privileges.

- **Basic Syntax**

- `mysqladmin [options] command [command_options]`
 - **options:** Authentication or connection settings (e.g., username, password, host).
 - **command:** Specific administrative task (e.g., status, shutdown).
 - **command options:** Additional parameters for the chosen command.
- Executes a command and exits, unlike interactive tools such as the MySQL client.

- **Checking Server Status**

- `mysqladmin -u root -p status`
- Includes uptime, threads, questions, open tables, and queries per second.

- **Viewing Active Processes**

- `mysqladmin -u root -p processlist`
- Lists running processes, their states, and related information.

- **Server Shutdown**

- `mysqladmin -u root -p shutdown`
- Gracefully stops the server and terminates all active connections.

- **Resetting Passwords**

- `mysqladmin -u <username> -p password 'new_password'`
- Requires the current password for verification.
- Use the `ALTER USER` statement if the password is unknown.

- **Flushing Privileges**

- `mysqladmin -u root -p flush-privileges`
- Reloads privilege tables to apply recent changes.

- **Flushing Logs**

- `mysqladmin -u root -p flush-logs`
- Cleans up server log files.

- **Reloading Configuration**

- `mysqladmin -u root -p reload`
- Reloads privilege tables and refreshes server settings.

- **Viewing Server Variables**

- `mysqladmin -u root -p variables`
- Lists server variables and their current values.

- **Remote Administration**

- Use the `-h` option to connect to a MySQL server on a remote host.
- `mysqladmin -u root -p -h <host> create new_database`

- **Additional Commands**

- Help Information: Use `mysqladmin --help` to display all available commands and options.

25. Configuring MySQL using the my.cnf File

MySQL has built-in settings that define the server's behavior and performance. It uses these values by default unless custom configuration settings override them. Custom configuration settings are declared in the MySQL *my.cnf* file, which the MySQL server reads at startup. This chapter will explore the *my.cnf* file and outline some of the main configuration options. Although we will refer to the *my.cnf* file in this chapter, the file is named *my.ini* on Windows systems.

25.1 Introducing the my.cnf File

The *my.cnf* file is the primary configuration file for MySQL, containing settings that modify how the MySQL server performs and operates and is located in one of the following directories, depending on the host operating system and MySQL installation:

- **Linux** - */etc/my.cnf*, */etc/mysql/my.cnf*, or */usr/local/mysql/etc/my.cnf*.
- **macOS** - */etc/my.cnf*, */etc/mysql/my.cnf*, */usr/local/mysql/etc/my.cnf*, or *~/my.cnf*.
- **Windows** - *C:\ProgramData\MySQL\MySQL Server <version>\my.ini*.
- **Custom Installations** - The *my.cnf* file can also be placed in other locations and may be specified on the command line at MySQL startup.

When the MySQL server starts, it will search for the *my.cnf* file in a predefined set of locations. These locations and the order in which they are searched can be identified by running the `mysql --help` command and looking for the following information:

```
Default options are read from the following files in the given order:  
/etc/my.cnf /etc/mysql/my.cnf /usr/local/mysql/etc/my.cnf ~/.my.cnf
```

MySQL uses the built-in default settings if the *my.cnf* file is missing or incorrectly configured.

25.2 The my.cnf File Structure

The *my.cnf* file is divided into sections, each beginning with a section header enclosed in square brackets (`[ ]`). Each section contains configuration directives that apply to a specific MySQL component.

A typical example *my.cnf* file might read in part as follows:

```
[mysqld]  
bind-address = 127.0.0.1  
port = 3306  
datadir = /var/lib/mysql  
socket = /var/lib/mysql/mysql.sock  
max_connections = 200  
query_cache_size = 64M  
innodb_buffer_pool_size = 1G
```

Configuring MySQL using the my.cnf File

```
[client]
port = 3306
socket = /var/lib/mysql/mysql.sock

[mysqldump]
quick
max_allowed_packet = 64M
```

Each section controls different aspects of MySQL:

- **[mysqld]** - MySQL server configuration properties.
- **[client]** - Configures the MySQL client.
- **[mysqldump]** - Configuration settings for the mysqldump utility.

25.3 Key my.cnf parameters

The my.cnf file supports a wide range of parameters that control different aspects of MySQL's behavior. Fortunately, only settings that differ from the default value must be included in the file. Below are some of the more commonly used parameters, grouped by category:

25.3.1 General server settings

These settings define the basic behavior of the MySQL server.

```
[mysqld]
port = 3306
bind-address = 127.0.0.1
max_connections = 200
skip_name_resolve = 1
```

- **port** - Specifies the port MySQL listens on (the default port 3306).
- **bind-address** - Defines the network interface MySQL should listen on (127.0.0.1 restricts access to the local machine).
- **max_connections** - Limits the number of simultaneous client connections.
- **skip_name_resolve** - Disables DNS resolution for client connections, improving performance.

25.3.2 Performance tuning

Performance optimization usually involves adjustments to memory, caching, and query execution settings. The following are examples of performance-related configuration options for the InnoDB storage engine:

```
[mysqld]
query_cache_size = 64M
query_cache_type = 1
innodb_buffer_pool_size = 1G
innodb_log_file_size = 256M
thread_cache_size = 16
```

- **query_cache_size** - Sets the size of the query cache, which stores query results for faster retrieval.
- **query_cache_type** - Enables (1) or disables (0) query caching.
- **innodb_buffer_pool_size** - Defines the memory allocated to InnoDB's buffer pool (should be 50-80% of total server RAM).

- **innodb_log_file_size** - Determines the size of each InnoDB redo log file.
- **thread_cache_size** - Caches client threads to improve performance when handling multiple connections.

25.3.3 Storage engine configuration

MySQL storage engine performance can be optimized through the my.cnf file. The available options are storage engine-specific. The following settings relate to the InnoDB engine:

```
[mysqld]
default_storage_engine = InnoDB
innodb_flush_log_at_trx_commit = 1
innodb_file_per_table = 1
```

- **default_storage_engine** - Defines the default storage engine (InnoDB is the preferred choice for most applications).
- **innodb_flush_log_at_trx_commit** - Controls transaction log flushing.
- **innodb_file_per_table** - Stores each table in a separate .ibd file instead of a shared tablespace.

25.3.4 Logging and debugging

MySQL provides extensive logging options for debugging and monitoring via the following options:

```
[mysqld]
log_error = /var/log/mysql/error.log
slow_query_log = 1
slow_query_log_file = /var/log/mysql/slow.log
long_query_time = 2
```

- **log_error** - Specifies the file where MySQL logs errors.
- **slow_query_log** - Enables slow query logging (1 for enabled, 0 for disabled).
- **slow_query_log_file** - Specifies the file for slow query logs.
- **long_query_time** - Defines the threshold (in seconds) for slow queries.

25.3.5 Security settings

Security is a critical aspect of MySQL configuration. Two key security options are as follows:

```
[mysqld]
skip-networking
secure-file-priv = /var/lib/mysql-files
```

- **skip-networking** - Disables remote connections for added security.
- **secure-file-priv** - Restricts the directory for file operations like LOAD DATA INFILE.

25.3.6 Applying my.cnf file changes

Changes to the my.cnf file will not take effect until the MySQL server is restarted. On Linux, run the following command within a terminal window to perform a server restart:

```
sudo systemctl restart mysql
```

On Windows, open a PowerShell session with administrator privileges and run the following commands where <xx> is replaced by the first two digits of the MySQL version:

Configuring MySQL using the my.cnf File

```
net stop MySQL<xx>
net start MySQL<xx>
```

For example, the commands to stop and start the server for MySQL 9.2 on Windows are as follows:

```
PS C:\> net stop MySQL92
PS C:\> net start MySQL92
```

On macOS, open a terminal window and run the following commands:

```
sudo launchctl \
  unload -F /Library/LaunchDaemons/com.oracle.oss.mysql.mysqld.plist

sudo launchctl \
  load -F /Library/LaunchDaemons/com.oracle.oss.mysql.mysqld.plist
```

Once the server has restarted, connect using the MySQL client and use the SHOW VARIABLES statement to check that any overridden settings are reflected in the running server instance:

```
SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_buffer_pool_size | 334227128 |
+-----+-----+
1 row in set (0.00 sec)
```

25.4 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of the my.cnf file:

<https://www.answerstopia.com/95yo>



25.5 Reference points

The main points covered in this chapter are as follows:

- **Overview**
 - The my.cnf file customizes MySQL server behavior.
 - Settings control general configurations, performance tuning, storage engines, logging, and security.
 - MySQL uses default configuration settings unless overridden by the my.cnf file.
 - The my.cnf file (my.ini on Windows) defines server behavior and performance settings.
- **my.cnf File Locations**
 - **Linux:** /etc/my.cnf, /etc/mysql/my.cnf, /usr/local/mysql/etc/my.cnf
 - **macOS:** /etc/my.cnf, /etc/mysql/my.cnf, /usr/local/mysql/etc/my.cnf, ~/.my.cnf
 - **Windows:** C:\ProgramData\MySQL\MySQL Server <version>\my.ini
- **my.cnf File Structure**

- Configuration sections:
 - **[mysqld]** -> MySQL server settings
 - **[client]** -> MySQL client settings
 - **[mysqldump]** -> mysqldump utility settings
- **Key my.cnf Parameters**
 - **General Server Settings**
 - **port** -> Defines MySQL listening port (default: 3306).
 - **bind-address** -> Restricts server access to specific network interfaces.
 - **max_connections** -> Limits the number of concurrent client connections.
 - **skip_name_resolve** -> Disables DNS lookups for performance improvement.
 - **Performance Tuning**
 - **query_cache_size** -> Caches query results for faster retrieval.
 - **innodb_buffer_pool_size** -> Defines memory allocation for InnoDB (50-80% of RAM).
 - **innodb_log_file_size** -> Defines size of InnoDB transaction log files.
 - **Storage Engine Configuration**
 - **default_storage_engine** -> Defines default table storage engine (InnoDB recommended).
 - **innodb_flush_log_at_trx_commit** -> Ensures ACID compliance (1) or better performance (0/2).
 - **innodb_file_per_table** -> Stores each table in a separate .ibd file.
 - **Logging and Debugging**
 - **log_error** -> Defines error log file location.
 - **slow_query_log** -> Enables logging of slow queries.
 - **long_query_time** -> Defines threshold for slow queries (in seconds).
 - **Security Settings**
 - **skip-networking** -> Disables remote MySQL access.
 - **secure-file-priv** -> Restricts file operations to a secure directory.
- **Applying my.cnf Changes**
 - Restart MySQL for changes to take effect:
 - Linux:


```
sudo systemctl restart mysql
```
 - Windows (PowerShell):


```
net stop MySQL<version>
```

Configuring MySQL using the my.cnf File

```
net start MySQL<version>
```

- **macOS**

```
sudo launchctl \  
  unload -F /Library/LaunchDaemons/com.oracle.oss.mysql.mysqld.plist  
  
sudo launchctl \  
  load -F /Library/LaunchDaemons/com.oracle.oss.mysql.mysqld.plist
```

26. Installing MySQL Workbench

The preceding chapters relied on the text-based MySQL client to enter and execute SQL statements and view results. While this interactive environment offers a quick and lightweight way to perform small database administration tasks, more intuitive and user-friendly options are available. One such option is MySQL Workbench. MySQL Workbench is one of several applications and web-based tools that provide a graphical user interface, allowing many tasks to be performed visually. Before we look at the main features of MySQL Workbench, this chapter will provide the steps to install it on Windows, macOS, and Linux-based systems.

26.1 Downloading MySQL Workbench

MySQL Workbench can be downloaded from the MySQL Developer website at the following URL:

<https://dev.mysql.com/downloads/workbench/>

Use the menu at the top of the download page to select the operating system (marked A in Figure 26-1) and version (B) menus to match your system version and processor architecture. Finally, click the Download button (C) next to the matching installation file:

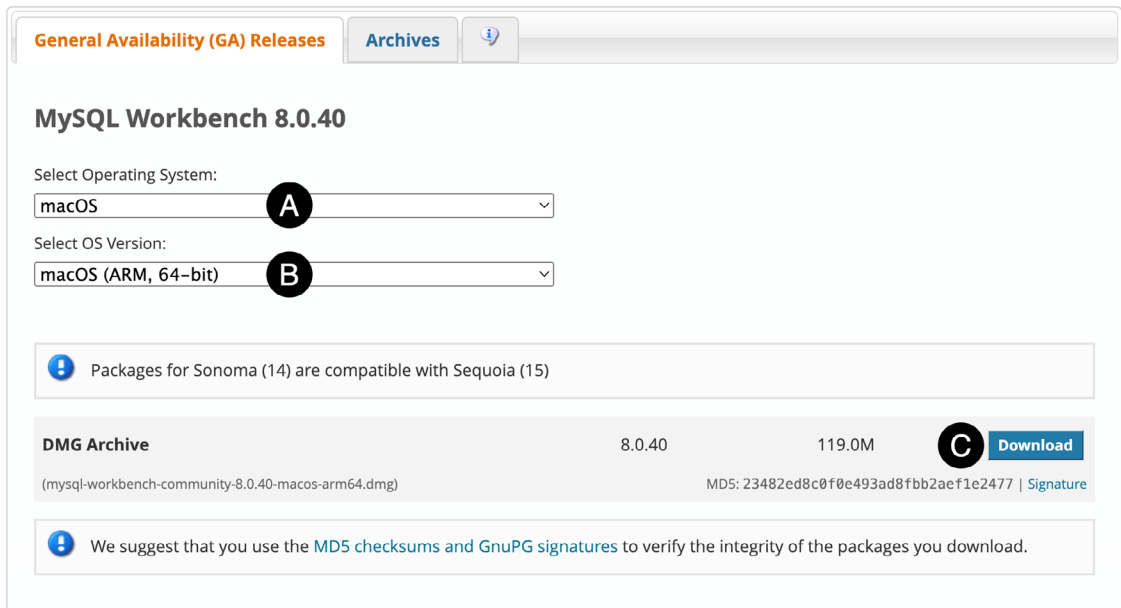


Figure 26-1

When prompted, either sign in with an Oracle web account or select the “No thanks, just start my download” option to proceed anonymously. Finally, save the file to your local filesystem. Once the download is complete, follow the instructions below for your operating system.

26.2 Installation on Windows

MySQL Workbench for Windows is available as a Microsoft Software Installer (.MSI) file. Using Windows Explorer, navigate to where you downloaded the MySQL MSI file and double-click it to begin the installation.

Installing MySQL Workbench

Once the installer has loaded, the screen shown in Figure 26-2 will appear:



Figure 26-2

Click the Next button to review and accept the licensing terms before moving on to the Setup Type screen shown below:

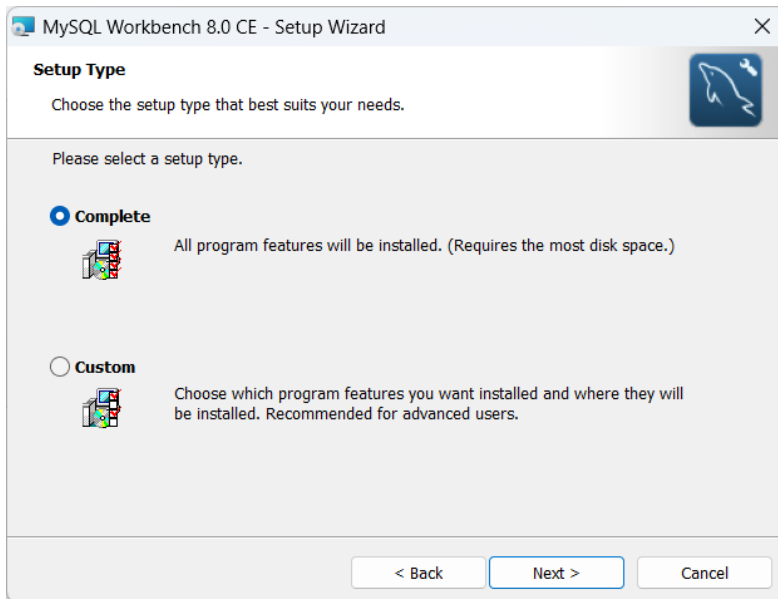


Figure 26-3

After completing the installation, the screen displayed in Figure 26-4 will appear. Ensure the “Launch MySQL Workbench now” option is checked, then click the Finish button to start Workbench:

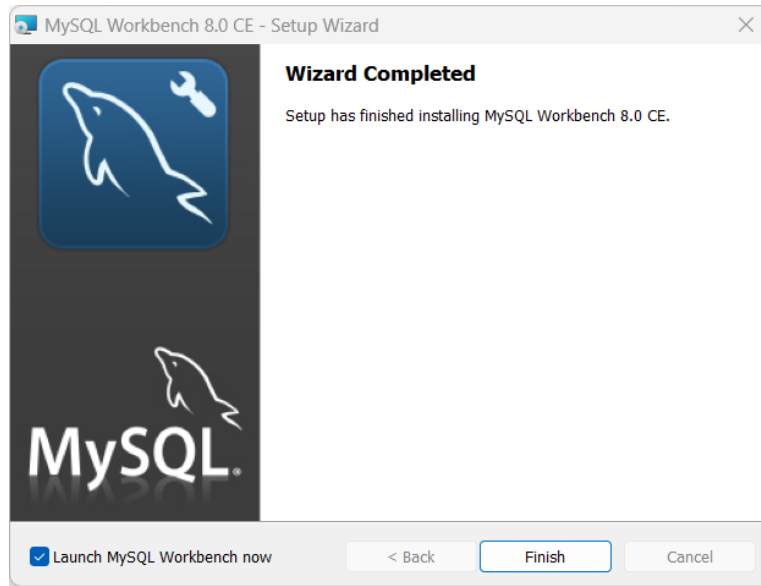


Figure 26-4

26.3 Installation on macOS

To complete the installation on macOS, locate the downloaded disk image (.dmg) file and double-click it to open it in a Finder window:



Figure 26-5

Within the Finder window, select the MySQL Workbench icon and drag it into the Applications folder. To start Workbench, open the macOS Finder and locate it in the Applications folder. Take this opportunity to drag and drop it onto your dock for easier access in the future. Click on the Workbench icon in the dock to launch the tool.

26.4 Installation on RPM-based Linux distributions

MySQL Workbench for Red Hat Linux-based systems (including RHEL, Fedora, CentOS, AlmaLinux, and Rocky Linux) is provided as a Red Hat Package Manager (RPM) file named using the following convention:

Installing MySQL Workbench

```
mysql-workbench-community-<version>.<os>.<arch>.rpm
```

For example, the RPM file for MySQL Workbench 8.0.40 for Red Hat Enterprise Linux 9 on 64-bit x86 systems is named as follows:

```
mysql-community-client-9.1.0-1.el9.x86_64.rpm
```

Before installing MySQL Workbench, we need to install other packages on which it depends. The first steps are enabling the CodeReady Linux Builder (CRB) and installing the Extra Packages for Enterprise Linux (EPEL) repository, which contains several required packages. Open a terminal window and enter the following commands, entering your password when prompted to do so:

```
crb enable
sudo dnf install \
    https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm \
    https://dl.fedoraproject.org/pub/epel/epel-next-release-latest-9.noarch.rpm
```

Next, use the command below to install the package dependencies:

```
dnf install libzip unixODBC mesa-libGL-devel gtk2-2.24.33-8.el9proj
```

To complete the installation, change directory to the location of the downloaded RPM file and run the following command, replacing the `<version>`, `<os>`, and `<arch>` values with those matching your file:

```
sudo rpm -ihv mysql-workbench-community-8.0.40-1.el9.x86_64.rpm
```

Once the installation is complete, execute the following command at the shell prompt:

```
mysql-workbench
```

Alternatively, MySQL Workbench can be launched from within the desktop environment, for example, within the applications screen of the GNOME desktop.

26.5 Installation on Ubuntu

At the time of writing, Ubuntu's latest long-term support (LTS) edition is 24.04, and the current version of MySQL Workbench is 8.0.411. Unfortunately, these two versions are incompatible, so if you are running Ubuntu 24.04 and the latest version of MySQL Workbench is still 8.0.41, you may need to download version 8.0.38 to complete the installation. To download this older version, run the following command in a terminal window:

```
wget https://cdn.mysql.com/Downloads/MySQLGUITools/mysql-workbench-
community_8.0.38-1ubuntu24.04_amd64.deb
```

To begin the installation, change directory to the folder containing the downloaded package file, replacing `<package filename>` with the name of the file you downloaded:

```
sudo dpkg -i <package filename>.deb
```

During the installation, the installer may report dependency problems like the following:

```
dpkg: dependency problems prevent configuration of mysql-workbench-community:
mysql-workbench-community depends on libatkmm-1.6-1v5 (>= 2.28.4); however:
Package libatkmm-1.6-1v5 is not installed.
mysql-workbench-community depends on libglibmm-2.4-1t64 (>= 2.66.7); however:
Package libglibmm-2.4-1t64 is not installed.
mysql-workbench-community depends on libgtkmm-3.0-1t64 (>= 3.24.9); however:
Package libgtkmm-3.0-1t64 is not installed.
```

```
mysql-workbench-community depends on libmysqlclient21 (>= 8.0.11); however:
Package libmysqlclient21 is not installed.
mysql-workbench-community depends on libodbc2 (>= 2.3.1); however:
Package libodbc2 is not installed.
mysql-workbench-community depends on libproj25 (>= 8.2.0); however:
Package libproj25 is not installed.
mysql-workbench-community depends on libsigc++-2.0-0v5 (>= 2.8.0); however:
Package libsigc++-2.0-0v5 is not installed.
mysql-workbench-community depends on libzip4t64 (>= 0.10); however:
Package libzip4t64 is not installed.
```

If you see dependency errors, run the following command to resolve them:

```
sudo apt -f install
```

Once the installation is complete, execute the following command at the shell prompt:

```
mysql-workbench
```

Alternatively, launch MySQL Workbench using the application launcher screen of the GNOME desktop.

26.6 Knowledge test



Click the link below or scan the QR code to test that you know how to install MySQL Workbench:

<https://www.answertopia.com/foh0>



26.7 Reference points

The main points covered in this chapter are as follows:

- **Overview**
 - MySQL Workbench provides a graphical user interface (GUI) for managing MySQL databases.
 - Offers a more intuitive alternative to the MySQL command-line client.
 - Available for Windows, macOS, and Linux.
- **Downloading MySQL Workbench**
 - Download from the MySQL Developer website:
 - <https://dev.mysql.com/downloads/workbench/>
 - Select the appropriate operating system and version before downloading.
- **Windows Installation**
 - Download the MSI installer file.
 - Double-click the MSI file to start installation.
 - Click Next to accept the license agreement.
 - Select the setup type and complete the installation.

Installing MySQL Workbench

- Ensure the 'Launch MySQL Workbench now' option is checked and click Finish.

• macOS Installation

- Download the .dmg file.
- Open the DMG file in Finder.
- Drag MySQL Workbench to the Applications folder.
- Launch Workbench from the Applications folder.
- (Optional) Drag the Workbench icon to Dock for easier access.

• Installation on RPM-based Linux

- Enable required repositories:

```
crb enable
sudo dnf install \
  https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm \
  https://dl.fedoraproject.org/pub/epel/epel-next-release-latest-9.noarch.
rpm
```

- Install dependencies:

```
dnf install libzip unixODBC mesa-libGL-devel gtk2-2.24.33-8.el9proj
```

- Install MySQL Workbench:

```
sudo rpm -ihv mysql-workbench-community-8.0.40-1.el9.x86_64.rpm
```

• Installation on Ubuntu

- If using Ubuntu 24.04, download version 8.0.38 of MySQL Workbench:

```
wget https://cdn.mysql.com/Downloads/MySQLGUITools/mysql-workbench-
community_8.0.38-1ubuntu24.04_amd64.deb
```

- Install the package:

```
sudo dpkg -i mysql-workbench-community_8.0.38-1ubuntu24.04_amd64.deb
```

- If dependency errors occur, resolve them with:

```
sudo apt -f install
```

- Launch Workbench

```
mysql-workbench
```

- Alternatively, open from GNOME Applications Menu.

27. MySQL Workbench Administration

This chapter explains how to use MySQL Workbench to administer and monitor MySQL server instances. MySQL Workbench also offers features for creating and modifying database models, and a SQL Editor environment for interactively executing SQL statements and creating SQL scripts. The next chapter will cover these topics in more detail.

27.1 Creating a server connection

MySQL Workbench uses a tabbed panel approach to presenting information. On the initial startup, only the Home tab shown in Figure 27-1 is displayed:

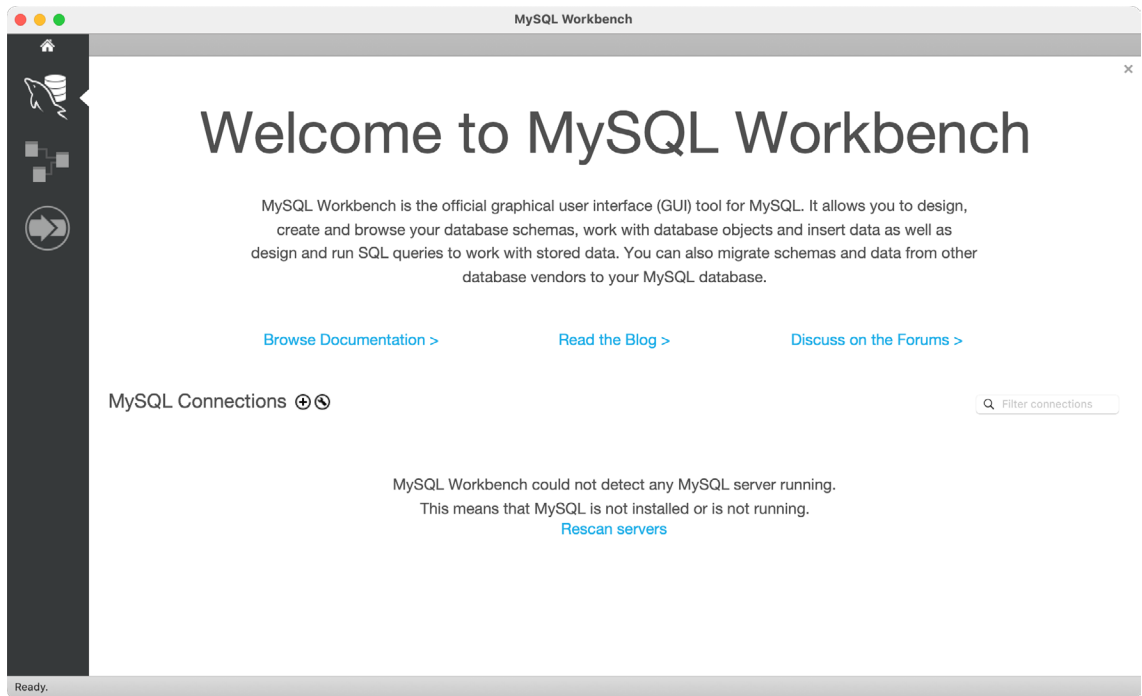


Figure 27-1

MySQL Workbench can administer both local and remote MySQL server instances. Before any administrative tasks can be performed, however, a connection to the target MySQL server must first be established. In the above figure, Workbench has scanned the local system and failed to detect an active MySQL server, so no connections are listed.

If, on the other hand, a local MySQL server is detected, it will be added automatically to the “MySQL Connections” list marked A in Figure 27-2:

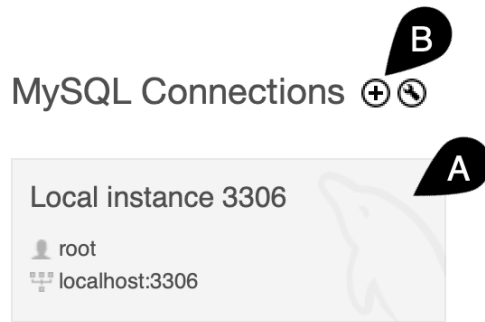


Figure 27-2

To create a new connection, click the + button (marked B above) to display the Setup New Connection dialog shown below:

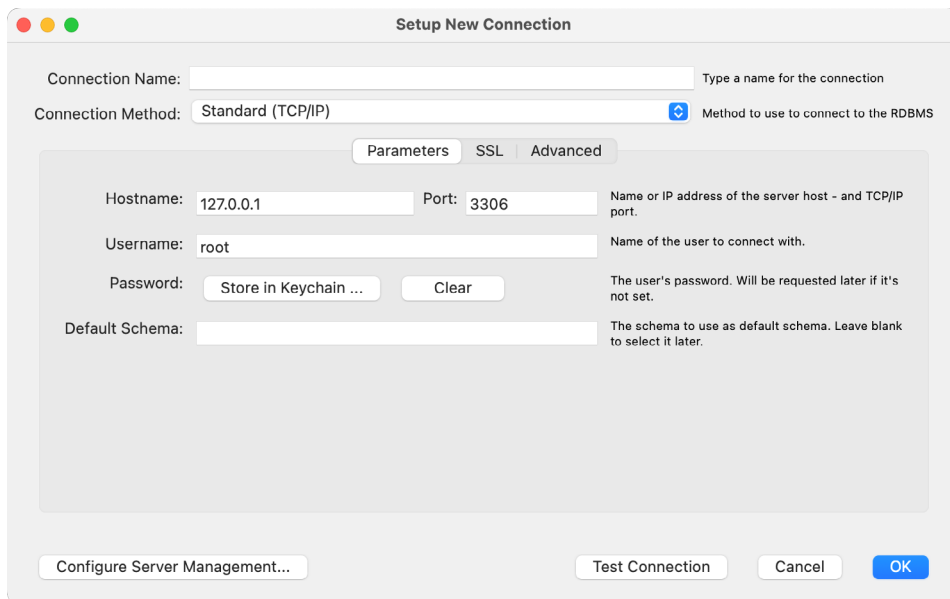


Figure 27-3

The connection dialog is used to create connections to remote and local servers. The minimum configuration requirements to establish a remote connection are as follows:

- **Connection name** - The name by which the connection will be referenced in MySQL Workbench.
- **Connection method** - The network protocol used to communicate with the server.
- **Hostname** - The hostname or IP address of the system on which the MySQL server is running. If the server is running locally, this can be set to 127.0.0.1.
- **Port** - The TCP port on which the MySQL server instance will listen for connection requests. The standard port number for MySQL connections is 3306.
- **Username** - The database user that will be logged into the database when the connection is established.
- **Password** - The database user's password. If the password is not entered in this dialog, Workbench will request

it each time the connection is established.

Note that the database user account specified above must be granted appropriate privileges to connect from a remote host, details of which were covered in the “MySQL Users, Privileges, and Security” chapter.

Once you have entered the connection details, click the Test Connection button to verify that the connection is configured correctly, entering the database user password if prompted:

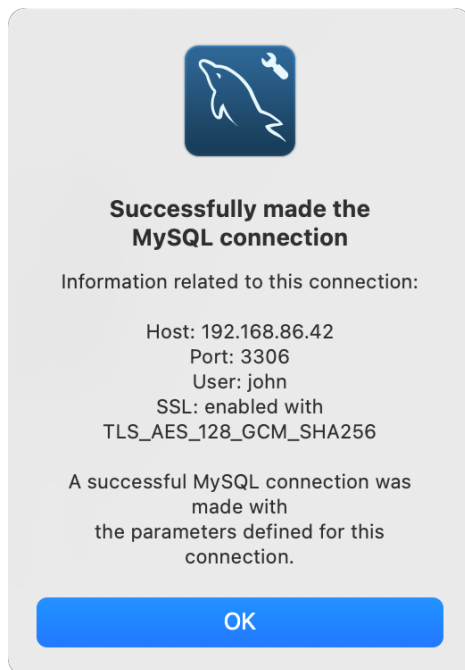


Figure 27-4

If the test is successful, close the result dialog above, then click OK in the Setup New Connection dialog to save the connection. On returning to the home screen, the new connection should be listed as shown in Figure 27-5:

MySQL Connections

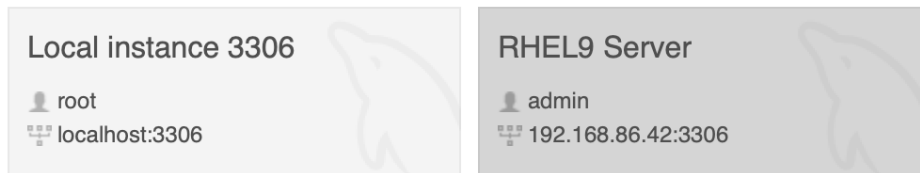


Figure 27-5

To connect to a server, click on the connection icon on the home screen and wait while Workbench communicates with the MySQL server instance.

27.2 The MySQL Workbench user interface.

Once a server connection has been established, the window shown in Figure 27-6 will appear:

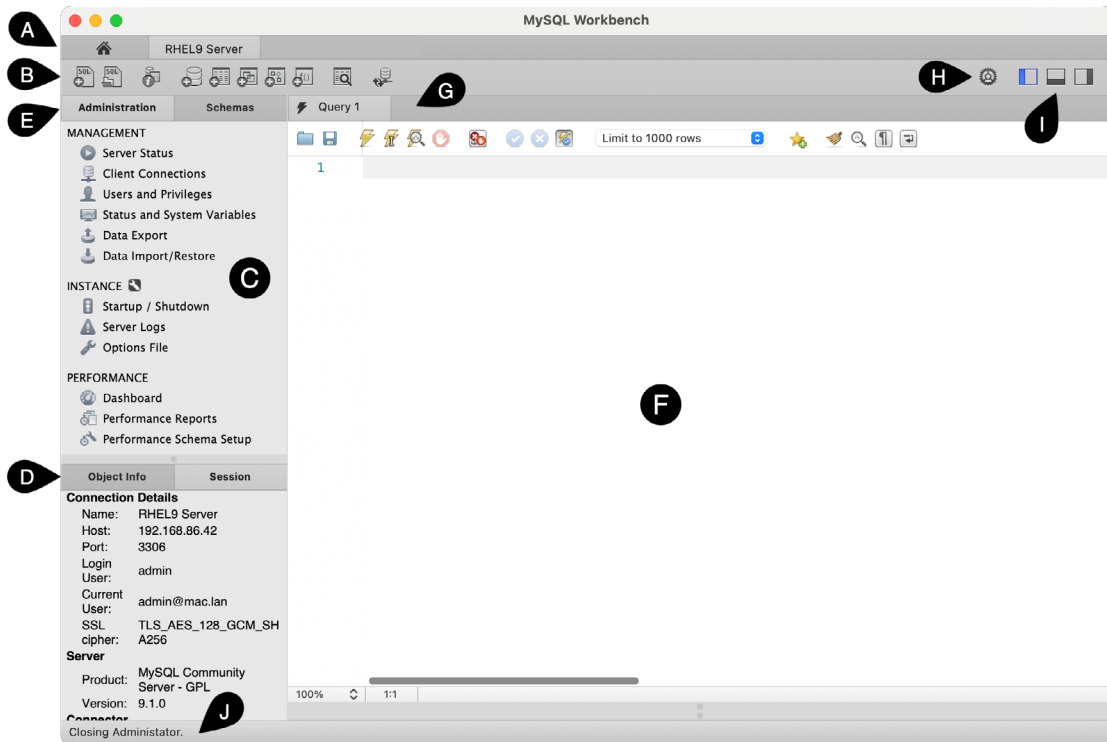


Figure 27-6

The top-level tab bar (marked A above) contains a tab to return to the home screen. MySQL Workbench can maintain concurrent connections to multiple servers, each appearing alongside the Home tab.

The toolbar (B) contains shortcuts to commonly performed tasks, including creating databases, tables, and stored routines and opening new tabs for performing queries. The sidebar panel (C) contains links to panels for administering the currently selected MySQL server. Use tab bar E to switch the sidebar between Administration and Schema views and tab bar D to toggle between information about the currently selected object and connection session.

When a link is selected from the sidebar, a corresponding tabbed panel will be added to the content area (F). Use the tab bar marked G to navigate between open panels.

The settings button (H) displays the Preferences dialog, where you can customize Workbench to your requirements. The panel buttons (I) toggle the appearance of the navigation sidebar, Action Output, and Context Help/Snippets panels.

Finally, the status bar (J) displays status messages reflecting the success or otherwise of Workbench activities.

27.3 Managing servers with MySQL Workbench

The navigation sidebar is divided into Management, Instance, and Performance categories. The management section includes the following options:

27.3.1 Server status

The Server Status panel provides an overall view of the status of the currently connected server. The left-hand section of the panel lists information about the server, such as the version, locations of files and directories, disk

space usage, and the features that have been enabled:

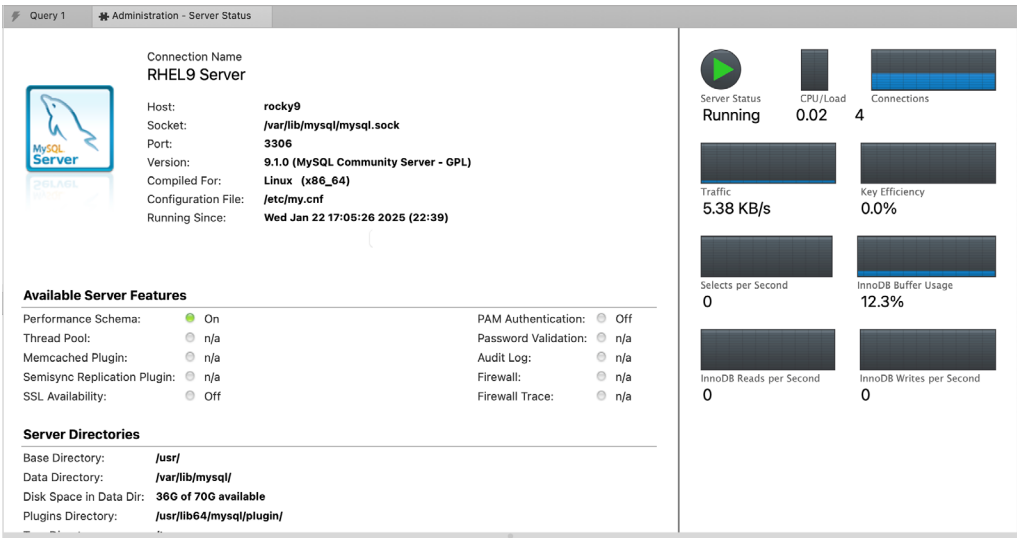


Figure 27-7

The right-hand section displays real-time graphs indicating whether the server is running, the CPU load on the host system, network traffic, and database activity.

27.3.2 Client connections

The Client Connections panel monitors active database connections and allows specific connections or queries to be terminated:

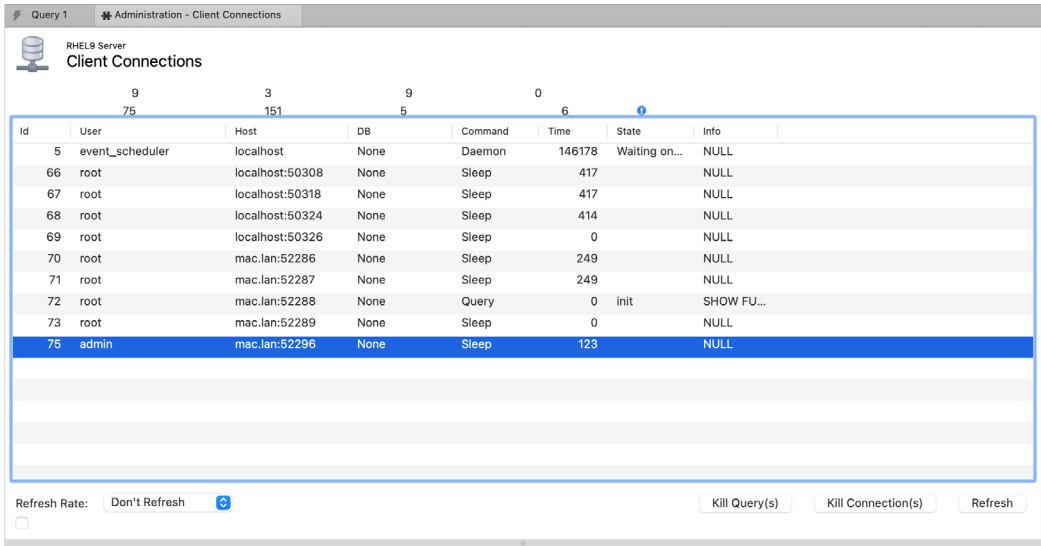


Figure 27-8

To terminate a connection or query, select the connection in the list and click either the Kill Query(s) or Kill Connection(s) button. By default the list must be updated manually using the Refresh button. To configure automatic refreshing, select an interval from the Refresh Rate menu.

27.3.3 Users and privileges

In the “MySQL Users, Privileges, and Security” chapter, we learned how to add and remove users and grant privileges using SQL statements. MySQL Workbench provides a visual alternative for managing users that makes it easy to make changes and view the current settings:

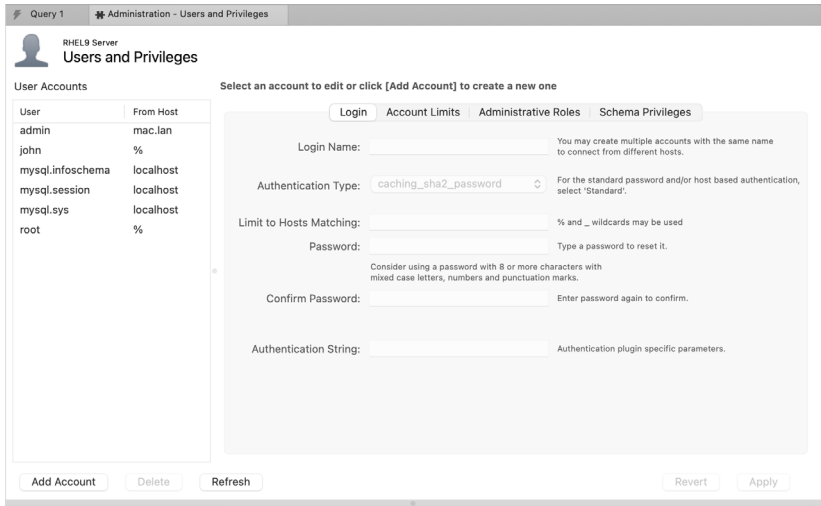


Figure 27-9

The Users and Privileges panel displays all users with database access and allows users to be added and deleted. Viewing and modifying user privileges, including account limits, administrative roles, and table access privileges for individual users, is as simple as turning checkboxes on and off and clicking a few buttons.

27.3.4 Status and system variables

MySQL server instances contain many variables that store state information and define server behavior. The Status and System Variables panel groups these variables into categories that can be browsed and searched:

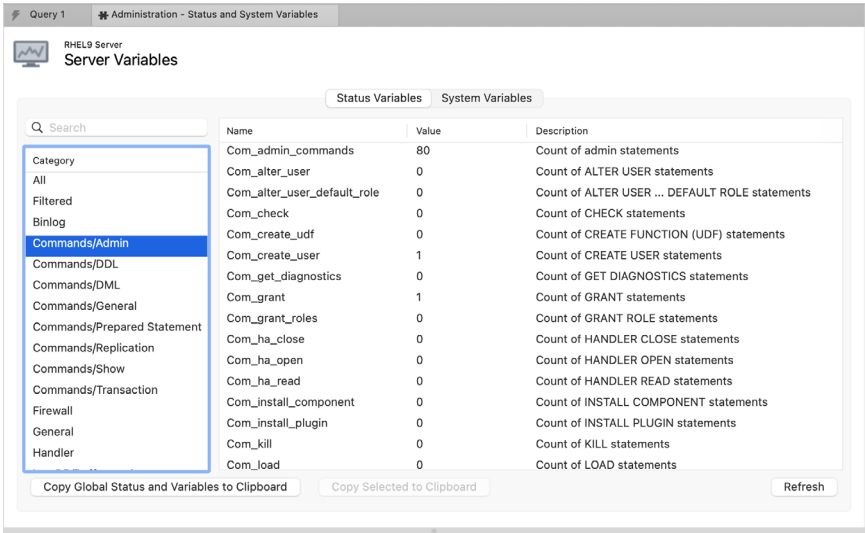


Figure 27-10

27.4 Importing and exporting data

MySQL Workbench provides several options for exporting and importing data. For example, data from specific databases and tables can be exported to dump files and imported into other servers.

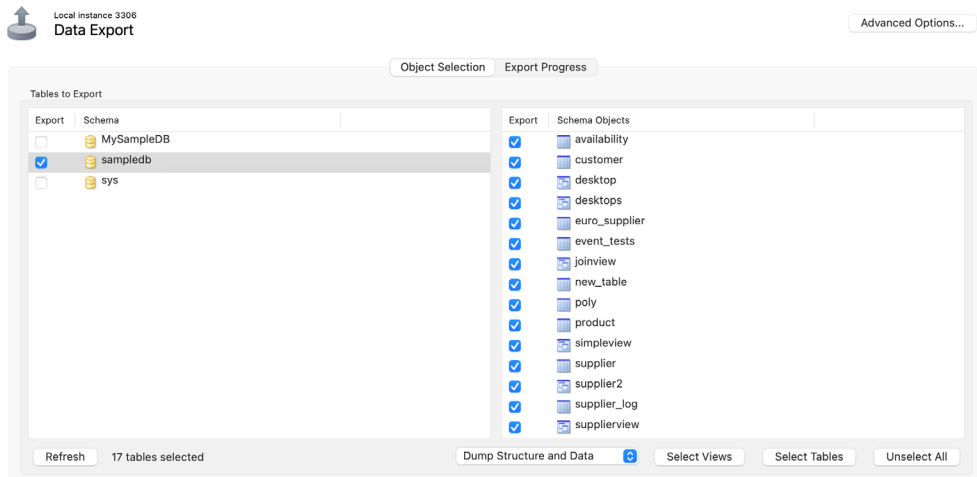


Figure 27-11

27.5 Performance monitoring

The performance dashboard (Figure 27-12) displays real-time statistics relating to network traffic, server status, and database storage engine activity:

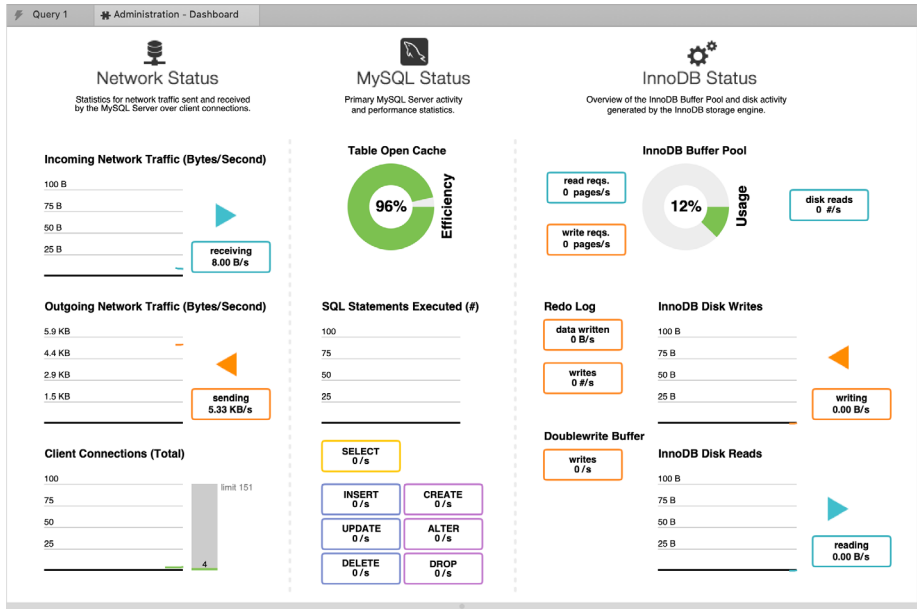


Figure 27-12

Use the Performance Reports option to display fine-grained details on low-level server activity, such as the memory usage for every event, I/O statistics on individual files, or wait times listed by user:

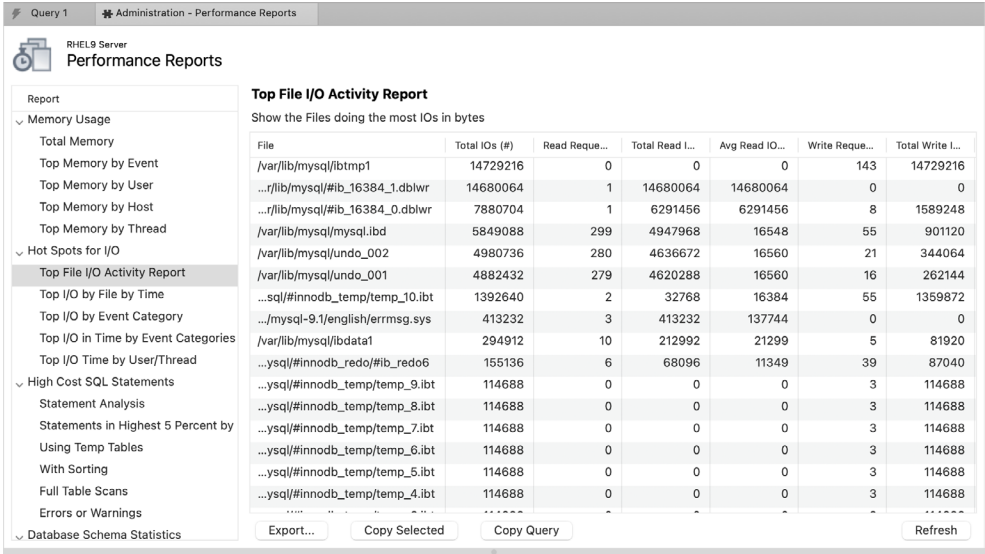


Figure 27-13

27.6 Instance management

The MySQL Workbench features covered so far involve performing tasks while logged into the MySQL server. However, some administrative tasks, such as starting and stopping the server instance, can only be performed outside the MySQL server environment. To perform these tasks, MySQL Workbench needs to know how to log into the operating system on which MySQL is running, the operating system type, and the location of the tools and scripts required to start and shut down the MySQL server instance.

To configure the instance administration features of MySQL Workbench, begin by clicking the settings button next to the instance category title, as indicated in Figure 27-14:

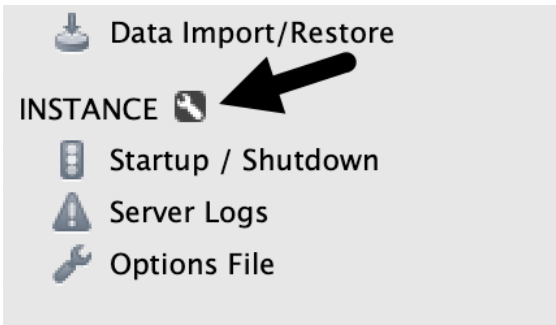


Figure 27-14

When setting up instance management, the first requirement is to provide the details required to log into the system on which the MySQL server is running. Within the Manage Server Settings dialog, select the Remote Management tab and configure the settings to enable Workbench to establish a secure connection to the host system. When specifying a username for the connection, choose a user with adequate permissions to start and stop the MySQL server. For example, if the MySQL server is running on a Linux host, the user account will most likely need sudo permission:

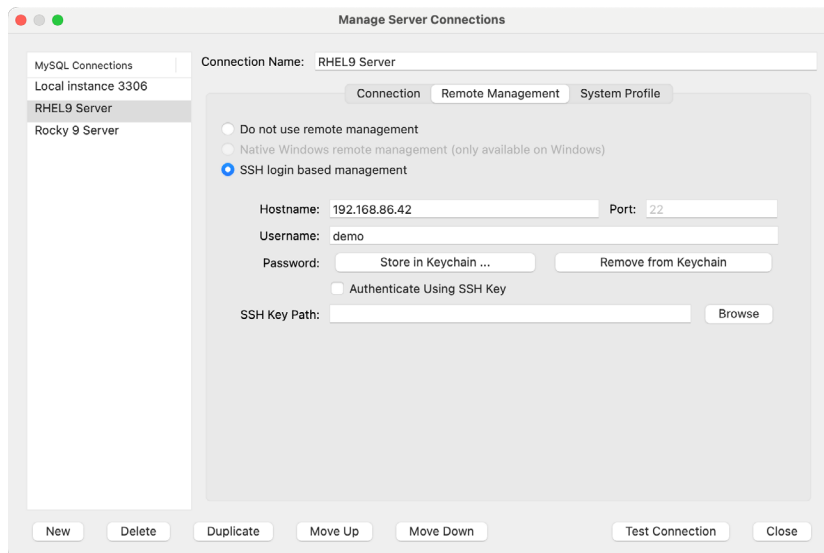


Figure 27-15

After entering the remote management credentials, click the Test Connection button to confirm that MySQL Workbench can access the host system successfully.

The next step is selecting the operating system type so MySQL Workbench knows how to start and stop the server. To configure the operating system details, select the System Profile tab in the Manage Server Connections dialog and set the System Type and Installation Type menus to match the host system:

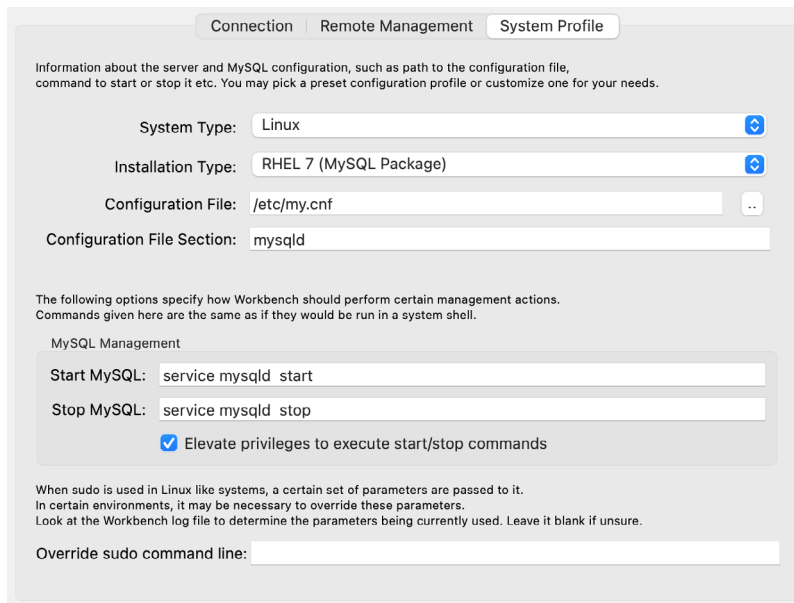


Figure 27-16

When remote management has been configured, select the Startup / Shutdown option in the navigation sidebar and, in the resulting panel, click the Stop Server button (highlighted in Figure 27-17) to shut down the current MySQL server instance:

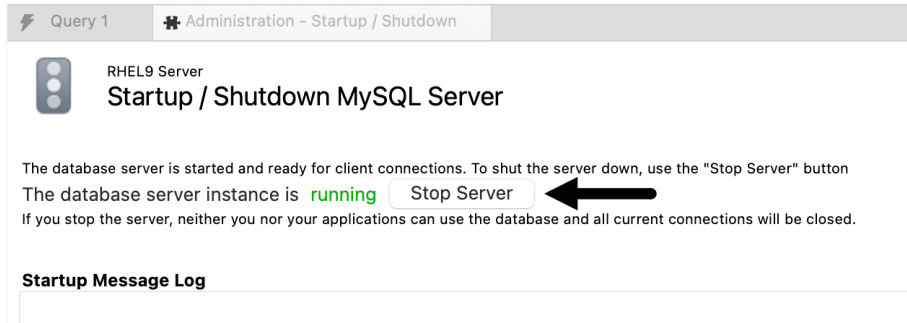


Figure 27-17

Review the output in the Startup Message Log to confirm the shutdown was successful before clicking the Start Server button to restart the server.

27.7 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL Workbench:

<https://www.answerstopia.com/q745>



27.8 Reference points

The main points covered in this chapter are as follows:

- **Overview**

- MySQL Workbench is a powerful GUI tool for managing MySQL databases.
- Supports local and remote MySQL server administration.
- Provides tools for performance monitoring, user management, and database security.
- Allows instance management, including remote server start and stop.

- **Creating a Server Connection**

- MySQL Workbench detects local MySQL servers automatically.
- Remote and local servers can be added manually.
- Required details for new connections include:
 - Connection name, hostname/IP, port (default: 3306), username, and password.
- Saved connections appear on the Home tab for easy access.

- **The MySQL Workbench User Interface**

- Uses a tabbed panel approach for multiple server connections.
- Key UI components:
 - Home tab for managing connections.

- Toolbar for creating databases, tables, and stored procedures.
- Sidebar for Administration and Schema views.
- Status bar for real-time server activity and success/error messages.
- Managing Servers with MySQL Workbench
- **Server Status**
 - Displays MySQL server version, directories, disk usage, and enabled features.
 - Real-time monitoring of CPU usage, network traffic, and database activity.
- **Client Connections**
 - Monitors active database connections and queries.
 - Allows terminating specific queries or connections.
 - Auto-refresh can be enabled to update the connection list.

Users and Privileges

- Provides a graphical interface for managing database users and privileges.
- Users can be added, removed, and granted privileges using checkboxes.
- Allows easy management of administrative roles and table access.

Status and System Variables

- Displays MySQL server system variables and their values.
- Variables are categorized and can be searched for easy navigation.

• **Importing and Exporting Data**

- MySQL Workbench allows exporting databases and tables to dump files.
- Data can be imported into another MySQL server instance from dump files.

• **Performance Monitoring**

- The Performance Dashboard provides real-time network and server statistics.
- Performance Reports offer in-depth analysis of memory, I/O, and wait times.

• **Instance Management**

- Enables starting and stopping MySQL server instances remotely.
- Requires setting up remote management credentials.
- Uses SSH or native OS tools for server control.
- Startup and shutdown logs help track server status.

28. MySQL Workbench Queries and Models

The previous chapters taught us how to install MySQL Workbench and connect to MySQL server instances. We also explored some of its management features, including user and privilege management, data import and export, and performance monitoring. This chapter will focus on executing SQL queries, creating scripts, and working with models in MySQL Workbench.

28.1 Introducing the SQL Editor

Once a connection has been established to a MySQL server, Workbench will open a SQL Editor panel, as shown in Figure 28-1 below:

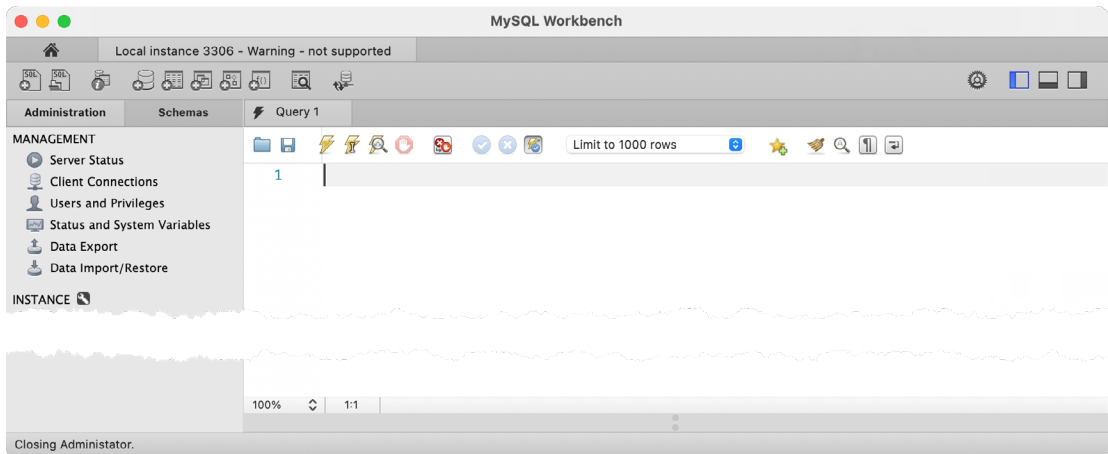


Figure 28-1

Multiple SQL Editor panels may be open simultaneously, each with its own tab within the tab bar location above the editor toolbar. To open new editor sessions, select the *File -> New Query Tab* menu option or click the first button in the main toolbar:

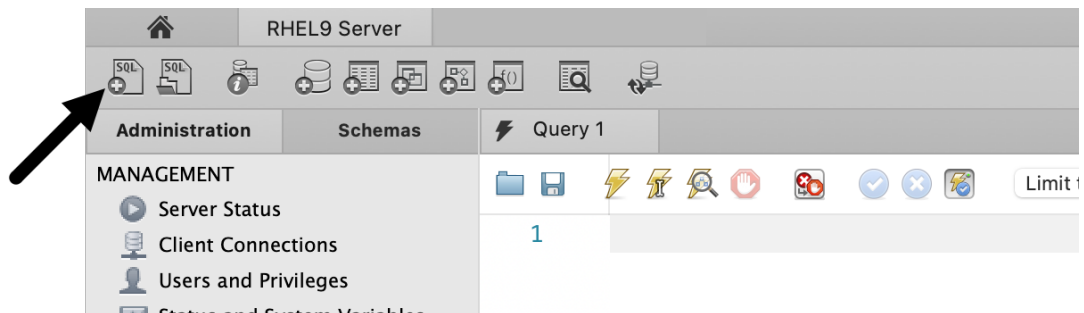


Figure 28-2

MySQL Workbench Queries and Models

The SQL Editor panel allows lines of SQL statements to be entered and executed using the buttons displaying lightning bolts in the toolbar. The leftmost “lightning” button executes either the currently selected statements or all the statements if none are selected, while the second button executes the statement on which the cursor is positioned.

If you have not already done so, start MySQL Workbench and connect to your server instance. Once connected, type the following statements into the “Query 1” editor panel:

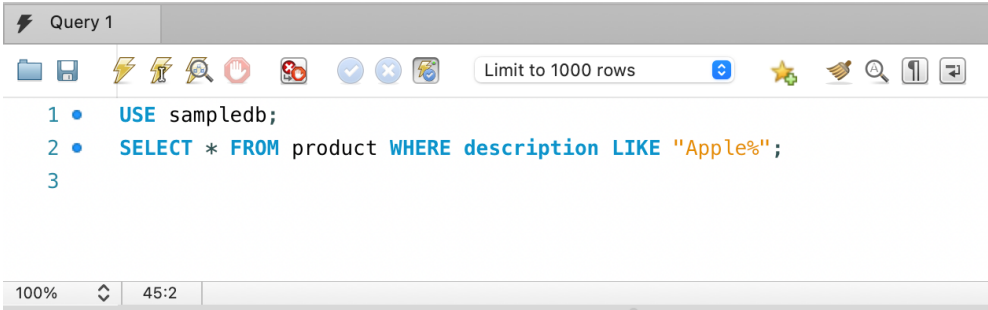


Figure 28-3

Unlike the MySQL client prompt, the USE statement must be terminated (;) when used within the SQL Editor environment. After entering the statements, click the leftmost of the two execute buttons in the toolbar to run the code. After execution, two additional panels will appear with the editor. The Action Output panel (Figure 28-4) records a list of actions executed in the editor, including the action type and the server response:

Action Output					
		Time	Action	Response	Duration / Fetch Time
✓	1	14:36:27	USE sampled	0 row(s) affected	0.018 sec
✓	2	14:36:27	SELECT * FROM pro...	7 row(s) returned	0.011 sec / 0.000013...

Figure 28-4

Click the Action Output menu in the title bar to display text output and action history.

When a query returns a result set, Workbench will also display the Result Grid panel shown below containing the extracted row and column values:

Result Grid						
Filter Rows: Search						
id	name	description	price	quantity	supplier_id	
1	MacBook Pro M4 14-in.	Apple laptop	2499.99	60	1	
2	MacBook Air M3	Apple laptop	1499.99	55	1	
3	Mac Mini M3	Apple desktop	999.99	100	1	
4	Mac Studio	Appledesktop	1999.99	15	1	
7	iPhone 17 Pro	Apple iPhone	999.99	30	1	
8	iPhone 17	Apple iPhone	799.99	30	1	
9	iMac	Apple desktop	1299.99	30	1	
NULL	NULL	NULL	NULL	NULL	NULL	

product 5 Apply Revert

Figure 28-5

The result grid can be used to edit the retrieved data and update the database table. As an example, double-click

on the price column of the Mac Studio product so that it highlights, change the value to 1499.99, and press the enter key:

id	name	description	price	quantity	supplier_id
1	MacBook Pro M4 14-in.	Apple laptop	2499.99	60	1
2	MacBook Air M3	Apple laptop	1499.99	55	1
3	Mac Mini M3	Apple desktop	999.99	100	1
4	Mac Studio	Appledesktop	1499.99	15	1
7	iPhone 17 Pro	Apple iPhone	999.99	30	1
8	iPhone 17	Apple iPhone	799.99	30	1
9	iMac	Apple desktop	1299.99	30	1
NULL	NULL	NULL	NULL	NULL	NULL

Figure 28-6

After making the change, click the Apply button at the bottom of the result grid panel to update the table. The dialog shown in Figure 28-7 will preview the update statement:

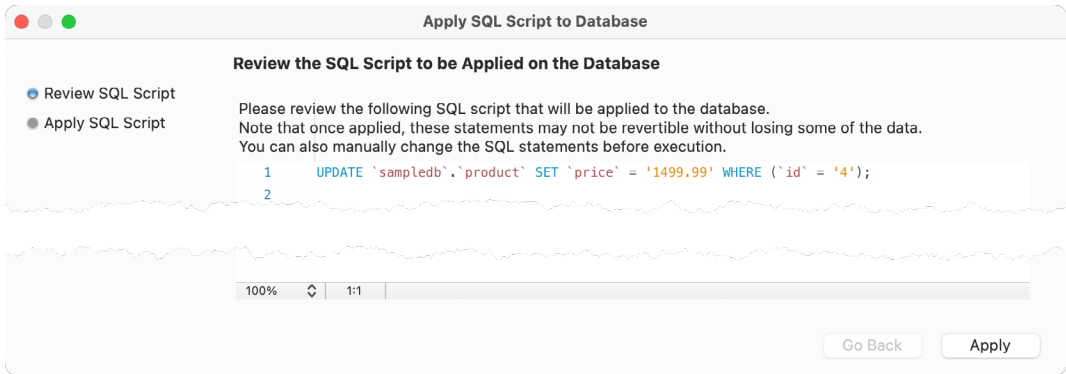


Figure 28-7

Click Apply to perform the update, then return to the result grid and click on the refresh toolbar button indicated below to update the result set with the new price:

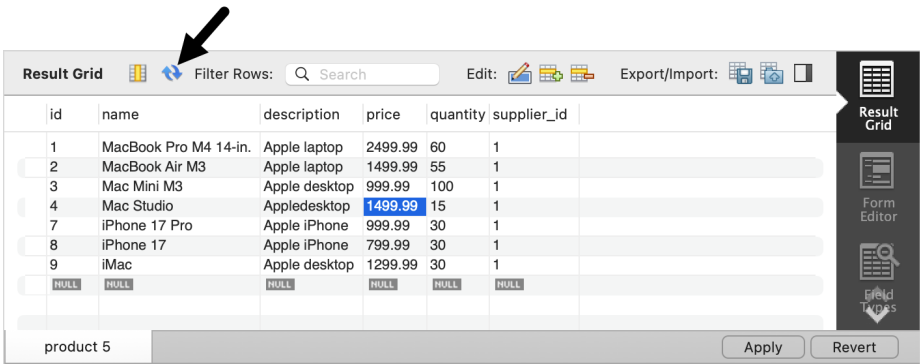


Figure 28-8

The sequence of SQL statements entered into the SQL Query panel may be saved as a script by selecting the *File -> Save Script* menu option or clicking the save button in the toolbar. Saved scripts are loaded into the SQL Editor using the *File -> Open SQL Script...* menu option or the corresponding toolbar button.

28.2 Creating a database model

One of the most powerful features of the MySQL Workbench tool is the ability to design and manage database models. A database model is a blueprint for a database structure, including the schema, users, privileges, scripts, and routines. Once a model has been defined, Workbench generates and executes it on the connected MySQL server to create a physical database. In addition to allowing new models to be designed, the workbench tool also includes reverse engineering capabilities, allowing models to be generated from existing databases.

Start MySQL Workbench and select the *File -> New Model* menu option to create a new model. The model panel will appear with an entry added to the top-level tab bar labeled MySQL Model:

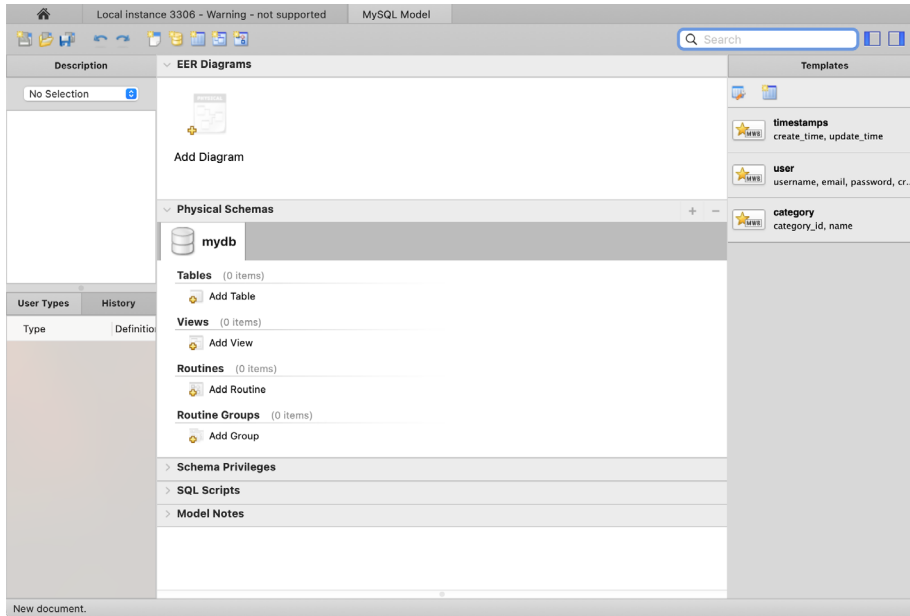


Figure 28-9

An initial schema named `mydb` is created as a basis for the model (keeping in mind that the term schema is synonymous with database). Double-click on the `mydb` tab in the Physical Schemas section and, in the resulting properties panel, change the name of the schema to `orderdb` before pressing the enter key:

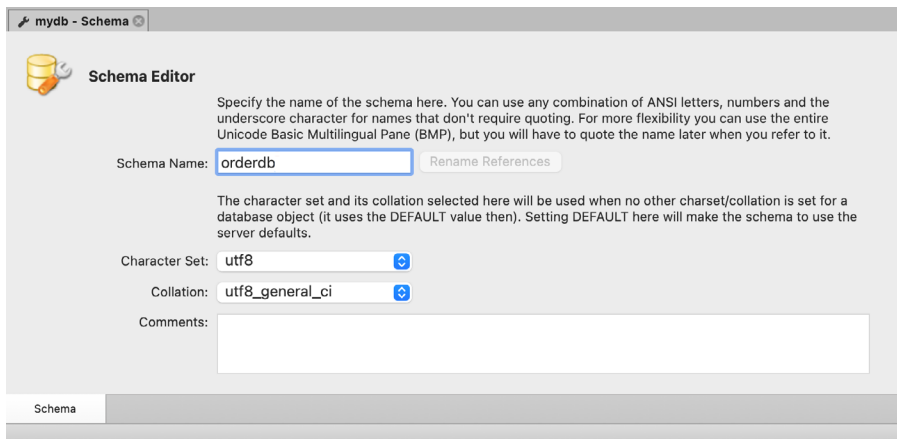


Figure 28-10

Close the schema property panel by clicking the small “x” in the Schema tab. Having given our schema a name, we can now add a table to the model by double-clicking the Add Table button in the Tables section of the Physical Schemas panel. In the table editor, change the table name to order (marked A in Figure 28-11), then select the Columns tab (B) to begin adding columns to the table. Next, click the <click to edit> column entry (C):

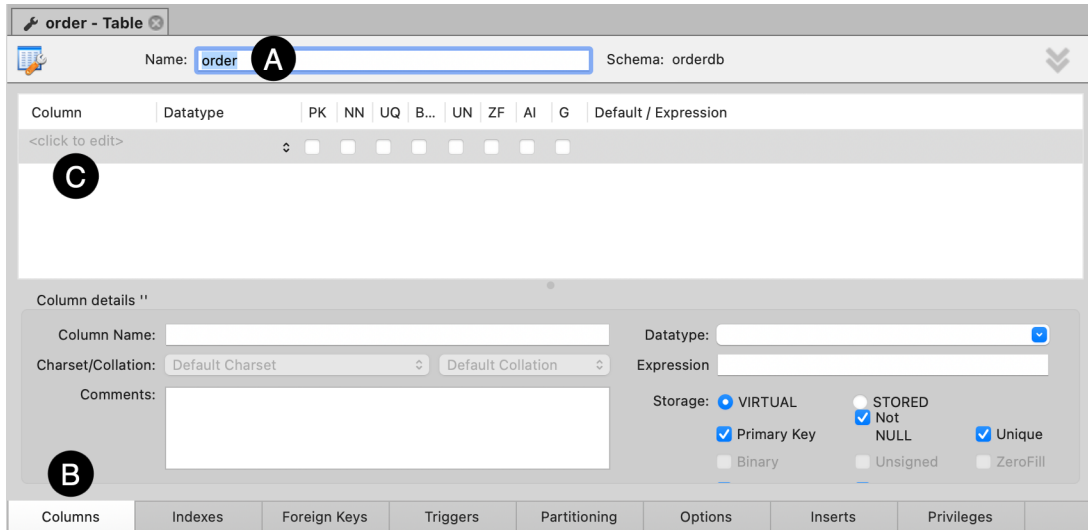


Figure 28-11

By default, Workbench will create a non-null primary key named idorder. For this exercise, we will keep this column, though in practice, this can be changed to meet your requirements:

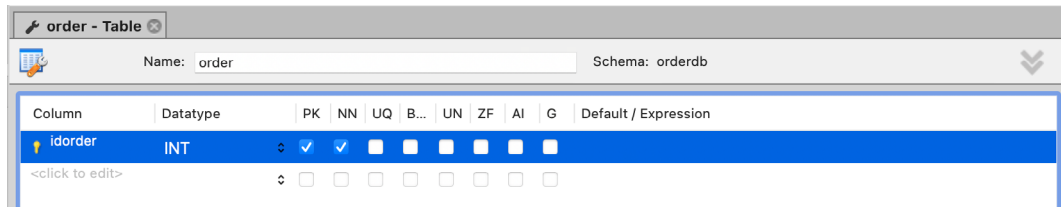


Figure 28-12

Select <click to edit> in the second line, name the column product, and accept the default value of VARCHAR for the data type. Repeat these steps to add a date column of type DATETIME and a quantity column of type INTEGER. Each of these should be declared as non-null by checking the boxes in the NN column:

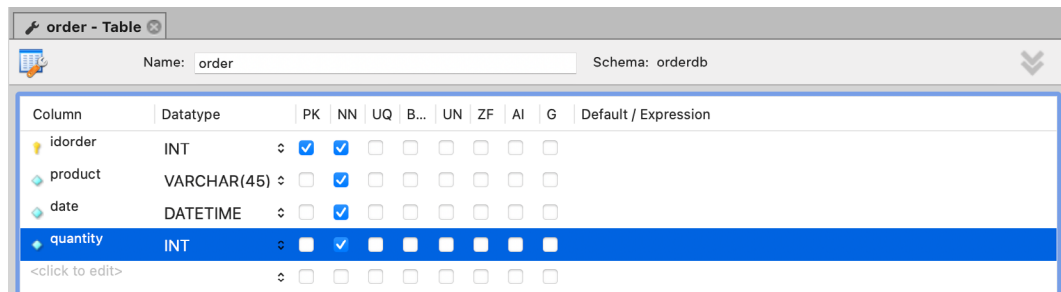


Figure 28-13

Before proceeding, save your progress using the *File -> Save Model* menu option.

28.3 Adding a user to the model

To add a user to the database, unfold the Schema Privileges section of the Physical Schemas panel and double-click on the Add User item:



Figure 28-14

Name the user account `order_admin` in the User panel and enter a password using the fields marked A in the figure below. From the list of available roles (B), select the `table.modify` and `table.insert` roles, using the arrow buttons (C) to assign them to the assigned roles column (D):

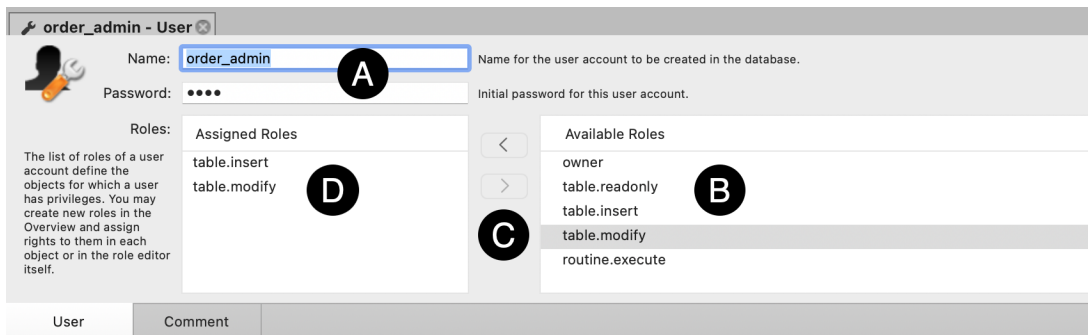


Figure 28-15

Although not required for this example, models may also contain scripts, routines, and views.

Save your progress once again using the *File -> Save Model* menu option.

28.4 Generating the database creation script

Having created a model, the next step is to generate an SQL script from the model to execute on a server to create the database. To achieve this, select the *Database -> Forward Engineer...* menu option. In the resulting dialog, select the server using the Stored Connections menu or enter the server details if they have not previously been saved.

Click Continue to access the database options screen. Unless you have specific requirements, leave the default selections unchanged and continue to the next screen to select the objects to be included in the script. Enable the MySQL Table Objects and MySQL User Objects options before clicking Continue:

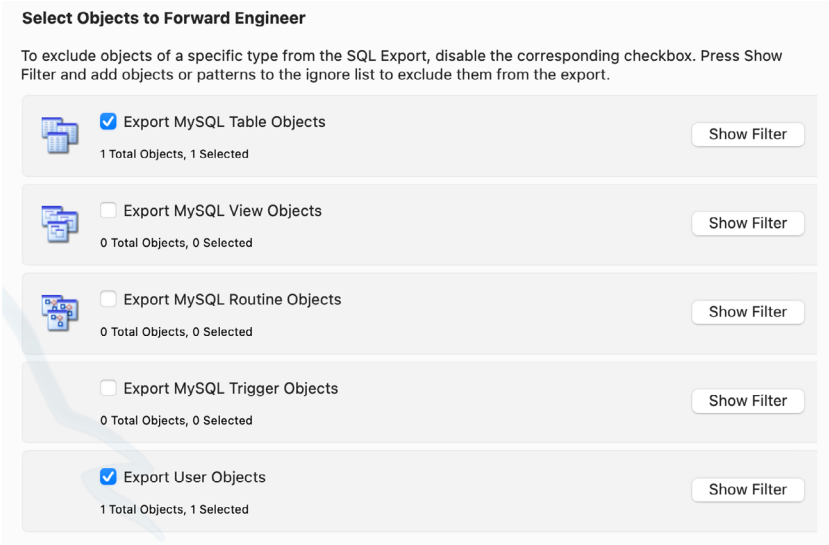


Figure 28-16

The next screen will display the generated script. Review the script to ensure it matches your database requirements:



Figure 28-17

Assuming the script appears as expected given the initial model from which it has been generated, click Continue to execute the script on the destination server or use the *File...* button to save the script if it is to be used later, perhaps within the SQL Editor or MySQL client.

The success or otherwise of the execution will then be reported in the following screen. Assuming a successful operation, the new database will now be present on the designated database server:

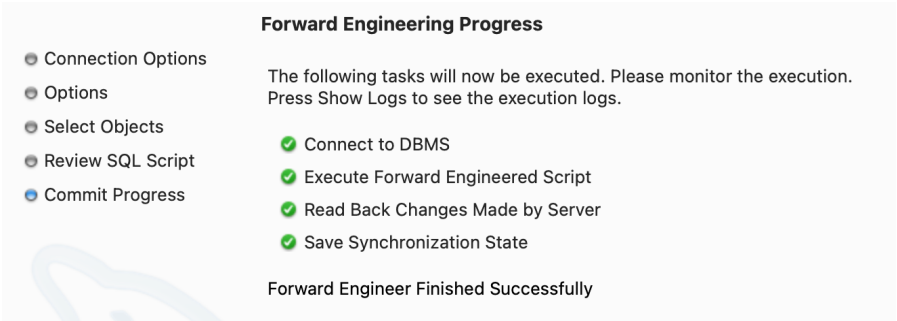


Figure 28-18

Close the Forward Engineering dialog and, using MySQL Workbench SQL editor or MySQL client, log in using the `order_admin` user and verify that the database and table have been created:

```
SHOW COLUMNS FROM orderdb.order;
```

Field	Type	Null	Key	Default	Extra
idorder	int	NO	PRI	NULL	
product	varchar(45)	NO		NULL	
date	datetime	NO		NULL	
quantity	int	NO		NULL	

4 rows in set (0.01 sec)

28.5 Generating EER diagrams

Enhanced entity relationship (EER) diagrams visually represent the relationships between the tables in a database model and are helpful when designing or understanding complex database structures. An EER diagram can be generated from an existing MySQL Workbench model by opening the saved model file and selecting the *Model -> Create Diagram from Catalog Objects* menu option. The diagram generated for our `orderdb` model will appear as shown in Figure 28-19:

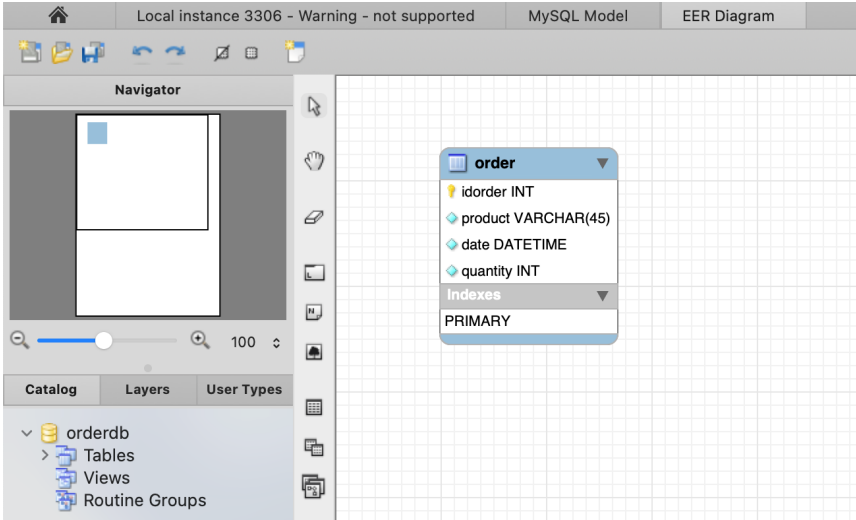


Figure 28-19

In addition to generating EER diagrams from a model file, MySQL Workbench can create diagrams by reverse engineering existing databases. To demonstrate reverse engineering, close the current model and diagram tabs and select the *Database -> Reverse Engineer...* menu option. Step through the screens in the Reverse Engineer Database dialog, providing the credentials to connect to your database server and retrieve the schema list. When prompted, select the *sampledb* database schema as shown below:

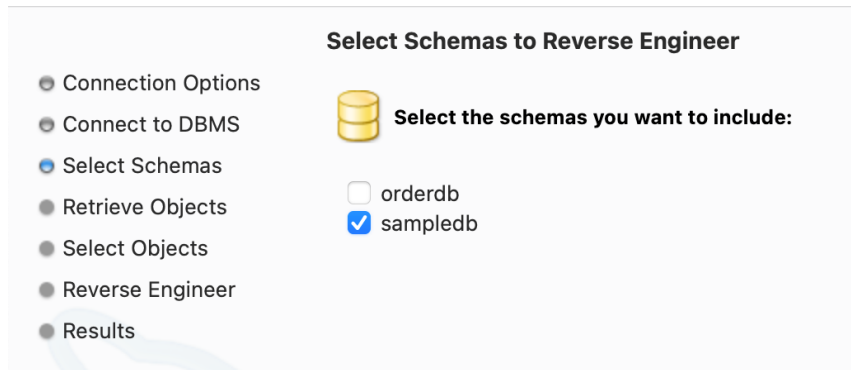


Figure 28-20

Continue through the screens until the object selection options appear. MySQL Workbench can import table, view, routine, and trigger objects. For this exercise, select only the Import MySQL Table Objects option. To avoid having to manually place the tables onto the diagram after it has been generated, make sure that the “Place imported objects on a diagram” checkbox is selected otherwise you will need to manually place the tables on the diagram after it has been generated:

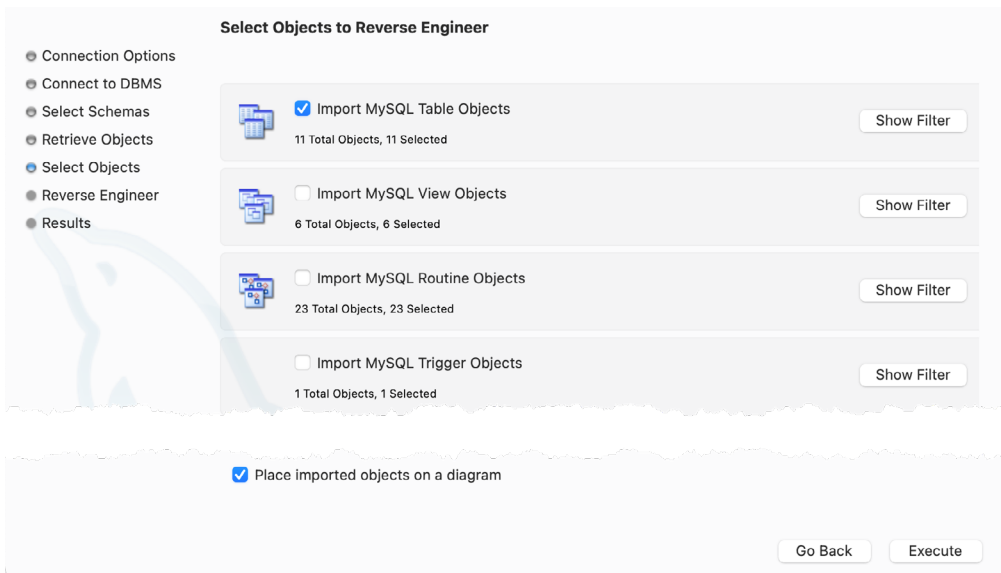


Figure 28-21

After making the object selections, click execute and close the dialog after importing the objects. Once the database has been reverse-engineered, the EER diagram will appear:

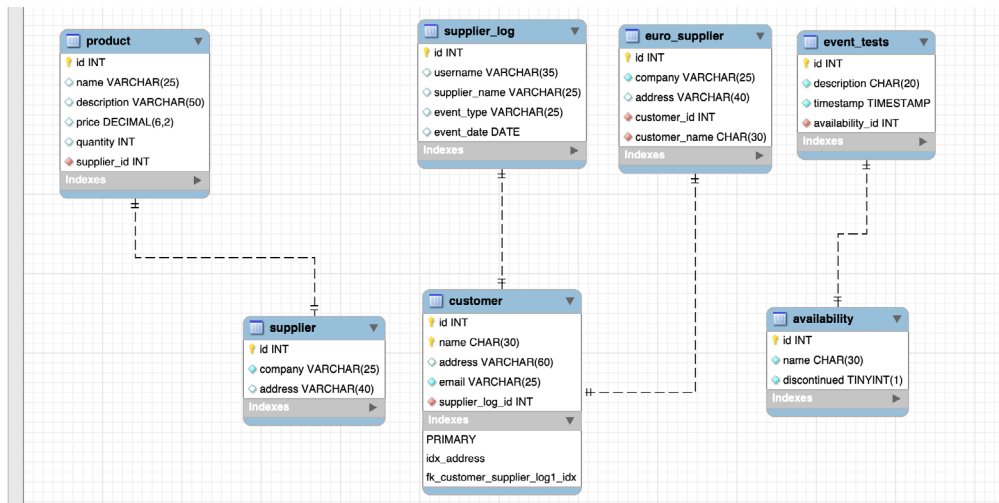


Figure 28-22

The diagram view is an interactive environment where tables can be moved and removed from the diagram and text added to label diagram elements. Double-clicking on a table will display the table editor (outlined in Figure 28-13 above), allowing changes to be made to the model.

In addition to the EER diagram, Workbench has also generated a model file for the reverse-engineered database. The model will appear in a new “MySQL Model” tab where changes can be made and applied to the database using the earlier steps.

Take some time to explore the EER diagram and save the model before exiting MySQL Workbench.

28.6 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of MySQL Workbench queries and models:

<https://www.answerstopia.com/zbst>



28.7 Reference points

The main points covered in this chapter are as follows:

• Overview

- MySQL Workbench allows users to execute SQL queries, create scripts, and manage database models.
- Supports both interactive query execution and structured database modeling.
- Provides tools for database creation, forward engineering, and reverse engineering.

• Introducing the SQL Editor

- SQL Editor is opened when a connection to a MySQL server is established.
- Multiple query tabs can be opened for executing SQL statements.

- Execute queries using toolbar buttons (lightning icons).
- Results are displayed in the Result Grid panel, allowing for direct data modification.
- Changes in the Result Grid must be applied and refreshed manually.
- SQL scripts can be saved and loaded for later execution.
- **Creating a Database Model**
 - MySQL Workbench allows designing database structures using models.
 - Models contain schemas, tables, users, privileges, scripts, and routines.
 - Reverse engineering can generate models from existing databases.
 - New models are created using the *File -> New Model* menu option.
 - Schemas and tables can be defined using a visual editor.
- **Adding Tables and Users**
 - Tables are added to models using the Physical Schemas panel.
 - Columns can be configured with specific data types and constraints.
 - Users can be added with assigned privileges and roles.
- **Generating a Database Creation Script**
 - Models are transformed into SQL scripts using the Forward Engineering feature.
 - Generated scripts can be executed on a MySQL server to create the database structure.
 - Users can review and modify scripts before execution.
 - Successful execution results in a fully configured database.
- **Generating EER Diagrams**
 - Enhanced Entity-Relationship (EER) diagrams provide a visual representation of database models.
 - Workbench can generate diagrams from model files or reverse-engineered databases.
 - Tables and relationships are displayed interactively.
 - Users can modify the diagram layout and make database changes through the diagram interface.
- **Reverse Engineering an Existing Database**
 - MySQL Workbench can generate models from existing databases using reverse engineering.
 - The process retrieves schema objects and generates a model representation.
 - Users can edit the reverse-engineered model and apply changes to the database.

29. Database Administration with phpMyAdmin

So far, we have explored MySQL database administration using the MySQL client command-line environment and graphical user interface-based MySQL Workbench application. A third option, and the topic of this chapter, is a web-based interface. This chapter will introduce phpMyAdmin, one of several popular web-based tools for managing MySQL databases.

29.1 Introducing phpMyAdmin

phpMyAdmin is a free tool that provides a graphical user interface for managing MySQL databases. Unlike the MySQL client and MySQL Workbench tools, which must be installed on each client computer, phpMyAdmin is hosted by a web server typically running on the same system as the MySQL server. This centralization allows the phpMyAdmin interface to be accessed from any web browser without the need to install additional software.

phpMyAdmin is available for several operating systems, including Windows and most Linux distributions, including Red Hat-based distributions, Ubuntu, and Debian. The installation steps on Ubuntu and Debian differ based on the operating system version, so for the purposes of this chapter, we cover installing phpMyAdmin on Red Hat-derived systems.

29.2 Setting up a web server on Red Hat-based Linux distributions

Before installing phpMyAdmin, the first step is to install and configure a web server on the system hosting the MySQL server instance. Open a terminal window, log into the host system, and run the following command to check if the Apache web server is installed and running:

```
sudo systemctl status httpd
```

If the web server is installed and running, the output from `systemctl` will read in part:

```
httpd.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset:
d>
  Drop-In: /usr/lib/systemd/system/httpd.service.d
           └─php-fpm.conf
  Active: active (running) since Mon 2025-01-27 10:50:23 EST; 3h 29min ago
  Docs: man:httpd.service(8)
  Main PID: 251590 (httpd)
```

If the service is reported as inactive (dead), run the following command to start the web server:

```
sudo systemctl start httpd
```

Alternatively, if `systemctl` reports that the `httpd` service is not found, install it using the following command:

```
dnf install httpd
```

After the installation finishes, run the following command to open firewall ports 80 and 443 to allow remote access to the web server. This step is not required if the web browser will be running locally on the server:

```
firewall-cmd --zone=public --permanent --add-service=http  
firewall-cmd --zone=public --permanent --add-service=https
```

Next, start the httpd service as follows:

```
sudo systemctl start httpd
```

If required, the web server can be configured to start automatically when the system boots using the following command:

```
sudo systemctl enable httpd
```

29.3 Testing the web server

If you can access the server's desktop environment, open a web browser and enter `http://127.0.0.1` in the address bar (127.0.0.1 is the loop-back network address that tells the system to connect to the local machine). If everything is set up correctly, the browser should load a test page like the one in Figure 29-1:



Figure 29-1

If the desktop environment is unavailable, connect from another system on the same local network as the server or use the external IP address assigned to the system if it is hosted remotely.

29.4 Installing phpMyAdmin

The easiest way to install phpMyAdmin on Red Hat-based servers (such as CentOS, Fedora, AlmaLinux, and Rocky Linux) is via the Extra Packages for Enterprise Linux (EPEL) repository. If you did not install EPEL in the MySQL Workbench chapter, you can do so now using the following commands:

```
crb enable  
sudo dnf install \  
https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm \  
https://dl.fedoraproject.org/pub/epel/epel-next-release-latest-9.noarch.rpm
```

With the EPEL repository added, run the following command to install phpMyAdmin:

```
sudo dnf install phpmyadmin
```

Once the packages are installed, the next step is to configure the phpMyAdmin security settings.

29.5 Securing phpMyAdmin access

Access to phpMyAdmin is gained by entering a database username and password. By default, phpMyAdmin will only allow connections from browsers running on the local system (in other words, the system on which phpMyAdmin is installed). These security settings can be viewed and modified in the *phpMyAdmin.conf* file located in the */etc/httpd/conf.d* directory and read as follows:

```
<Directory /usr/share/phpMyAdmin/>
    AddDefaultCharset UTF-8

    Require local
</Directory>

<Directory /usr/share/phpMyAdmin/setup/>
    Require local
</Directory>
```

Access is controlled by placing restrictions on phpMyAdmin directories using the standard Apache `<Directory>` directive. These restrictions can be based on various factors, such as IP addresses and subnets. For instance, the following setting permits access from 192.168.86.21:

```
.
.
Require ip 192.168.86.21
.
.
```

Alternatively, the following configuration allows access from all addresses on the specified subnet:

```
.
.
Require ip 192.168.86.0/24
.
.
```

Though not necessarily recommended, the following example removes all access restrictions:

```
.
.
Require all granted
.
.
```

If your web server is accessible to external systems via a public IP address, ensure that your site is secured using SSL (HTTPS) before changing the phpMyAdmin access settings and consider enabling two-factor authentication from within the phpMyAdmin settings screen outlined in the next chapter.

Once the changes to the *phpMyAdmin.conf* file are complete, restart the web server to apply the new configuration:

```
sudo systemctl restart httpd
```

29.6 Opening phpMyAdmin

Test that phpMyAdmin is correctly installed and configured by opening a browser window and navigating to `http://<server>/phpmyadmin`, where `<server>` is replaced with the server's IP address, hostname, or URL.

If the installation was successful, the following page will load:

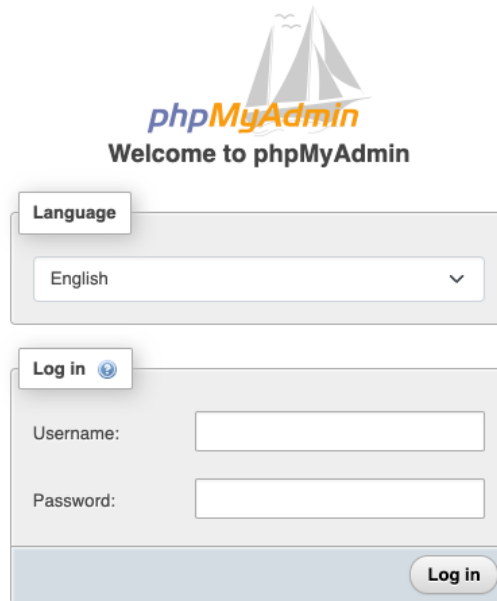


Figure 29-2

Complete the login process by entering the username and password of a database user and clicking the log-in button.

29.7 Knowledge test



Click the link below or scan the QR code to test that you know how to install phpMyAdmin:

<https://www.answertopia.com/3g81>



29.8 Reference points

The main points covered in this chapter are as follows:

• Overview

- phpMyAdmin is a web-based MySQL administration tool.
- It provides a graphical user interface for managing MySQL databases.
- Allows database administrators to manage databases using a browser without installing additional client applications.

• Installation and Setup

- phpMyAdmin requires a web server, typically Apache.
- It can be installed on Linux distributions via package managers such as *dnf* for Red Hat-based systems.
- Security settings can be configured in the *phpMyAdmin.conf* file to restrict access to specific IP addresses.
- The firewall must be configured to allow HTTP and HTTPS traffic for remote access.
- **Security and Access Control**
 - By default, phpMyAdmin restricts access to the local system.
 - Administrators can modify the 'Require' directive in Apache configuration to allow access from other IP addresses.
 - For security, phpMyAdmin should be accessed via HTTPS and should have strong authentication mechanisms.
- **Logging into phpMyAdmin**
 - Access phpMyAdmin via *http://<server>/phpmyadmin* where *<server>* is the IP address or hostname of the MySQL server.
 - Users must enter a valid MySQL username and password to log in.

30. A Guided Tour of phpMyAdmin

With phpMyAdmin installed and accessible in a web browser, this chapter will briefly overview its main features. Topics covered include the phpMyAdmin user interface, the key tools available for database administration, and how to perform tasks such as creating databases, managing tables, executing SQL queries, and importing/exporting data.

30.1 The phpMyAdmin home screen

The phpMyAdmin interface is designed to be intuitive and to make it easy to administer and manage MySQL databases. After signing in, phpMyAdmin will display the home page illustrated in Figure 30-1 below:

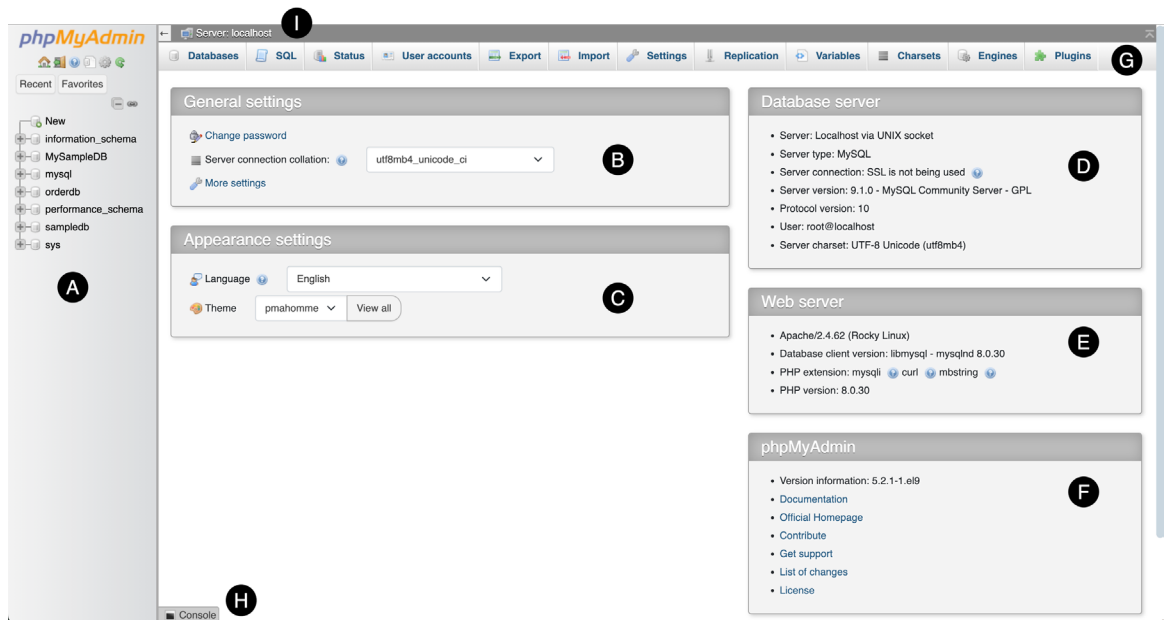


Figure 30-1

The home screen consists of the following areas:

- **A - Navigation panel** - The navigation panel includes a toolbar beneath the phpMyAdmin logo containing quick links to common activities such as returning to the home page, accessing online help, and logging out. The hierarchy tree provides an overview of the databases and tables managed by the MySQL server, including tables, columns, and indexes. Use the + button to unfold sections of the tree for more detail.
- **B - General settings** - Provides access to basic settings such as configuring two-factor authentication, password changes, and the behavior of phpMyAdmin features.
- **C - Appearance settings** - Includes options to change the user interface language and choose from a range of theme options.
- **D - Database server** - Lists information about the database server, including server and connection type,

A Guided Tour of phpMyAdmin

MySQL version, and the current user.

- **E - Web server** - Displays details about the web server on which phpMyAdmin is running, including server type and version and information about the PHP version and extensions.
- **F - phpMyAdmin** - Contains the phpMyAdmin version and a list of useful links for documentation, support, and licensing details.
- **G - Toolbar** - This toolbar contains navigation tabs for all the main phpMyAdmin features. The options are context-sensitive, and change based on the current screen.
- **H - Console** - Displays the console panel where SQL statements can be typed and executed.
- **I - Navigation bar** - Much like a website, phpMyAdmin is structured as a hierarchy of pages. In fact, you can use your web browser's back and forward buttons to navigate through your phpMyAdmin screen history. As you navigate deeper into the phpMyAdmin page hierarchy, the navigation bar maintains a record of your path, allowing you to jump back to specific pages by clicking the links provided.

30.2 Using the console

Clicking the Console button (marked H in Figure 30-1 above) displays a panel containing a command prompt where SQL statements can be typed and executed:



Figure 30-2

Enter statements into the console and press Ctrl+Enter to execute them. To execute the statements as they are entered, select the Options link and enable the option to execute queries on enter. The execution results will appear in the main panel. Figure 30-3, for example, shows the results from running the SHOW DATABASES statement in the console:

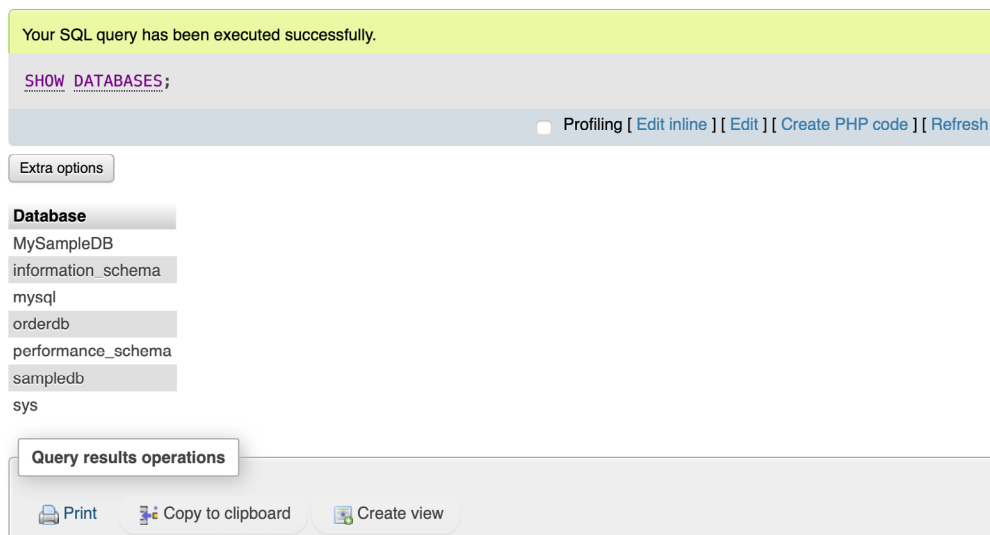


Figure 30-3

30.3 Managing databases

The Databases screen (Figure 30-4) lists the databases managed by the server and allows new ones to be created:

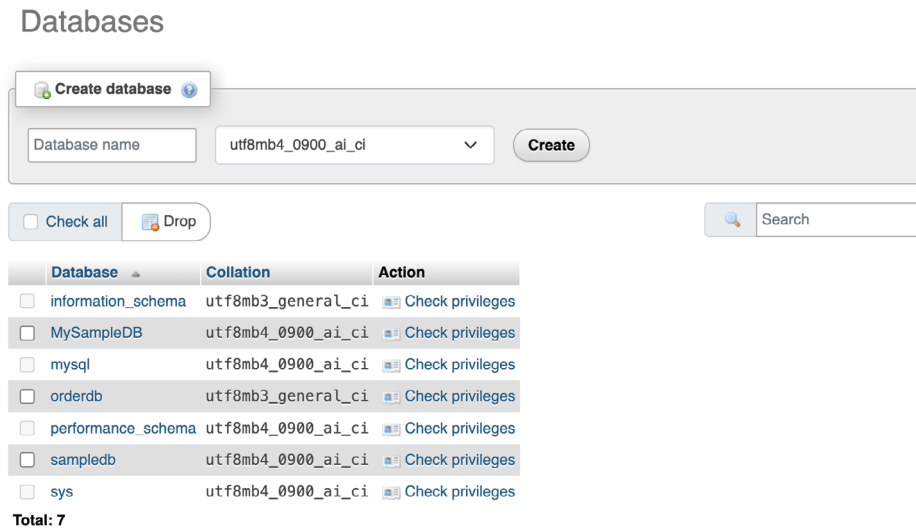


Figure 30-4

Click on a database to navigate to the Structure screen, which contains a list of the tables belonging to the database:

Table	Action	Rows	Type	Collation	Size	Overhead
☐ availability		6	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	—
☐ customer		0	InnoDB	utf8mb4_0900_ai_ci	32.0 KiB	—
☐ desktop		~0	View	---	—	—
☐ desktops		~0	View	---	—	—
☐ euro_supplier		4	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	—
☐ event_tests		67	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	—

Figure 30-5

The structure screen also includes tabs to manage stored routines, events, triggers, and table privileges. To modify the structure of a table, locate it in the list and click on the Structure link to display the table structure editor shown below:

Table structure

Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	int			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	name	char(30)	utf8mb4_0900_ai_ci		No	None			Change Drop More
☐ 3	discontinued	tinyint(1)			No	None			Change Drop More

Check all

With selected:

Browse

Change

Drop

Primary

Unique

Index

Spatial

To review and edit individual table records, return to the Structure page and select the Browse link for a table to display the following screen:

				id	name	discontinued
☐	 Edit	 Copy	 Delete	1	Mac Mini M3	1
☐	 Edit	 Copy	 Delete	2	Mac Studio	0
☐	 Edit	 Copy	 Delete	3	iMac	1
☐	 Edit	 Copy	 Delete	4	iPhone 17	1
☐	 Edit	 Copy	 Delete	5	MacBook Air M3	0
☐	 Edit	 Copy	 Delete	6	iPhone 17 Pro	1

Figure 30-6

30.4 Running SQL queries

In addition to the console, phpMyAdmin includes a SQL editor for entering and executing queries. Selecting the SQL tab in the toolbar will open the editor. Enter SQL statements into the editor and execute them by clicking the Go button:

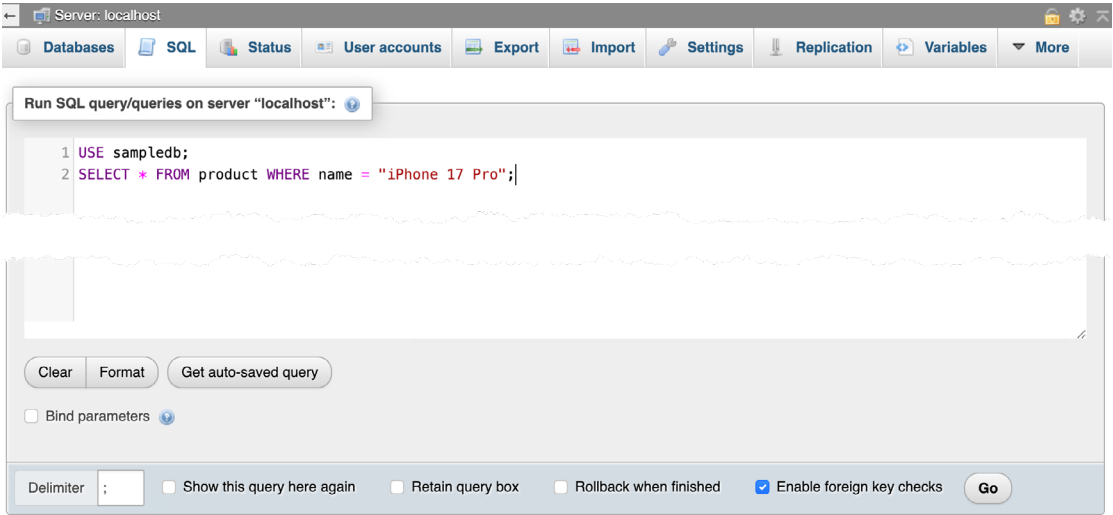


Figure 30-7

30.5 Monitoring server status

The Status page displays information relating to the performance of the MySQL server. Options are available to list running processes, generate query statistics, browse variables, and monitor the server in real time. The state page also includes an advisor feature that identifies performance issues and recommends solutions. Figure 30-8 shows the status page in Monitor mode:

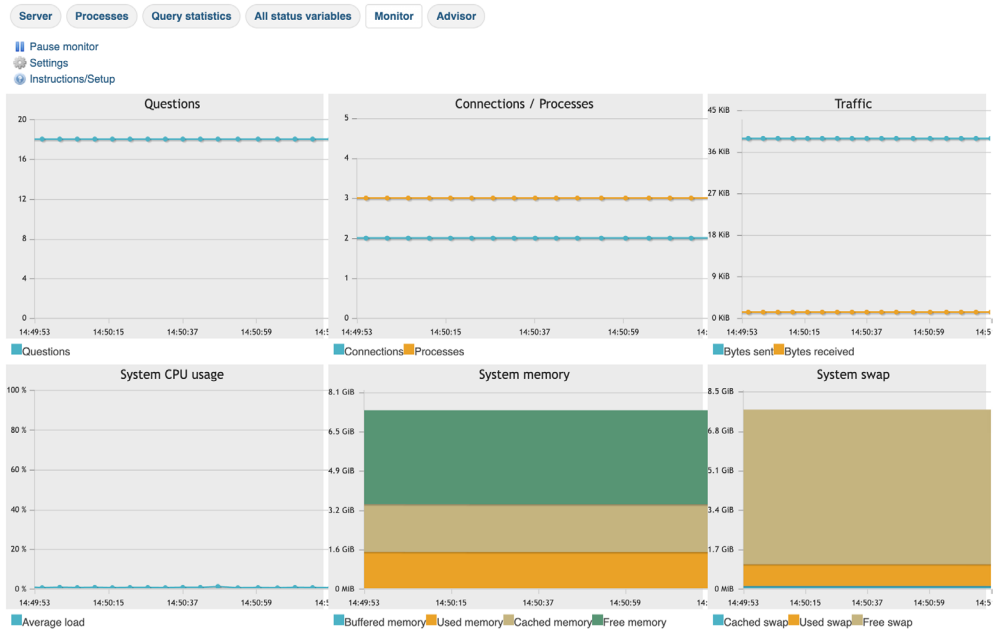


Figure 30-8

30.6 User accounts

As the name suggests, the user accounts screen is where database user management tasks are performed, including adding and deleting users, locking accounts, and modifying privileges:

User accounts overview

User name	Host name	Password	Global privileges	Grant	Action
☐ admin	mac.lan	Yes	USAGE	No	Edit privileges Export Lock
☐ john	%	Yes	USAGE	No	Edit privileges Export Lock
☐ mysql.infoschema	localhost	Yes	SELECT	No	Edit privileges Export Unlock
☐ mysql.session	localhost	Yes	SHUTDOWN, SUPER	No	Edit privileges Export Unlock
☐ mysql.sys	localhost	Yes	USAGE	No	Edit privileges Export Unlock
☐ root	%	Yes	ALL PRIVILEGES	Yes	Edit privileges Export Lock

Figure 30-9

30.7 Importing and exporting data

The import and export screens save database data to files that can be imported on other servers. Options are available to save all the server's databases or make custom selections based on individual databases and tables.

30.8 Working with database diagrams

phpMyAdmin includes a database designer tool that displays database relationships in diagram form and allows new tables and relationships to be created. To access the designer, select a database from the navigation panel (marked A in Figure 30-1 above). When the Structure screen appears, select the Designer tab from the toolbar to display the database diagram:

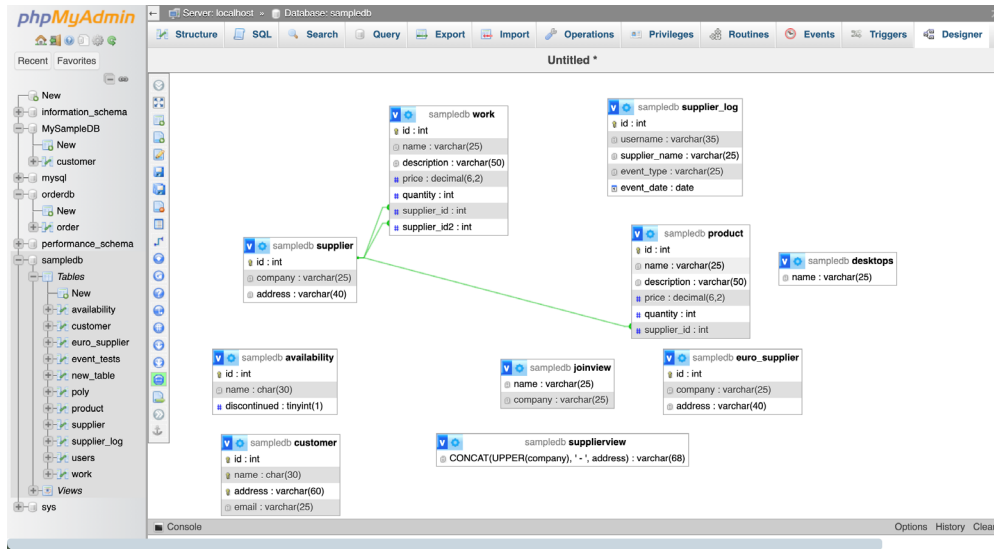


Figure 30-10

Click and drag items to move them, and use the vertical toolbar on the left-hand edge of the diagram panel to create and edit tables and establish new relationships.

This chapter has covered many phpMyAdmin features, but in this book's context, it is impossible to cover them all in detail. Now that you are familiar with the user interface, take some time to explore phpMyAdmin.

30.9 Knowledge test



Click the link below or scan the QR code to test your knowledge and understanding of phpMyAdmin:

<https://www.answertopia.com/pt8w>



30.10 Reference points

The main points covered in this chapter are as follows:

• Home Screen Overview

- Navigation panel provides access to databases and tables.
- General and appearance settings allow customization of the interface.
- Displays database server, web server, and phpMyAdmin version details.
- The console allows SQL queries to be executed interactively.

• Managing Databases and Tables

- The Databases screen lists all databases managed by the MySQL server.
- Users can create new databases and modify table structures.
- The table editor allows changes to columns, indexes, and constraints.

- Records in tables can be browsed and edited from the Structure page.
- **Running SQL Queries**
 - SQL queries can be executed using the console or the SQL editor.
 - The SQL editor provides a structured environment to enter and run queries.
 - Results from queries are displayed in real time within the interface.
- **Monitoring MySQL Server Performance**
 - The Status page displays real-time server performance statistics.
 - Users can monitor running processes, query statistics, and database activity.
 - The performance advisor suggests optimizations based on server activity.
- **Managing User Accounts and Privileges**
 - User accounts can be created, modified, or deleted through phpMyAdmin.
 - Privileges and access levels can be configured for individual users.
 - Users can be locked or restricted from certain databases and tables.
- **Importing and Exporting Data**
 - phpMyAdmin supports importing and exporting databases in various formats.
 - Databases can be backed up and restored using export and import features.
 - Options exist to export entire databases or specific tables selectively.
- **Database Diagrams and Schema Management**
 - The Designer tool visually represents database relationships.
 - Users can create new tables and establish foreign key relationships graphically.
 - Database structures can be adjusted through drag-and-drop functionality.

Index

Symbols

@ 146
 % 102
 < 92
 <= 92
 <> 92
 != 92
 = 92
 >= 92
 \G delimiter 39

A

ADD 67
 AFTER 68, 181
 AGAINST 138
 aggregate functions 86
 column alias 88
 ALTER EVENT 192
 ALTER TABLE 67, 115
 ALTER USER 45
 AND 96
 ARCHIVE storage engine 57
 AS 88
 AT 188
 auto-commit 76
 autocommit variable 77
 AUTO_INCREMENT 55, 62

B

BEFORE 181
 BEGIN 78, 145
 BETWEEN AND 92
 bind-address 204

C

CASE 166
 WHEN clause 167
 CHANGE 69
 COALESCE() 123
 column
 ADD 67
 adding 67
 changing data type 70
 deleting 70
 modifying 69
 renaming 69
 columns 4
 data types 4
 COMMIT 78
 Common Table Expressions 153
 comparison operators 92
 composite index 139
 CONCAT() 115
 CONTAINS SQL 148
 CONTINUE 172
 Control flow 157
 CASE statement 166
 conditional logic 165
 ELSEIF clause 165
 IF statement 165
 LEAVE clause 160
 looping 157
 LOOP statement 160
 REPEAT loop 159
 WHEN clause 167
 WHILE loop 157
 CREATE DATABASE 53
 CREATE EVENT 188, 189
 CREATE FUNCTION 148
 CREATE HANDLER 171
 CREATE INDEX 138
 CREATE OR REPLACE VIEW 134
 CREATE PROCEDURE 145
 CREATE SCHEMA 53

Index

CREATE SPATIAL INDEX 140
CREATE TABLE 54, 71
CREATE TRIGGER 181
 syntax 181
CREATE UNIQUE INDEX 138
CREATE USER 44
CREATE VIEW 127, 130
CROSS JOIN 116
CSV storage engine 57
CURRENT_TIMESTAMP 189

D

Database

- creating 53
- data updates 75
- deleting 56
- schema 4
- server 4
- tables 4

Database Management System 3

Databases

- basics 3
- client-server 3
- vs. DBMS 3

database server 4

Data directory

- Windows 9

data insertion 61

data types 4

DAY 189

DBMS 3

DECIMAL data type 68

DECLARE HANDLER 172

DEFAULT 56

default_storage_engine 58, 205

DELETE 78

DESC 85

DESCRIBE 56

DETERMINISTIC 148

DO 188

downloading 19

DROP COLUMN 70

DROP DATABASE 56

DROP EVENT 193

DROP INDEX 141

DROP TABLE 71

- deleting 56

DROP TRIGGER 185

DROP USER 44

DROP VIEW 134

E

EER diagram 234

ELSEIF 165

encryption 54

END 145

ENGINE 58

Enhanced entity relationship diagram 234

Errata 2

Error handling 171

- conditions 171
- custom conditions 171, 177
- handler creation 172
- handlers 171
- NOT FOUND 175
- SIGNAL 177
- SQLEXCEPTION 172
- SQLSTATE 171
- SQLWARNING 175

escaping 105

Events 187

- altering 192
- creating 188
- creation syntax 188
- deleting 193
- event scheduler 187
- event_scheduler variable 187
- introduction 187
- one-time 189
- preserving 191
- recurring 189
- show details 190

Event scheduler

- enabling 187

event_scheduler variable 187
 EVERY 188
 EXIT 172
 expressions 86

F

fields 5
 filtering results 91, 95
 FIRST 68
 FOR EACH ROW 181
 foreign key 113, 115
 full outer join 122
 full-text index 138
 functions 148
 as subqueries 152
 creating 148
 deterministic 149
 interaction clauses 148
 non-deterministic 149

G

GRANT 46
 INSERT 46
 SELECT 46
 grantable levels 46
 GRANTS 46

H

HOUR 189

I

IF 165
 IF NOT EXISTS 53
 IGNORE 76
 IN 97, 102, 146
 Indexes 137
 composite index 139
 deleting 141
 full-text index 138
 limitations of 141
 non-unique index 138
 primary key 137

regular index 138
 secondary index 138
 showing 140
 spatial index 140
 unique index 137
 inner join 117
 InnoDB 57
 innodb_buffer_pool_size 204
 innodb_file_per_table 205
 innodb_flush_log_at_trx_commit 205
 innodb_log_file_size 205
 INOUT 146
 INSERT 55, 61
 LOW_PRIORITY 65
 using SELECT 64
 VALUES 61
 installation 19
 Installation
 Linux 29
 macOS 19
 Windows 7
 INT data type 70
 INTERVAL 189
 IS 93

J

Joins 113
 and views 130
 cross join 116
 foreign key 113
 full outer join 122
 inner join 117
 left join 118
 right join 119

K

Keys
 primary 5

L

LAST_INSERT_ID() 55
 launch control 24

Index

launchctl

load

unload 24

launchd 21

LEAVE 160

left join 118

LIKE 72, 101

LIMIT 83

Linux

apt install 31

Debian installation 31

downloading 29

mysql client 33

mysqld.log file 30

mysql_secure_installation 31

RPM installation 30

starting server 30

stop server 30

Ubuntu installation 31

log_error 205

logical operators 95

long_query_time 205

LOOP 160

LOOP statement 160

LOW_PRIORITY 65

M

macOS

Configuration pane 23

downloading 19

installation 19

Instances pane 23

launchctl 24

load 24

unload 24

launchd support 21

mysql_secure_installation 25

mySQL.server

start 24

stop 24

Preferences pane 22

securing 25

starting server 24

max_connections 204

MEMORY storage engine 57

metacharacters 105

repetition 108

MINUTE 189

MODIFIES SQL DATA 148

MODIFY 70

MODIFY COLUMN 69

MONTH 189

Monty Widenius 2

MSI file 7

my.cnf 203

applying changes 205

[client] section 204

debugging options 205

file location 203

logging options 205

[mysqld] section 204

[mysqldump] section 204

overview 203

performance tuning 204

security settings 205

server settings 204

storage engine settings 205

structure 203

my.ini. *See* my.cnf

MySQL

Community Edition 2

Enterprise Edition 2

MySQL AB 1

mysqladmin 197

database creation 199

database deletion 199

log flushing 199

password management 198

privilege flushing 199

remote server connections 199

server shutdown 198

server status 198

server variables 199

syntax 197

- mysql client 37
 - case-sensitivity 39
 - connecting to server 37
 - exit 40
 - keyboard shortcuts 39
 - quit 40
 - remote server connection 38
 - starting on Linux 33
 - starting on macOS 26
 - starting on Windows 16
- MySQL Configurator 9
- mysqld.log file 30
- mysql_secure_installation 15, 31, 33
- mysql.server 24
- MySQL Server 8
- mysql.service 32
- mysql.user table 43
- MySQL Workbench 209, 215, 227
 - administration 215
 - client connections 219
 - database models 230
 - data import/export 221
 - downloading 209
 - EER diagrams 234
 - instance management 222
 - Linux RPM installation 211
 - macOS installation 211
 - performance monitoring 221
 - server management 218
 - server status 218
 - SQL Editor 227
 - status variables 220
 - system variables 220
 - Ubuntu installation 212
 - user interface 215
 - users and privileges 220
 - Windows installation 209

N

- NEW 181
- NON DETERMINISTIC 149
- non-unique index 138

- NO SQL 148
- NOT 97
- NOT FOUND 175
- NOT IN 97
- NOT NULL 55
- NULL 54, 62
 - checking for 93

O

- OFFSET 83
- OLD 181
- ON COMPLETION 191
- ON SCHEDULE 188
- operator precedence 96
- OR 95
- Oracle 2
- ORDER BY 84
- OUT 146

P

- password verification 49
- PATH environment variable
 - macOS 23
 - Windows 13
- phpMyadmin
 - database management 247
- phpMyAdmin 239, 245
 - accessing 241
 - console 246
 - database relationship diagrams 249
 - data import/export 249
 - home screen 245
 - installation 240
 - introduction 239
 - opening 242
 - phpMyAdmin.conf 241
 - running SQL queries 248
 - securing 241
 - server status 248
 - user accounts 249
 - web server setup 239
- phpMyAdmin.conf 241

Index

- PID file 23
- port 204
- PRESERVE 191
- Primary index 137
- Primary key 5, 55, 137
 - LAST_INSERT_ID() 55
- PRIMARY KEY 55, 137
- Privileges 43
 - grantable levels 46
 - granting 46
- procedures
 - creating 145
- proxy user 47

Q

- query_cache_size 204
- query_cache_type 204
- Querying 81

R

- READS SQL DATA 148
- records 5
- regex 103
- REGEXP 101, 104
- REGEXP_REPLACE 110
- regular expressions 101, 103
 - character group matching 104
 - character matching 104
 - character ranges 104
 - character type matching 106
 - escaping 105
 - match by text position 109
 - metacharacters 105
 - special characters 105
 - text replacement 110
- regular index 138
- RENAME TABLE 69
- REPEAT 159
- RETURN 148
- RETURNS 148
- right join 119
- ROLLBACK 78

- root
 - removing access 49
- row
 - adding 61
 - adding multiple 63
- rows 5
 - remove all from table 56

S

- schema 4
- secondary index 138
- SELECT 63, 81
 - filtering results 91, 95
 - limiting results 83
 - retrieve single column 81
 - retrieving multiple columns 82
 - sorting 84
- SELECT DISTINCT 84
- SELECT INTO 146
- SET 75
- SET GLOBAL 50
- SET PASSWORD 45
- show all 53
- SHOW COLUMNS FROM 67
- SHOW CREATE EVENT 191
- SHOW DATABASES 53
- SHOW EVENTS 190
- SHOW GRANTS 46
- SHOW INDEX 140
- SHOW TRIGGERS 184
- SHOW WARNING 53
- SIGNAL 177
- skip_name_resolve 204
- slow_query_log 205
- slow_query_log_file 205
- SMALLINT data type 70
- sorting results 84
- spatial index 140
- special characters 105
- SQL
 - overview 5
- SQL Editor 227

SQLEXCEPTION 172
 SQLSTATE 171, 173
 handler conditions 175
 result codes 171
 SQLWARNING 175
 START TRANSACTION 78
 Storage engines 57
 ARCHIVE 57
 CSV 57
 InnoDB 57
 MEMORY 57
 show default engine 58
 Stored routines 145
 functions 145
 procedures 145
 Structured Query Language 5
 subqueries 98
 SUM() 86
 Sun Microsystems 1
 systemctl 30, 32
 start 30
 status 30
 stop 30

T

Table

adding columns 67
 adding rows 63
 altering 67
 changing column type 70
 column renaming 69
 columns 4
 creation 54
 default column vales 56
 deleting 56, 71
 deleting columns 70
 display schema 56
 duplication 71
 encryption 54
 engine selection 58
 fields 5
 foreign key 115

inserting multiple rows 63
 modifying columns 69
 querying 81
 records 5
 renaming 69
 row deletion 78
 row insertion 61
 rows 5
 show columns 67
 updating data 75
 thread_cache_size 205
 TIMESTAMP 189
 format 189
 Transactions 78
 Triggers 181
 accessing data 181
 after insert 182
 before delete 184
 before insert 183
 CREATE TRIGGER 181
 creating 181
 deleting 185
 displaying information 184
 Limitations of 185
 TRUNCATE TABLE 56

U

UNION 120
 UNION ALL 120
 Unions 113, 120
 Unique index 137
 UNIQUE INDEX 138
 UPDATE 75
 UPDATE SET 115
 USAGE ON 46
 USE 54
 user
 adding 44
 deleting 44
 information 43
 passwords 45
 privileges 46

Index

proxy 47

V

Validate Password

configuration 49

installing 49

uninstalling 49

variables 49

VALUES 61

Variables 146

SELECT INTO 146

user-defined 146

Views 127

altering 134

and joins 130

creating 127

deleting 134

introduction to 127

replacing 134

table updates 131

W

WEEK 189

WHEN 167

WHERE 75, 91, 95

comparison operators 92

in join statements 120

logical operators 95

overview 91

WHERE IN 98, 99

subqueries 98

WHILE 157

wildcards 101

multiple-character 102

single character 102

Windows

Dedicated Computer 10

Development Computer 10

downloading 7

installation types 10

installing 7

Manual 10

mysql client 16

MySQL Configurator 9

mysql_secure_installation 15

securing 15

Server Computer 10

Windows service 11

Windows service 11

WITH QUERY EXPANSION 139